

A COMPLETE COURSE ■ FROM NOTHING TO A WORKING KERNEL

Operating Systems from Scratch

Learn how a computer really works by building, step by step, a real 32-bit operating system.

This course assumes no prior knowledge of computer architecture, assembly language or operating systems. It starts by explaining what a processor is, and ends with an interactive shell running in user space, on top of virtual memory, on top of a filesystem written from scratch.

Everything explained here is implemented in the source code that accompanies this document: the **lytlnyblOS** project.

Course material derived from the source code and documentation of the lytlnyblOS project

How to use this course

Who this is for

This course is written for someone who knows **nothing** about operating systems. Nothing about assembly language. Nothing about how a computer starts up. If you can write a C program that prints "Hello", you have enough to begin. If you have also met a pointer without running away, even better.

What it does ask of you is a particular kind of patience. An operating system is a machine of interlocking parts: almost none of them makes complete sense until you have seen the next one. There will be moments where you read “we will cover this in Chapter 24” and have to accept it. That is normal, and it is deliberate — trying to explain everything at once is precisely why most people give up.

What you are going to build

The system studied here is called **lytlnyblOS**. It is deliberately small: around 5,500 lines of C and assembly. But it is *complete* in the way that matters. It boots from nothing on a virtual machine, takes control of the processor, manages memory, runs several programs at once, isolates them from each other, reads and writes a disk through its own filesystem, and leaves you at a command line where you can type `ls /bin` and get an answer.

None of that comes for free. There is no library underneath. There is no Linux underneath. When the system prints a letter on screen, it is because someone wrote a byte to memory address `0xB8000`. When it reads a key, it is because someone programmed the interrupt controller to raise a signal. That “someone” is going to be you, and understanding exactly why each piece sits where it does is the point of this document.

The journey in one sentence

You will go from “the computer powers on and knows how to do nothing” to “an unprivileged user program asks the kernel to print text for it, and the kernel obliges” — understanding every step in between.

How this is organised

The course has thirteen parts plus appendices. The order is **not negotiable**: each part builds on the previous one, exactly as the operating system itself does.

- **Parts I and II** are groundwork: what a processor is, what memory is, what an operating system actually is and which minimum pieces it needs. There is no project code yet, but nothing later stands up without this.
- **Parts III to VII** build a minimal kernel: boot, enter protected mode, print to screen, handle interrupts, and talk to the timer and keyboard. By the end of Part VII you have a system that reacts to the outside world.
- **Parts VIII and IX** are the heart of it: memory management and processes. This is where “a program that runs” becomes “an operating system”.
- **Parts X to XII** open the system up: user space, system calls, disk, filesystem and the shell.
- **Part XIII** is real engineering: how it all compiles, how to debug it, and a complete worked case study of diagnosing a real fault in this project.

The boxes

Five kinds of box appear throughout the text. It is worth knowing what each colour means.

Concept

Blue. A central idea you will reuse many times. If you could only keep one thing from the page, this is it.

An analogy

Purple. A comparison with something from everyday life. Useful for building intuition, but *every analogy lies a little*: when intuition and code disagree, the code wins.

Watch out

Amber. A classic mistake, a subtle trap, or something that costs hours of debugging in practice.

This breaks the system

Red. Serious faults: memory corruption, reboot loops, the kind of thing that does not produce an error message but simply makes nothing work.

Try it yourself

Green. Something you can run or look at right now to confirm that what you just read is true. Use them: reading about operating systems without verifying anything is the best way to believe you have understood something you have not.

What you need installed

All development happens on Linux (on Windows, through WSL). You need five tools:

Tool	Purpose	Why that one
<code>nasm</code>	Assembler	Intel syntax, produces flat binaries with no header
<code>i686-elf-gcc</code>	C compiler	A <i>cross</i> compiler: generates code for a machine with no operating system
<code>ld</code>	Linker	Places each chunk of code at the exact address we want
<code>qemu-system-i386</code>	Emulator	Emulates a whole PC; if the system hangs, you close a window
<code>gdb</code>	Debugger	Lets you stop the emulated processor and inspect its registers

The delicate one is the cross compiler. Your system's `gcc` produces programs for *your* Linux: they expect a standard library, a dynamic loader, and a kernel to provide services. Our operating system has none of that, because it *is* the thing that would provide it. Hence a `gcc` configured for the `i686-elf` target: “32-bit x86 processor, ELF format, no operating system behind it”. Part III explains this in detail.

Check your environment

```
nasm -v
i686-elf-gcc --version
qemu-system-i386 --version
make --version
gdb --version
```

If all five answer, you can run `make run` inside `lytlnybl/` and watch the system boot.

An honest warning about the code

This course explains a *teaching* operating system. That means some decisions were made so the code reads clearly, not so it is robust. At several points you will find red boxes pointing at real limitations of the project: missing validation, memory that is never freed, edge cases that hang the machine.

That is not a flaw in the course — **it is part of the course**. Learning where a real operating system would have to be more careful is as valuable as learning how it behaves when everything goes right. Part XIII collects all of those points and explains how each would be fixed.

Let us start at the real beginning: what a computer is.

Contents

How to use this course	i
I Foundations: the bare machine	1
1 What a computer really is	2
1.1 Three parts and a loop	2
1.2 Memory: a list of numbered boxes	3
1.3 Devices and buses	4
1.4 Why a powered-on computer does nothing	5
2 Binary, hexadecimal and bits	6
2.1 Why hexadecimal	6
2.2 The five bit operations	7
3 Inside the x86 processor	9
3.1 Registers: the processor's own variables	9
3.2 Flags: the invisible result	10
3.3 Rings: why not everyone may do everything	10
3.4 The control registers	11
4 The memory of a PC	13
4.1 The stack	13
4.2 The memory map of a PC	14
5 Talking to hardware	16
5.1 The port functions	16
5.2 Waiting for hardware	16
II What an operating system is	18
6 The minimum pieces	19
6.1 The problem it solves	19

6.2	The seven pieces	20
6.3	Why this order and no other	21
7	Kernel and user space	23
7.1	Two worlds	23
7.2	The only door	23
7.3	The ten services of lytlhybIOS	24
8	Map of the whole system	26
8.1	The journey, end to end	27
8.2	What each directory holds	27
8.3	What the build produces	28
III	Booting	29
9	From the power button to the first instruction	30
9.1	The first few milliseconds	30
9.2	How the boot device is chosen	30
10	The 512 bytes of the bootloader	32
10.1	The whole program	32
10.2	16-bit segmentation: why 0x07C0	33
10.3	Printing without an operating system	33
11	Reading the disk and loading the kernel	35
11.1	Service 0x13	35
11.2	The final jump	36
12	The E820 memory map	38
12.1	The problem	38
12.2	How you ask	38
12.3	The format of each entry	39
12.4	The agreement between bootloader and kernel	40
IV	Protected mode: unlocking the processor	42
13	Why real mode will not do	43
13.1	Three fatal limitations	43
13.2	What you lose	44

14 The GDT: memory's driving licence	45
14.1 The problem it solves	45
14.2 A descriptor, dissected	46
14.3 The lytlmybIOS GDT	46
14.4 Selectors	48
15 The jump to 32 bits	50
15.1 The sequence	50
16 Entering C	53
16.1 C without a standard library	53
16.2 The linker script	54
16.3 And finally, C	55
V Seeing what happens: VGA text mode	57
17 Video memory	58
17.1 An 80 by 25 grid	58
17.2 Computing a cell's address	59
18 The text driver	60
18.1 Terminal state	60
18.2 Initialising and clearing	61
18.3 Writing a string	61
18.4 A divide-by-zero waiting to happen	62
18.5 Printing numbers	63
VI Interrupts: the axis of everything	64
19 What an interrupt is	65
19.1 The problem	65
19.2 Three kinds	65
19.3 What the processor does by itself	66
20 The IDT: the table of handlers	67
20.1 256 entries	67
20.2 The flags byte	68
20.3 Interrupt gate versus trap gate	69

20.4 Loading the table	69
21 Interrupt service routines	70
21.1 Why C is not enough	70
21.2 The two macros	70
21.3 The common stub	71
21.4 The <code>registers_t</code> structure	72
21.5 The C handler	73
22 The PIC: routing hardware interrupts	75
22.1 The chip	75
22.2 Why remapping is mandatory	75
22.3 The remapping sequence	76
22.4 EOI: the acknowledgement you must not forget	77
22.5 Turning interrupts on	78
VII Two drivers: heartbeat and ears	79
23 The timer	80
23.1 Why a timer matters so much	80
23.2 The PIT 8253/8254	80
23.3 What happens on every tick	81
23.4 Waiting without a scheduler	82
24 The keyboard	83
24.1 Scancodes are not characters	83
24.2 Press and release	83
24.3 Modifier keys	84
24.4 From keystroke to blocked process	85
24.5 Special keys	87
VIII Memory management	88
25 The physical memory manager	89
25.1 Pages and frames	89
25.2 A bitmap	90
25.3 Allocating and freeing	90
25.4 Initialisation: deciding what is off limits	91

25.5 The rounding bug	92
26 Virtual memory, slowly	94
26.1 The problem, told as a story	94
26.2 Why a dictionary of every address is impossible	95
26.3 Splitting an address	96
26.4 Why a page is 4096 bytes	99
26.5 The TLB and why stale entries kill	101
26.6 Anatomy of a table entry	102
26.7 A real address space, captured live	102
26.8 The same address in two address spaces	105
27 Paging in practice	107
27.1 Turning it on	107
27.2 Mapping a page	108
27.3 A process's own address space	109
27.4 The address space, laid out	110
28 Page faults	111
28.1 The most useful exception	111
28.2 Two sources of information	111
28.3 Reading a fault	112
29 The kernel heap	114
29.1 Why pages are not enough	114
29.2 The design: a linked list of blocks	114
29.3 Allocating	115
29.4 Growing	116
IX Processes and scheduling	118
30 What a process is	119
30.1 Program versus process	119
30.2 The process control block	120
30.3 The five states	121
30.4 Creating a process	121
31 Context switching	123

31.1	The idea	123
31.2	The code	123
31.3	The scheduler	125
31.4	Destroying a process	126
X	User space	129
32	The TSS and the way back in	130
32.1	The problem	130
32.2	One useful field	130
32.3	What actually happens on entry	132
33	Entering ring 3	133
33.1	There is no “enter user mode” instruction	133
33.2	Building the address space	134
33.3	Loading a program from disk	134
34	System calls	137
34.1	The user side	137
34.2	The kernel side	138
34.3	The dispatcher	138
34.4	sbrk: growing the user heap	139
34.5	The validation that is missing	140
34.6	The full round trip	141
35	A minimal C library	142
35.1	What the library provides	142
35.2	printf with variadic arguments	142
35.3	malloc on top of sbrk	144
35.4	The entry point problem	145
XI	Storage and the filesystem	146
36	Talking to the disk	147
36.1	LBA: one number instead of three	147
36.2	The registers	147
36.3	Reading a sector	148
36.4	Writing	149

37 Designing a filesystem	151
37.1 The problem	151
37.2 The layout	151
37.3 The superblock	152
37.4 Inodes	152
37.5 Directories are files	153
37.6 Path resolution	154
38 File descriptors and the manager	155
38.1 The open file table	155
38.2 Reading across blocks	157
39 mkfs: formatting from outside	159
39.1 The chicken and egg	159
39.2 What it does	160
39.3 The build pipeline	160
XII The shell	162
40 The read-eval-print loop	163
40.1 The whole program	163
40.2 Tokenising	164
40.3 Executing	165
40.4 cat: reading a file	166
40.5 write: the growing buffer	167
40.6 run: starting another program	168
XIII Engineering: building, debugging, improving	170
41 The build system	171
41.1 Two compilers, one project	171
41.2 Finding sources automatically	171
41.3 Pattern rules	172
41.4 The critical link order	172
41.5 From ELF to a disk image	173
41.6 What is missing from this build	173
42 Debugging a kernel	175

42.1 Four techniques	175
42.2 Printing	175
42.3 GDB against QEMU	176
42.4 QEMU tracing	177
42.5 Reading the screen without a screen	177
43 Case study: a reboot loop	179
43.1 The symptom	179
43.2 Step 1: catch the fault instead of the reboot	179
43.3 Step 2: find out where	180
43.4 Step 3: the pattern before the crash	180
43.5 Step 4: what is being divided	181
43.6 Step 5: who could overwrite it	181
43.7 Step 6: confirm by fixing	182
43.8 Why the prebuilt image was fine	182
43.9 The method, generalised	183
44 Known issues and how to fix them	184
44.1 Serious: correctness and safety	184
44.2 Moderate: robustness	185
44.3 Minor: hygiene	185
44.4 A suggested order of work	186
45 Where to go from here	187
45.1 Extensions, by difficulty	187
45.2 A closing thought	188
XIV Appendices	189
A Glossary	190
B x86 quick reference	194
B.1 General-purpose registers	194
B.2 Instructions used in this project	194
B.3 CPU exceptions	195
B.4 Page table entry flags	195
C Memory map of lytlnyblOS	196

C.1	Physical memory	196
C.2	Virtual memory (per process)	196
C.3	Disk image	197
D	System call reference	198
D.1	fsops sub-operations	198
D.2	Reserved descriptors	198
E	Source file index	199
F	Notes on the exercises	200
G	The runnable examples	202
G.1	Running one	202
G.2	How an example changes the system	203
G.3	Checking they still build	203

PART I

Foundations: the bare machine

CHAPTER 1

What a computer really is

By the end of this chapter you will know

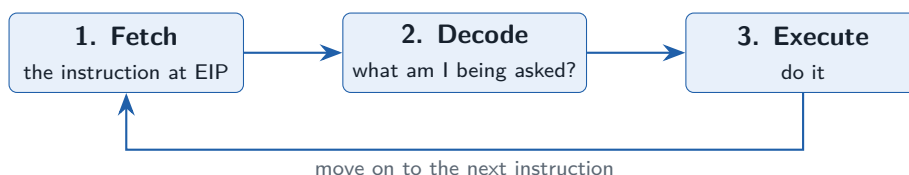
- What a processor does, and why it only ever does one thing
- What memory is, and why it is just a giant list of numbers
- What role buses and devices play
- Why a powered-on computer with no software does absolutely nothing

1.1 Three parts and a loop

Strip away the screen, the keyboard, the case and the marketing. What is left is three things:

1. A **processor** (CPU) that knows how to execute very simple instructions, very fast.
2. A **memory** (RAM), which is an extremely long row of numbered boxes, each holding one small number.
3. Some **devices** (disk, keyboard, video card, timer) connected by wires we call *buses*.

And a loop. The processor always does the same thing, millions of times per second:



That is all. There is no magic, no intelligence, no “the computer decides”. There is a register called **EIP** holding a number: the memory address of the next instruction to execute. The CPU reads whatever is at that address, interprets it as an order, carries it out, and moves on. If the order was “jump to address such-and-such”, **EIP** becomes something else and the loop continues from there.

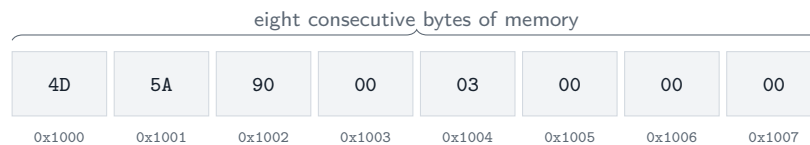
The only question that matters

At any moment, the question that explains a computer’s behaviour is: *what is in **EIP**, and what is in the memory it points at?* All the work an operating system does amounts, in the

end, to controlling the answer to that question.

1.2 Memory: a list of numbered boxes

Picture an infinitely long street with houses down one side. Each house has a number (its **address**) and holds exactly one number between 0 and 255 (a **byte**).



That is literally what RAM is. There are no “variables”, no “objects”, no “strings”. There are boxes with numbers in them. Everything else is an *interpretation* imposed by software.

The same byte 0x41 can be:

- the number 65,
- the letter 'A' if we read it as ASCII text,
- part of the instruction `inc ecx` if the CPU reads it as code,
- a dark grey pixel if the graphics card reads it as a colour.

Data has no type

A byte in memory does not “know” what it is. Type exists only in the programmer’s head and in the code the compiler generates. This apparently philosophical point causes half the faults you will see in this course: when the system interprets data as code, or writes data over code, there is nothing stopping it by default.

1.2.1 Why things come in fours

Our processor is **32-bit**. That means two things at once:

- It works comfortably with 32-bit numbers, that is, 4 bytes. We call that group of 4 bytes a **double word** (*dword*).
- Memory addresses are also 32-bit numbers. With 32 bits you can count up to $2^{32} = 4,294,967,296$. That is why a 32-bit system can address at most **4 GB** of memory. It is not an arbitrary limitation: it is arithmetic.

Byte order (endianness)

x86 is **little-endian**: storing the number 0x12345678 at address 1000 lays the bytes down backwards from how you wrote them — 0x78 at 1000, 0x56 at 1001, 0x34 at 1002, 0x12 at 1003. It takes getting used to, and it confuses people badly when reading memory dumps in

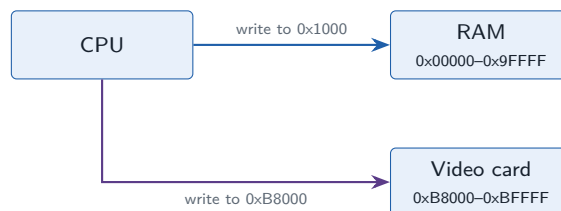
a debugger. It is not a whim: it simplifies the hardware when you only read the low byte of a large number.

1.3 Devices and buses

Processor and memory are not enough: we need to see something on screen and read something from the keyboard. Devices hang off shared wires called **buses**, and the processor talks to them in two distinct ways.

1.3.1 Way 1: memory-mapped

Some devices appear *as if they were memory*. The video card in text mode is the perfect example: write a byte to address `0xB8000` and it is not stored in RAM — it travels to the graphics card and a letter appears in the top-left corner of the screen.



For the programmer this is convenient: writing to the screen is as simple as assigning through a pointer. You will see exactly that in Part V.

1.3.2 Way 2: I/O ports

Other devices live in a *separate* address space, the **I/O port space**. It holds 65,536 numbered mailboxes reachable only through two special processor instructions: `in` (read from a port) and `out` (write to a port).

The keyboard, for example, lives at port `0x60`. The interrupt controller at `0x20` and `0x21`. The disk in the range `0x1F0-0x1F7`.

lytlnybl/kernel/interrupts.asm

```

1  outb:
2      mov al, [esp + 8]    ; the value to send
3      mov dx, [esp + 4]    ; the port number
4      out dx, al          ; send it
5      ret
  
```

Those four lines are the entire mechanism. From C they are wrapped in functions, and from then on “talking to hardware” means calling `outb(port, value)`.

An analogy

Memory-mapped I/O is like leaving a note on someone’s desk: you write it, and the note is

there. Ports are like an intercom: you have to dial the office number (**DX**), say the phrase (**AL**) and press the button (**out**). The effect is not stored anywhere; it simply *happens*.

1.4 Why a powered-on computer does nothing

Here is the question almost nobody asks, and which is the starting point of this whole course: when you press the power button, what does the processor execute?

The CPU wakes up with **EIP** pointing at a fixed address, hard-wired into the silicon. There is no RAM at that address (RAM has just powered on and contains garbage): there is a read-only memory chip holding the **firmware**, which on a classic PC is called the **BIOS**.

In other words: the computer does not “boot itself”. It boots because the manufacturer put a program in a chip, at exactly the address the processor looks at when it wakes. And that program, after checking the hardware, has to find *another* program on disk and hand over to it. And that other program has to load the operating system.

The boot chain

Power on → firmware (BIOS) → boot sector (512 bytes on disk) → operating system kernel → user programs. Each link has exactly one job: leave the next one in a position to start. All of Part III is this chain.

Exercise 1.1. Without reading ahead, reason it out: if the boot sector is exactly 512 bytes, why can it not contain the whole operating system? What is the minimum it would have to do?

CHAPTER 2

Binary, hexadecimal and bits

By the end of this chapter you will know

- Read and write hexadecimal fluently
- Understand the bit operations: AND, OR, XOR, NOT and shifts
- Recognise the `| FLAG`, `& ~FLAG` and `& MASK` idioms that appear all over the code

This chapter is short and purely practical. If you are already comfortable with bits, skim it and move on. If not, read it slowly: without this, 60 % of operating system code looks like noise.

2.1 Why hexadecimal

A byte is 8 bits. In binary a byte reads 11010110: exact but unreadable. In decimal, 214: readable but telling you nothing about the bits.

Hexadecimal (base 16) is the perfect middle ground because **each hex digit corresponds to exactly 4 bits**. A byte is always two hex digits. No more, no less.

Hex	Binary	Decimal	Hex	Binary / Decimal	
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	A	1010	10
3	0011	3	B	1011	11
4	0100	4	C	1100	12
5	0101	5	D	1101	13
6	0110	6	E	1110	14
7	0111	7	F	1111	15

With that table, 0xD6 translates at a glance: D = 1101, 6 = 0110, so 11010110. And back again just as fast.

Round numbers in hexadecimal

0x1000 = 4096 = 4 KB. And that is no coincidence: **4096 bytes is the size of a memory page** on x86. That is why you see addresses like 0xB8000, 0x100000 or 0xC0000000

everywhere: they end in three hex zeros, which means they are page-aligned. When you see an address ending in 000, think “start of a page”.

Some figures worth memorising, because they appear constantly:

Value	Decimal	What it is in this course
0x1000	4,096	one page / one memory frame
0x200	512	one disk sector, and the filesystem block size
0xB8000	753,664	start of video memory in text mode
0x7C00	31,744	where the BIOS loads the boot sector
0x100000	1,048,576	1 MB; end of “low memory”
0x400000	4,194,304	4 MB; where user code starts
0xC0000000	3,221,225,472	3 GB; where the kernel heap starts
0xFFFFFFFF	4,294,967,295	the highest possible 32-bit address

2.2 The five bit operations

2.2.1 AND (&): switching off and asking

`a & b` leaves a 1 only where *both* had a 1. It has two uses:

```

1  /* 1. ASK whether a bit is set */
2  if (status & ATA_STATUS_BSY) { /* the disk is busy */ }
3
4  /* 2. KEEP part of a number (masking) */
5  uint32_t base_address = entry & 0xFFFFF000; /* the top 20 bits */
6  uint16_t table_index  = (virtual >> 12) & 0x3FF; /* 10 bits */

```

The second form is everywhere in paging. `0xFFFFF000` has the top 20 bits set and the low 12 clear: it keeps a page address and throws away the permission bits. `0x3FF` is ten ones: it keeps an index from 0 to 1023.

2.2.2 OR (|): switching on

`a | b` leaves a 1 where *either* had one. This is how a value is built from flags:

```

1  map_page(dir, virtual, physical, PAGE_PRESENT | PAGE_WRITABLE | PAGE_USER);

```

Each constant has exactly one bit set, so `|` combines them without collision. This idiom — define flags as powers of two, combine with `|` — appears dozens of times.

2.2.3 NOT (~) with AND: switching one bit off

```

1  bitmap[i / 32] &= ~(1 << (i % 32)); /* clear bit i */
2  (*table)[index] &= ~(PAGE_PRESENT); /* mark the page as not present */

```

$\sim X$ inverts every bit; ANDing with that clears exactly the bits that were set in X and leaves everything else alone. It is the canonical way to say “remove this flag and touch nothing else”.

2.2.4 Shifts (<< and >>)

Shifting left by n multiplies by 2^n ; shifting right divides. But their most frequent use is *placing* and *extracting* fields:

```
1  /* Placing: build a VGA attribute byte from two 4-bit colours */
2  terminal->color = ((uint8_t)background << 4) | (uint8_t)foreground;
3
4  /* Extracting: split a virtual address into its three fields */
5  uint16_t directory_index = virtual >> 22;          /* bits 31..22 */
6  uint16_t table_index     = (virtual >> 12) & 0x3FF; /* bits 21..12 */
7  uint16_t offset          = virtual & 0xFFF;        /* bits 11..0  */
```

The language of bits

Five idioms cover almost everything: $x \mid F$ sets, $x \&= \sim F$ clears, $x \& F$ asks, $x \& \text{MASK}$ trims, $x \gg n$ extracts. Recognise them on sight and systems code stops looking cryptic overnight.

Exercise 2.1. The physical memory manager keeps one bit per 4 KB page. The function that marks page i as used is `bitmap[i / 32] |= (1 << (i % 32));`. Explain why it divides by 32, and why it takes the remainder modulo 32.

CHAPTER 3

Inside the x86 processor

By the end of this chapter you will know

- What registers are and what each one is for
- What flags are and why conditional jumps exist
- What privilege rings are (ring 0 and ring 3)
- What the control registers `CR0`, `CR2` and `CR3` do

3.1 Registers: the processor's own variables

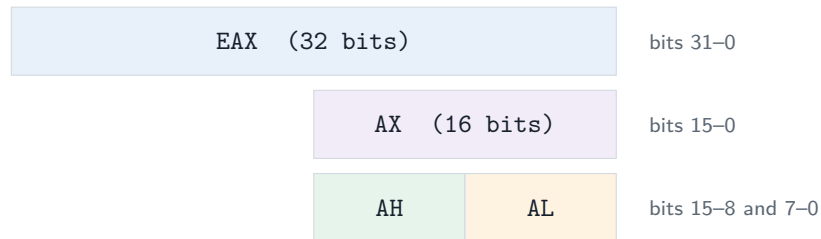
Memory is huge but slow. Inside the chip itself there are a handful of extremely fast boxes: the **registers**. On 32-bit x86 there are eight general-purpose ones, each 32 bits wide.

Register	Name	Typical use
<code>EAX</code>	Accumulator	Operation results; function return value; system call number
<code>EBX</code>	Base	General purpose; first argument in our system calls
<code>ECX</code>	Counter	Loop counter; second argument in our system calls
<code>EDX</code>	Data	I/O operations (holds the port); third argument
<code>ESI</code>	Source	Source pointer in memory copies
<code>EDI</code>	Destination	Destination pointer in memory copies
<code>EBP</code>	Base pointer	Marks the start of the current function's frame
<code>ESP</code>	Stack pointer	Points at the top of the stack. Special: push and pop modify it

The names come from historical convention; apart from `ESP` (and partly `EBP`), the processor lets you use them for whatever you like.

3.1.1 One register, three names

For compatibility with processors from the 1970s and 80s, each register can be used whole or in pieces:



This explains code that is otherwise baffling. In the bootloader you will see:

```

1 mov ah, 0Eh    ; BIOS function: print character
2 mov al, 'H'    ; the character to print
3 int 10h        ; call the BIOS video service

```

Two different values are being loaded into *two halves of the same register* **AX**, because that is the interface the BIOS expects.

3.2 Flags: the invisible result

Every arithmetic or logic operation, besides its result, updates a special register called **EFLAGS**. Each bit answers a question about what just happened:

Flag	Bit	Means
ZF (Zero)	6	The result was exactly zero
CF (Carry)	0	There was a carry, or an error; the BIOS uses it to signal failure
SF (Sign)	7	The result is negative
IF (Interrupt)	9	Interrupts are enabled

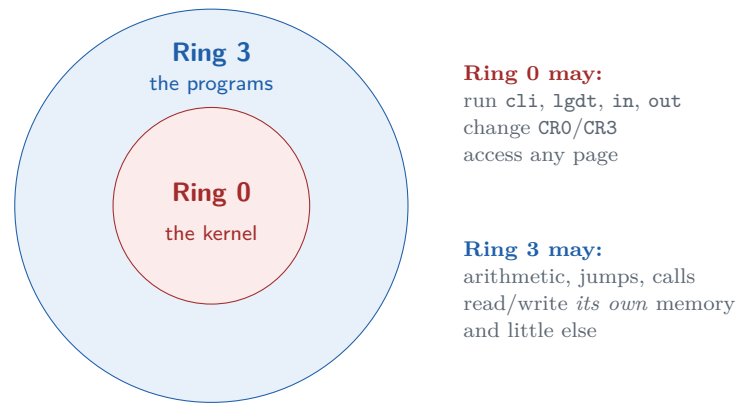
A conditional jump does not look at the result: it looks at the flags. `cmp a, b` internally performs a subtraction and throws the result away, keeping only the flags; `je` (“jump if equal”) really jumps if **ZF** is set.

IF and the value 0x202

All over the process code you will see `eflags = 0x202`. In binary that is 0000 0010 0000 0010: bit 1 is always set (reserved, the processor requires it) and bit 9 is **IF**. So 0x202 means “a clean state **with interrupts enabled**”. If a process started with 0x002, it would never yield control back to the scheduler and would hang the system.

3.3 Rings: why not everyone may do everything

The x86 processor has four privilege levels, called **rings**, numbered 0 to 3. In practice almost every system uses only two:



If a ring 3 program tries to execute `cli` (disable interrupts), the processor does not run it: it raises a **general protection fault**, and the kernel decides what to do with that program. Usually, kill it.

Isolation is not done in software

This is the key idea of the whole course. An operating system does not stop a program from doing damage by “watching it”: it **configures the hardware so the program is physically incapable of it**. The kernel sets the rules once (in the GDT, in the page tables) and from then on the silicon enforces them, at zero cost. If the kernel sets those rules wrongly, the isolation simply does not exist.

3.4 The control registers

Besides the general-purpose registers, some registers control how the processor operates. This course uses three:

CR0

Master switches. Bit 0 (*PE*) turns on protected mode. Bit 31 (*PG*) turns on paging. Setting those two bits, in the right order, is literally the moment the system becomes a modern one.

CR2

When a page fault occurs, the processor leaves *the address that was accessed* here. It is the first question any fault handler asks.

CR3

Holds the physical address of the active page directory. **Changing CR3 means changing address space**, which makes it the heart of a context switch.

lytlnybl/memory/vmm.asm

```

1  set_cr3:
2      mov eax, [esp + 4]
3      mov cr3, eax          ; switch the active address space
4      ret
5
6  set_cr0:
7      mov eax, cr0
8      or  eax, 0x80000000    ; bit 31 = PG: enable paging
9      mov cr0, eax
10     ret

```

Three instructions. With them, the processor stops seeing physical memory directly and starts seeing a translated world. Part VIII explains what that means.

Exercise 3.1. Why do you think **CR3** holds a *physical* address rather than a virtual one? Think about what would happen if the processor had to translate the address of the page directory using the page directory itself.

CHAPTER 4

The memory of a PC

By the end of this chapter you will know

- How the stack works and why it grows downwards
- What a stack frame is, and exactly what happens on a function call
- How a PC's memory is laid out at boot
- What a stack overflow is and why it is so destructive

4.1 The stack

The **stack** is a region of memory with a very simple discipline: the last thing you put in is the first thing you take out. The processor supports it directly with two instructions:

- `push value`: subtracts 4 from `ESP` and writes the value there.
- `pop dest`: reads the value at `ESP` and adds 4 to `ESP`.

Notice the crucial detail: **push subtracts**. The stack grows towards *lower* addresses. It is a historical convention with an enormous practical consequence: if you place the stack just above your data, growing it overwrites them.



4.1.1 What happens on a function call

When code executes `call my_function`, this happens:

1. The arguments are pushed onto the stack (on 32-bit x86, right to left).
2. `call` pushes the **return address**: the address of the following instruction. Then it jumps.
3. The function pushes `EBP` and does `mov ebp, esp`, so `EBP` stays fixed at the start of its frame while `ESP` keeps moving.
4. The function subtracts from `ESP` to reserve its local variables.
5. On return, `leave` undoes the above and `ret` pops the return address and jumps to it.

```

1 vga_text_writeline:
2   push ebp           ; save the previous frame
3   mov  ebp, esp       ; establish the new frame
4   sub  esp, 8         ; reserve 8 bytes of locals
5   ; ... body of the function ...
6   leave              ; mov esp,ebp ; pop ebp
7   ret                ; jump to the return address

```

Why a stack overflow is catastrophic

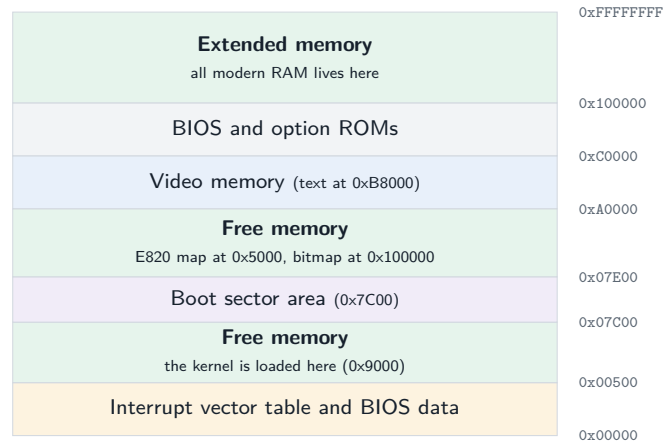
The return address lives *on the stack*, among the data. If a function writes past the end of a local array, it clobbers that address. When the `ret` arrives, the processor jumps to whatever that corrupted value says: usually, nowhere.

In a normal program that produces a segmentation fault and the operating system kills it.

In a kernel there is nobody above to do the killing. The processor tries to execute garbage, faults, tries to handle the fault, faults again, and on the third failure it resets itself. Part XIII walks through a real case of exactly this, step by step.

4.2 The memory map of a PC

When the system boots, memory is neither empty nor uniform. There are regions reserved for historical reasons going back to 1981:



The holes in that list matter: if the memory manager hands out a page that is really video memory, the system starts writing data and strange symbols appear on screen. Worse: if it hands out a page holding its own code, it overwrites itself.

This is why E820 exists

Since the real map depends on the machine, the BIOS offers a service (function 0xE820) that returns a list of regions: address, size and type (1 = usable, anything else = do not touch). The lytlnyblOS bootloader asks for that list and stores it at 0x5000 so the kernel can read it later. You will see that code in Chapter 12.

Exercise 4.1. Looking at the map, why do you think the boot sector is loaded at 0x7C00 rather than at 0x0? Hint: look at what lives at 0x0, and how much room is left above 0x7C00 before the next reserved block.

CHAPTER 5

Talking to hardware

By the end of this chapter you will know

- How `in` and `out` are implemented and called from C
- Why you sometimes have to wait for hardware
- What polling is and why it is a bad long-term idea

5.1 The port functions

All conversation with devices goes through four tiny assembly functions, because C has no way to express the `in` and `out` instructions.

lytlnybl/kernel/interrupts.asm

```
1  outb:                                ; void outb(uint16_t port, uint8_t value)
2      mov al, [esp + 8]
3      mov dx, [esp + 4]
4      out dx, al
5      ret
6
7  inb:                                  ; uint8_t inb(uint16_t port)
8      mov dx, [esp + 4]
9      in al, dx                        ; the result lands in AL, and AL is part of EAX,
10     ret                             ; which is where C expects a return value
```

The `inb` detail is worth pausing on: there is no instruction that “returns” a value. The C calling convention simply says the return value travels in `EAX`, and `in al, dx` already leaves the byte in `AL`, the low byte of `EAX`. The function works by construction.

The 16-bit versions (`inw`/`outw`) are identical with `AL` replaced by `AX`, and are used for the disk, which transfers two bytes at a time.

5.2 Waiting for hardware

Devices are vastly slower than the processor. Send two commands back to back to an old chip and the second may arrive before it has digested the first. There are two ways to deal with that.

5.2.1 A blind pause

```

1 io_wait:
2     out 0x80, al    ; write to a port that does nothing
3     ret            ; the bus round trip burns the time we need

```

Port 0x80 is connected to nothing useful on a PC. Writing to it has no effect, but the bus cycle takes long enough (a few microseconds) to give a slow chip time to catch up. It is an old, ugly, perfectly effective trick. It is used when reprogramming the interrupt controller.

5.2.2 Asking until it is ready

The disk is more sophisticated: it has a status register you can poll.

lytlnybl/kernel/drivers/ata.c

```

1 static void ata_wait_bsy() {
2     uint8_t status;
3     do {
4         status = inb(ATA_STATUS);
5     } while (status & ATA_STATUS_BSY);    /* spin until it stops being busy */
6 }

```

This is **polling**: asking in a loop. It is simple and correct, but while the loop spins **the processor does nothing else**. During a disk read the whole system is frozen.

Polling versus interrupts

The alternative is to ask the device to *signal* when it finishes, via an interrupt, and run another process meanwhile. That is what real systems do, and it is considerably more complex. lytlnyblOS uses interrupts for keyboard and timer but polling for disk: a very typical mix in small systems. It is a conscious trade-off, not an oversight.

In short

You now have the vocabulary of the machine: registers, flags, stack, addresses, ports and rings. Part II steps up a level and answers the underlying question: given that hardware, what exactly must an operating system contribute, and why?

PART II

What an operating system is

CHAPTER 6

The minimum pieces

By the end of this chapter you will know

- Which problems an operating system solves, and which it does not
- The seven pieces without which there is no operating system
- In what order they must be built, and why that order is mandatory

6.1 The problem it solves

Imagine you want to run two programs at once on the computer from Chapter 1. With no operating system, you have to answer a pile of awkward questions yourself:

- Where does each program load? If both expect to sit at 0x400000, they clobber each other.
- How do they share the processor? There is only one **EIP**.
- If one enters an infinite loop, how do I get control back?
- If one writes to a wrong address, how do I stop it wrecking the other?
- Both want to write to the screen. Who wins?
- Both want to save a file. How do they know which disk sectors are free?

An operating system is exactly the answer to that list. It is not “the program that starts when you power on”; it is **the referee that shares limited hardware between programs that do not know each other and do not trust each other**.

The three jobs of a kernel

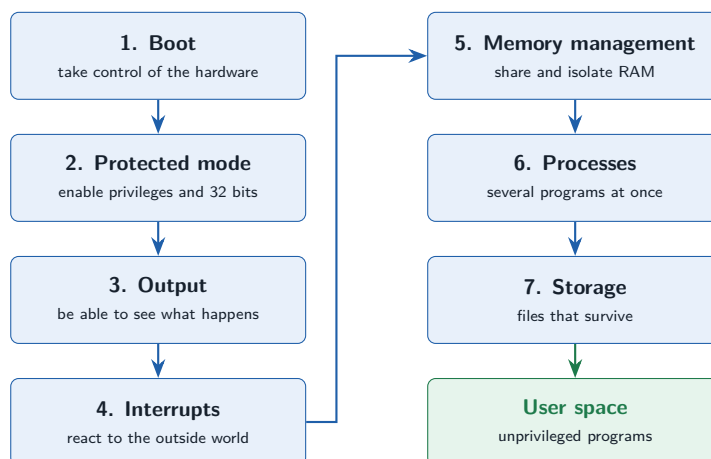
Everything a kernel does falls into one of three categories:

1. **Abstract**: turn “disk sectors” into “files”, “physical pages” into “your own memory”, “one processor” into “every program has its own”.
2. **Share**: decide who uses what, and for how long.
3. **Protect**: guarantee that one program cannot damage another or the kernel itself.

Every technical chapter in this course implements one of the three, for one specific resource.

6.2 The seven pieces

A minimal but genuine operating system — able to run several isolated programs — needs these seven pieces. Not one fewer.



Let us walk through them, without yet worrying about implementation.

6.2.1 1. Boot: taking control

Somebody has to go first. The firmware loads 512 bytes from disk and jumps into them; those 512 bytes have to load the rest. It is a problem of pulling yourself up by your own bootstraps — hence *bootstrapping*, and hence *boot*.

6.2.2 2. Protected mode: unlocking the processor

The processor starts up pretending to be a chip from 1978, for compatibility. It sees only 1 MB of memory, works in 16 bits and has no protection at all. It must be switched explicitly. Without this, pieces 5 to 7 are literally impossible.

6.2.3 3. Output: being able to see what happens

This looks trivial and is not. Debugging an operating system with no way to print is like fixing an engine with your eyes shut. It is the first thing built after you have a working execution environment, and the reason is purely practical.

6.2.4 4. Interrupts: reacting

Without interrupts, a program that enters an infinite loop keeps the processor forever. Interrupts are the mechanism by which hardware can *take* control away from running code and hand it to the kernel. They are the basis of:

- multitasking (the timer interrupts and the kernel switches process),
- devices (the keyboard interrupts when a key is pressed),
- error handling (dividing by zero interrupts),
- system calls (a program interrupts itself to ask for something).

Interrupts are the axis

If you had to keep one mechanism to understand an operating system, this would be it. Almost everything interesting a kernel does happens *inside an interrupt handler*. In lytlmybIOS, process switching, keyboard input and every system call all go through one.

6.2.5 5. Memory management: sharing and isolating

It comes in three stacked layers:

Layer	Unit	Answers the question
Physical memory	4 KB page	Which chunks of RAM are free?
Virtual memory	4 KB page	What does each program see when it looks at an address?
Heap	bytes	How do I ask for 17 bytes without wasting a whole page?

The middle layer, virtual memory, is the hardest and the most important: it is what makes it possible for two programs to use the same address 0x400000 without colliding. It gets a chapter of its own, explained very slowly.

6.2.6 6. Processes: several programs at once

A **process** is a program *in execution* plus all its state: its register values, its stack, its memory, its open files. The kernel stores that state in a structure and swaps it dozens of times a second, creating the illusion of simultaneity.

6.2.7 7. Storage: making things survive

RAM is erased when you power off. For a file to exist tomorrow, it has to be written to disk; and for it to be *findable* tomorrow, the disk needs a structure imposed on it: a **filesystem**.

6.3 Why this order and no other

The order is not pedagogical: it is a chain of real technical dependencies.

To build...	you first need...
Protected mode	to have booted and have the code in memory
VGA text mode	protected mode (to address above 1 MB comfortably)
Interrupts	protected mode (the interrupt table is different) and output (to see faults)
Timer	working interrupts and a remapped PIC
Physical memory	the memory map the bootloader stored
Virtual memory	the physical manager (to request pages for the tables)
Kernel heap	virtual memory (to map more pages as it grows)
Processes	the heap (to allocate structures) and interrupts (to switch)
User space	processes, virtual memory and a TSS
System calls	user space and interrupts
Filesystem	a disk driver
The shell	absolutely everything above

The cost of skipping a link

It is tempting to try loading user programs before paging is properly sorted, or to write the filesystem before the heap. You can, but you pay for it: the code fills up with special cases that later have to be unpicked. This course follows the order the project is actually built in, and that is not a coincidence.

CHAPTER 7

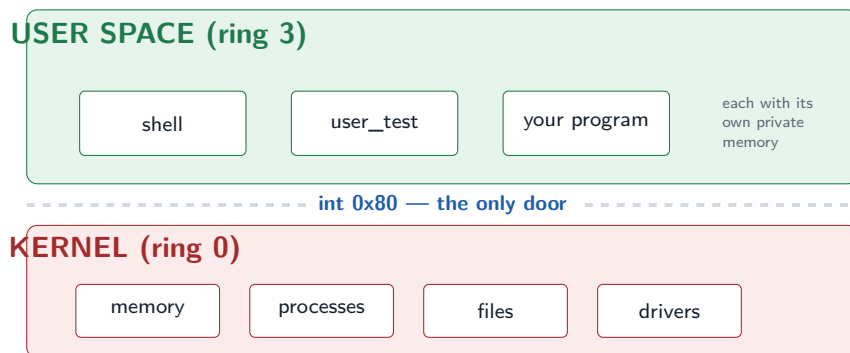
Kernel and user space

By the end of this chapter you will know

- What “user space” means exactly
- What a system call is and why it is not an ordinary function call
- What has to happen to cross the boundary in each direction

7.1 Two worlds

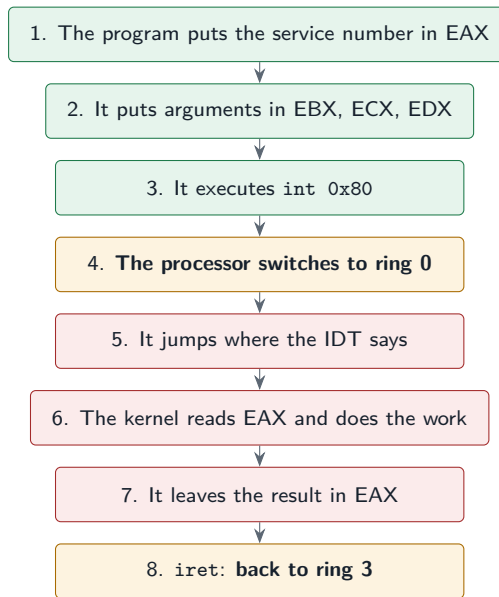
We saw that the processor has privilege rings. An operating system uses them to split software into two worlds with different rules:



The dashed line is the boundary. It is a *real* boundary, enforced by the processor, not a convention. A user program cannot call a kernel function even if it knows the address: the page is marked supervisor-only and the attempt raises an exception.

7.2 The only door

If a program cannot call the kernel, how does it ask it to print something? Through the one mechanism the processor allows: **raising an interrupt on purpose**.



Steps 4 and 8 are the important ones, and the *hardware* performs them, not software. It is the processor that decides to raise privilege, and it only does so by jumping to an address the kernel registered in advance. The program does not choose where it lands: it chooses *what it asks for*, not *where it goes*.

Why this is safe

A malicious program cannot “jump into the kernel at a convenient line” because the only entry is the one the interrupt table defines. The kernel controls the landing point. All the attacker controls are the numbers in the registers — which is why **validating those numbers is the single most important thing a system call handler does**. Part XIII shows what happens when that validation is missing.

7.3 The ten services of lytlhybIOS

Our system offers ten calls. That is a ridiculously small number (Linux has over three hundred) but enough for a complete shell.

EAX	Name	Arguments	What it does
0x01	exit	—	Terminates the process and wakes its parent
0x02	getpid	—	Returns the process identifier
0x03	yield	—	Voluntarily gives up the processor
0x04	sleep	ticks	Sleeps for n timer ticks
0x05	write	fd, buffer, n	Writes to the screen or to a file
0x06	read	fd, buffer, n	Reads from the keyboard or from a file
0x07	sbrk	increment	Grows the process heap
0x08	fsops	operation, path	ls, mkdir, touch, rm, run
0x09	open	path	Opens a file and returns a descriptor
0x0A	close	fd	Closes a descriptor

Notice that `write` serves both the screen and files. That is an idea inherited from Unix: **everything is a descriptor**. The program does not know whether 1 is the screen or a file; it only knows it is writing to “descriptor 1”. The kernel decides what that means.

Exercise 7.1. Look at the table and work out the minimum set of calls the shell’s `cat file.txt` command needs, and in what order. (The answer is in Part XII, but try first.)

CHAPTER 8

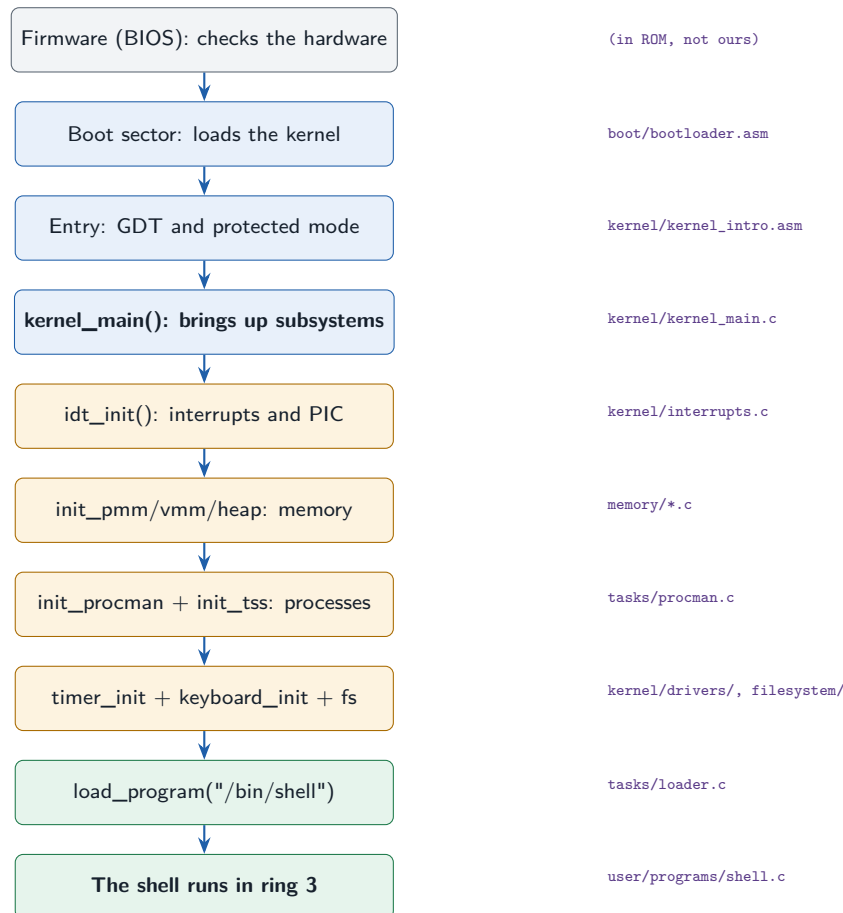
Map of the whole system

By the end of this chapter you will know

- See the complete boot journey end to end
- Know which project file implements each stage
- Have a mental map to come back to whenever you get lost

This chapter is the reference to return to whenever you wonder “where in the system am I?”. You do not need to understand it all now; you need to know it exists.

8.1 The journey, end to end

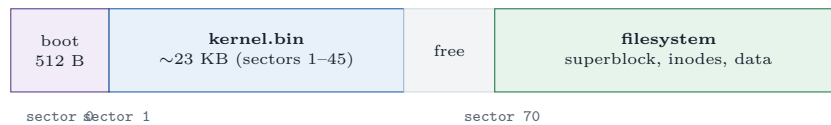


8.2 What each directory holds

Directory	Contents
boot/	The boot sector. 512 bytes of 16-bit assembly.
kernel/	Protected-mode entry, interrupts, utilities and <code>kernel_main</code> .
kernel/drivers/	VGA text mode, PIT timer, PS/2 keyboard and ATA disk.
memory/	Physical manager (bitmap), virtual manager (paging) and heap.
tasks/	Processes, context, scheduler and program loader.
filesystem/	The filesystem in two layers: <code>fs.c</code> (low level) and <code>fs_manager.c</code> (descriptors and paths).
user/libc/	The minimal standard library linked into every user program.
user/programs/	The programs: the shell and a test program.
tools/	<code>mkfs.c</code> : compiled for <i>your</i> machine, not for the system.
include/	All headers, mirroring the structure above.
rootfs/	The files <code>mkfs</code> injects into the disk image.

8.3 What the build produces

`make` generates a single 10 MB disk image laid out like this:



The 60-sector ceiling

The boot sector asks the BIOS for exactly 60 sectors (30,720 bytes). The kernel currently occupies around 23 KB, so there is headroom — but not unlimited. If the kernel exceeded that size, the BIOS would load only part of it and the system would fail in a baffling way: no error message, just executing garbage. It is a limit worth remembering before adding large code to the kernel.

In short

You now know which pieces exist, what each is for, and where its code lives. From here we build, in order, starting at the literal beginning: the first 512 bytes.

PART III

Booting

CHAPTER 9

From the power button to the first instruction

By the end of this chapter you will know

- What the firmware does before anything of yours exists
- How the BIOS decides where to boot from
- What the 0xAA55 signature is and why nothing boots without it

9.1 The first few milliseconds

You press the button. The processor initialises with fixed values: `CS` holds 0xF000, `EIP` holds 0xFFFF0, and it is in **real mode**, behaving like an 8086 from 1978.

At the resulting address there is a jump into the BIOS code. From there, the firmware does its work:

1. **POST** (Power-On Self Test): check that RAM responds, that a keyboard exists, that disks answer.
2. Initialise the basic controllers: interrupt controller, timer, disk controller.
3. Install its own service table in low memory, so software can ask it for things.
4. Look for a device to boot from.

9.2 How the boot device is chosen

The BIOS walks the devices in the configured order. From each one it reads the first sector — 512 bytes — and checks exactly one thing: that **the last two bytes are 0x55 and 0xAA** (which read as a little-endian 16-bit number form 0xAA55).

If the signature is there, the BIOS considers that sector bootable: it copies it to address 0x7C00 and jumps there. If not, it moves to the next device.

The contract is two bytes long

That is the entire contract between firmware and your operating system: 512 bytes ending in 0xAA55. There is no file format, no header, no integrity check. The BIOS loads those bytes and jumps. What happens next is entirely your responsibility.

That is why the last line of the lytlnyblOS bootloader is exactly this:

lytlnybl/boot/bootloader.asm

```
1 times 510-($-$$) db 0      ; pad with zeros up to byte 510
2 dw 0xAA55                  ; the magic signature in bytes 510 and 511
```

\$ means “the current address” and \$\$ “the start address of the section”, so \$-\$\$ is “how many bytes I have written so far”. The directive pads with zeros up to 510 and then places the signature. If your code went past 510 bytes, NASM would error out on a negative count: it is a hard limit.

Why 0x7C00 and nowhere else

The address 0x7C00 is a relic of the original IBM PC 5150, which had 32 KB of RAM. The designers wanted the boot sector as high as possible without touching the last kilobyte (reserved), and 0x7C00 is exactly 32 KB – 1024. Forty-five years later, that number is still there.

CHAPTER 10

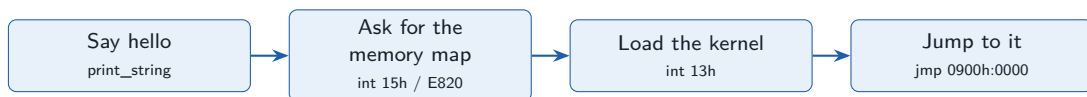
The 512 bytes of the bootloader

By the end of this chapter you will know

- Read the complete lytlnyblOS bootloader and understand every line
- Understand 16-bit segmentation and the `segment×16 + offset` calculation
- Know why the bootloader has to care about the `DS` register

10.1 The whole program

Before dissecting it, look at the overall shape. The lytlnyblOS bootloader does four things, in this order:



lytlnybl/boot/bootloader.asm

```
1 start:
2     mov ax, 07C0h
3     mov ds, ax           ; DS = 0x07C0, so that labels resolve correctly
4
5     mov si, title_string
6     call print_string
7
8     mov si, message_string
9     call print_string
10
11    call store_memory_map
12    call load_kernel_from_disk
13
14    jmp 0900h:0000        ; jump to the kernel, loaded at physical address 0x9000
```

Twelve lines and the system is already in motion. Let us understand the first two, which are the most confusing.

10.2 16-bit segmentation: why 0x07C0

In real mode the processor has 16-bit registers. With 16 bits you can only count to 65,535, that is 64 KB. But the 8086 wanted to address 1 MB. Intel's solution was to combine *two* 16-bit registers:

$$\boxed{\text{segment}} \times 16 \quad + \quad \boxed{\text{offset}} \quad = \quad \boxed{\text{physical address}}$$

Multiplying by 16 is the same as shifting four bits left, that is, adding a hex zero. So:

$$07C0 \times 16 = 7C00 \quad \text{and} \quad 7C00 + 0000 = 7C00$$

Which is exactly where the BIOS left our code. By setting `DS = 0x07C0`, any label such as `title_string` (which the assembler turns into a small offset, say `0x6E`) resolves to physical address `0x7C6E`: the right place.

If you forget to set DS

If you do not initialise `DS`, it holds whatever the BIOS left there, usually 0. Then `title_string` would resolve to physical address `0x6E`, which is in the middle of the interrupt vector table. You would print garbage — or hang the machine. It is the number one mistake of anyone writing their first bootloader, and it produces no message at all: it simply does not work.

Same place, many names

An odd consequence of this scheme is that one physical address has many representations: `07C0:0000`, `0000:7C00` and `0700:0C00` all point at `0x7C00`. It is not a design error; it is what happens when segments overlap every 16 bytes.

10.3 Printing without an operating system

There is no video driver yet, so we use a BIOS service. They are invoked with `int`, after putting the function number in `AH`.

lytlnybl/boot/bootloader.asm

```
1 print_string:
2     mov ah, 0Eh          ; video service function 0x0E: teletype output
3 print_char:
4     lodsb                ; AL = [DS:SI], then SI = SI + 1
5     cmp al, 0             ; end of string?
6     je printing_finished
7     int 10h              ; BIOS video service: print AL
8     jmp print_char
9 printing_finished:
10    mov al, 10d           ; line feed
```

```

11     int 10h
12     mov ah, 03h        ; read cursor position
13     mov bh, 0
14     int 10h
15     mov ah, 02h        ; set cursor position
16     mov dl, 0          ; column 0
17     int 10h
18     ret

```

The `lodsb` instruction is an x86 shorthand: load into `AL` the byte pointed at by `DS:SI` and increment `SI`. It is exactly the equivalent of `c = *p++;` in C, in a single instruction.

The end of the routine has an amusing historical detail: code 10 is only a *line feed*, which moves down a row but does not return to the left margin. So afterwards you must read the cursor position and set the column to 0 by hand. A direct inheritance from typewriters.

Exercise 10.1. Why does `print_string` use `SI` and not, say, `BX` to walk the string? Hint: look at which register `lodsb` uses implicitly.

CHAPTER 11

Reading the disk and loading the kernel

By the end of this chapter you will know

- What CHS addressing is and why it still exists
- How to ask the BIOS to read sectors
- Why the kernel is loaded at 0x9000

11.1 Service 0x13

The BIOS offers disk access through interrupt 0x13. Function 2 reads sectors, and expects its parameters spread across half the register file:

lytlnybl/boot/bootloader.asm

```
1  load_kernel_from_disk:
2      mov ax, 0900h
3      mov es, ax          ; ES = 0x0900 -> destination physical address 0x9000
4
5      mov ah, 02h         ; function 2: read sectors
6      mov al, 60          ; how many sectors to read
7
8      mov ch, 0h          ; cylinder 0
9      mov cl, 02h         ; sector 2 (counting starts at 1!)
10     mov dh, 0h          ; head 0
11     mov dl, 80h         ; drive 0x80 = first hard disk
12
13     mov bx, 0h          ; destination offset -> ES:BX = 0900:0000
14
15     int 13h
16     jc kernel_load_error ; the BIOS sets the carry flag on failure
17     ret
```

11.1.1 CHS: cylinder, head, sector

Old disks were spinning platters and were addressed by their physical geometry: **cylinder** (which ring), **head** (which side of the platter) and **sector** (which slice of the ring). Today disks are electronic and that geometry is fictional, but the interface persists for compatibility.

Sectors start at 1

In CHS, cylinders and heads are numbered from 0, but **sectors are numbered from 1**. So `c1 = 2` means “the second sector of the disk”, the first one *after* the boot sector. This inconsistency has caused countless bugs since 1983.

11.1.2 Where it loads

`ES:BX` gives the destination. With `ES = 0x0900` and `BX = 0`, the physical address is `0x9000`. Why there? Look back at the memory map in Chapter 4:

- below `0x500` sit the interrupt vector table and BIOS data: untouchable;
- at `0x7C00` we are ourselves: we cannot write over our own running code;
- from `0xA0000` onwards begins video memory.

`0x9000` is comfortably free, leaves plenty of room below and above, and allows loading up to `0xA0000` (some 60 KB) without hitting anything.

Why 60 sectors

$60 \times 512 = 30,720$ bytes. The figure is hand-picked: enough for the current kernel (~23 KB) with headroom, and short of the 63-sectors-per-track limit that a single CHS read imposes. If the kernel grew further, this number would have to grow *and* you would have to watch that the read does not cross a track boundary.

11.2 The final jump

```
1  jmp 0900h:0000
```

This is a **far jump**: besides changing `IP`, it changes `CS`. Changing both matters, because the kernel code will compute its own addresses assuming `CS` holds `0x0900`. From this instruction on, the bootloader has finished its job and the kernel is in control.

In practice: editing the boot sector

```
examples/ch12-bootloader/ -- make run
```

The real bootloader with four marked edits: your own message, two characters printed with the BIOS teletype call directly, and a pause using `INT 15h / AH=86h` so you can read the result before the kernel clears the screen.

It still loads the kernel and jumps to it, so the system boots normally afterwards. Watch the `times 510-($-$$)` line at the bottom: add too much and NASM fails with a negative count, which is the 512-byte limit enforcing itself.

Watch the real boot

Build and run with `make run`. You will see, in order:

```
Welcome to the lytlnybl bootloader!  
Loading up the kernel for you...  
Hello World!, i am lytlnybl0S, in real mode
```

The first two are printed by the bootloader from 0x7C00. The third is already printed by the kernel from 0x9000, still in real mode. If you see all three, the disk load worked.

CHAPTER 12

The E820 memory map

By the end of this chapter you will know

- Why the kernel cannot guess how much RAM exists
- How the map is requested from the BIOS
- How it is stored so the kernel can find it later

12.1 The problem

The kernel needs to know which memory regions it may use. It cannot assume: every machine has a different amount of RAM and holes in different places (memory reserved by the BIOS, by ACPI, by memory-mapped devices).

There is only one moment when you can ask: **while still in real mode**, because as soon as we enter protected mode we lose access to BIOS services. That is why this job belongs to the bootloader, even though the kernel is the one that needs the data.

12.2 How you ask

The service is called E820 and works by iteration: you call it in a loop and each call returns one region, until it says it is done.

lyt1nybl/boot/bootloader.asm

```
1 store_memory_map:
2     xor ax, ax
3     mov ds, ax          ; DS = 0, to work with absolute addresses
4
5     xor ebx, ebx        ; EBX = 0 means "first call"
6     xor bp, bp          ; BP will count the entries
7
8     mov di, memory_map_buffer ; ES:DI = where to write (0x5000)
9 next_entry:
10    mov ax, 0
11    mov es, ax
12
```

```

13  mov eax, 0xE820
14  mov edx, 0x534D4150 ; 'SMAP' in ASCII: mandatory signature
15  mov ecx, 24          ; size of each entry, in bytes
16
17  int 15h
18  jc done              ; carry = no more entries
19
20  add di, 24           ; advance the write pointer
21  inc bp               ; one more entry
22
23  test ebx, ebx        ; did EBX return to 0? then that was the last
24  jnz next_entry
25 done:
26  mov [memory_map_entries], bp
27  mov ax, 07C0h
28  mov ds, ax           ; restore DS before returning
29  ret

```

Details worth noticing:

EBX is the loop state

The BIOS uses it as a bookmark. You start at 0, it returns a value you must keep untouched for the next call, and when it returns to 0 the iteration is over. Touch **EBX** between calls and you break the loop.

The 'SMAP' signature

0x534D4150 is “SMAP” in ASCII. The BIOS checks it to distinguish this request from other 0x15 service functions.

DS temporarily becomes 0

Because we want to write to absolute address 0x5000. With **DS=0**, **di = 0x5000** points exactly there. It must be restored afterwards, or the rest of the bootloader prints garbage.

12.3 The format of each entry

Each entry occupies 24 bytes:

Offset	Size	Field
0	8 bytes	Base address of the region
8	8 bytes	Length in bytes
16	4 bytes	Type: 1 = usable , anything else = reserved
20	4 bytes	Extended attributes (we do not use them)

And in C, the kernel reads it with a structure that mirrors that layout:

lytlnybl/include/memory/pmm.h

```

1 typedef struct {
2     uint64_t base;
3     uint64_t length;
4     uint32_t type;
5     uint32_t acpi_extended;
6 } __attribute__((packed)) memory_map_entry_t;

```

What `__attribute__((packed))` does

By default the compiler inserts padding between struct fields to align them and make access faster. That is fine for internal structures, but disastrous when the structure must mirror a layout defined by hardware or by another program. `packed` tells the compiler: “add no padding, lay the fields out back to back”. It is used in every structure representing an external format: GDT entries, IDT entries, the TSS, and the filesystem structures.

12.4 The agreement between bootloader and kernel

The bootloader writes to two fixed addresses and the kernel reads them from there:

Address	Size	Contents
0x4FFC	2 bytes	Number of entries in the map
0x5000	$24 \times n$	The entries themselves

lytlnybl/memory/pmm.c

```

1 void init_pmm() {
2     uint16_t map_entry_count = *(uint16_t*)0x4FFC;
3     memory_map_entry_t* memory_map = (memory_map_entry_t*)0x5000;
4     /* ... */
5 }

```

Coupling through magic numbers

This is a fragile agreement: two different files, written in different languages, agreeing on a constant typed by hand in both. If someone changes `0x5000` in the bootloader and forgets to change it in `pmm.c`, the kernel will read garbage as if it were a memory map and conclude that it may use regions that do not exist.

Serious systems solve this with a documented boot protocol (Multiboot, for example) in which the loader passes a pointer to a structure in a register. For a teaching system the shared constant is acceptable — but it is worth knowing it is technical debt.

In short

The bootloader has done its job: the kernel sits at `0x9000`, the memory map sits at `0x5000`, and control has just jumped into kernel code. But we are still in real mode, 16-bit, with no protection at all. The next part changes that.

PART IV

Protected mode: unlocking the processor

CHAPTER 13

Why real mode will not do

By the end of this chapter you will know

- The three fatal limitations of real mode
- What the system gains by switching modes
- What is lost in the switch, and why timing matters

13.1 Three fatal limitations

Real mode exists for compatibility with software from 1981. It has three problems that make it useless for a genuine operating system:

Only 1 MB of memory

With the `segment×16 + offset` scheme, the highest possible address is $FFFF \times 16 + FFFF = 10FFEF$, barely over 1 MB. It does not matter that the machine has 16 GB: in real mode they do not exist.

16-bit registers

Everything is 16 bits. Pointers, counters, arithmetic. Manipulating a 32-bit address requires constant juggling.

No protection whatsoever

And this is the decisive one. In real mode *any* code can write to *any* address, execute *any* instruction, and switch off interrupts whenever it feels like it. There are no rings, no pages, no permissions. An operating system whose job is to isolate programs cannot run on a mode that offers no isolation mechanism.

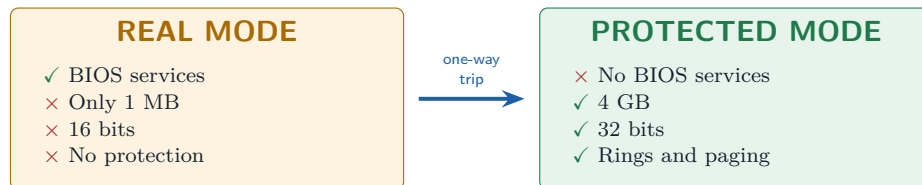
What you gain

32-bit **protected mode** provides exactly what is missing: 4 GB addressable, 32-bit registers, the four privilege rings, segments with permissions, and — most importantly — the ability to enable **paging**, the foundation of virtual memory.

13.2 What you lose

The switch is not free: on entering protected mode, **BIOS services stop working**. `int 10h`, `int 13h`, `int 15h`: all of them are 16-bit code that assumes real mode.

Hence the ordering in the bootloader we saw in the previous part: print, read the disk and request the memory map first (all through the BIOS), and only then switch modes. From that moment on, anything you want from the hardware you will have to program yourself.



The GDT: memory's driving licence

By the end of this chapter you will know

- What a segment descriptor is and what information it carries
- How to decode a descriptor byte by byte
- What a selector is and how it relates to privilege rings
- Why we use a “flat” model and what that means

14.1 The problem it solves

In protected mode, segment registers (**CS**, **DS**, **SS**...) no longer hold a number to be multiplied by 16. They now hold a **selector**: an index into a table of **descriptors**. And each descriptor states, for that region of memory:

- where it starts (*base*, 32 bits),
- how far it extends (*limit*, 20 bits),
- what may be done with it (execute, read, write),
- **from which privilege ring it may be used.**

That table is the **GDT** (Global Descriptor Table). It is the first thing that must be built, because without it the processor refuses to enter protected mode.

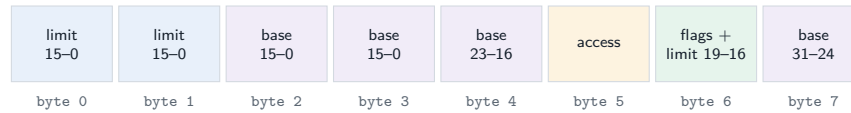
An analogy

In real mode, a memory address is like saying “house number 30 on the street that starts at kilometre 7”. Anyone can name any house.

In protected mode you first show a badge (the selector). The badge refers to a record (the descriptor) stating which area you may visit, whether you may enter or only look through the window, and what clearance you need. The processor checks the record on *every* access, in hardware, at no cost. If you do not qualify, you do not get in.

14.2 A descriptor, dissected

Each descriptor occupies 8 bytes, with its fields chopped up and scattered in a way explicable only by backwards compatibility with the 80286:



The important field is the **access** byte. Its eight bits mean:

Bit	Name	Meaning
7	P	Present. If 0, the descriptor is invalid and using it raises an exception
6-5	DPL	Privilege level: 00 = ring 0 (kernel), 11 = ring 3 (user)
4	S	1 = code or data segment; 0 = system segment (TSS, gates)
3	E	1 = executable (code); 0 = data
2	DC	Direction / conforming
1	RW	For code: is it readable? For data: is it writable?
0	A	Accessed; set by the processor itself

14.3 The lytlnyBIOS GDT

Our table has six entries. Here they are, with the access byte decoded:

lytlnybl/kernel/kernel_intro.asm

```

1 gdt_start:
2 gdt_null:
3     dq 0                ; entry 0: mandatorily null
4
5 gdt_code:                ; selector 0x08 - kernel code
6     dw 0xFFFF           ; limit low
7     dw 0x0000           ; base low
8     db 0x00             ; base middle
9     db 0x9A             ; access: present, ring 0, code, readable
10    db 0xCF              ; flags: 4K granularity + 32-bit, limit high
11    db 0x00              ; base high
12
13 gdt_data:                ; selector 0x10 - kernel data
14     dw 0xFFFF           ; limit low
15     dw 0x0000           ; base low
16     db 0x00             ; base middle
17     db 0x92             ; access: present, ring 0, data, writable
18     db 0xCF              ; flags: 4K granularity + 32-bit, limit high
19     db 0x00              ; base high
20
21 gdt_user_code:           ; selector 0x18 - user code
22     dw 0xFFFF           ; limit low
23     dw 0x0000           ; base low
24     db 0x00             ; base middle

```

```

25     db 0xFA                ; access: present, RING 3, code, readable
26     db 0xCF
27     db 0x00
28
29 gdt_user_data:             ; selector 0x20 - user data
30     dw 0xFFFF
31     dw 0x0000
32     db 0x00
33     db 0xF2                ; access: present, RING 3, data, writable
34     db 0xCF
35     db 0x00
36
37 gdt_tss:                   ; selector 0x28 - filled in later, from C
38     dw 0
39     dw 0
40     db 0
41     db 0
42     db 0
43     db 0
44 gdt_end:

```

14.3.1 Decoding 0x9A and 0xFA

These two bytes are the difference between the kernel and a user program. Bit by bit:

	P	DPL	S	E	DC	RW	A	
0x9A = 10011010	1	0	0	1	1	0	1	0 code, ring 0
0xFA = 11111010	1	1	1	1	1	0	1	0 code, ring 3
0x92 = 10010010	1	0	0	1	0	0	1	0 data, ring 0
0xF2 = 11110010	1	1	1	1	0	0	1	0 data, ring 3

Those two DPL bits are the boundary between the two worlds of Chapter 7. All of the system's isolation rests, literally, on them.

14.3.2 Base 0, maximum limit: the flat model

Notice that the four useful descriptors all have base 0x00000000 and limit 0xFFFFF with 4 KB granularity, which covers the full 4 GB.

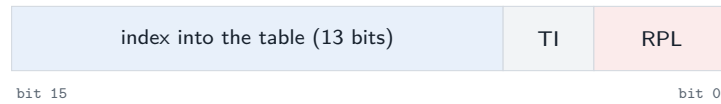
That means *every segment is the same segment*: all of memory. It is called the **flat model**, and its advantage is enormous: the logical address you write in C is the linear address, with no strange translation. Segments stop being a way to separate memory and are left doing **one job only**: carrying the privilege bits.

Division of labour in modern x86

Since the 386, memory isolation is done with **paging**, not segmentation. Segments are left flat and used only for privilege level. That is exactly what Linux does, and what lytlnyblOS does. Segmentation survives as a mandatory piece of paperwork you must fill in before you can use the thing that actually matters.

14.4 Selectors

A selector is a 16-bit number with three fields:



Because the low two bits are the RPL, the index is shifted three positions up. Hence selectors go in steps of eight:

Selector	Index	RPL	Points at
0x00	0	0	null descriptor
0x08	1	0	kernel code
0x10	2	0	kernel data
0x18 3 = 0x1B	3	3	user code
0x20 3 = 0x23	4	3	user data
0x28	5	0	the TSS

Which is why creating a user process looks like this:

```

lytlnybl/tasks/procman.c
1 void create_uprocess(process_t* new_process) {
2     new_process->regs.cs = 0x18 | 3;    /* user code, RPL = 3 */
3     new_process->regs.ds = 0x20 | 3;    /* user data, RPL = 3 */
4     new_process->regs.ss = 0x20 | 3;
5     /* ... */
6 }
  
```

Exercise 14.1. What would happen if a user process were created with `cs = 0x18` without the `| 3`? Think about what the processor checks when loading **CS**. (Hint: one rule says the RPL may not be more privileged than the descriptor’s DPL, and another forbids “dropping” privilege through a plain `iret`.)

In practice: reading the live GDT

```
examples/ch16-gdt/ -- make run
```

Asks the processor for its own GDT with `SGDT`, walks the six descriptors, decodes each access byte into **P** / **DPL** / **S** / **E**, and prints which selectors the kernel is currently running with.

It surfaces something the source alone cannot tell you: `kernel\_intro.asm` writes `0x92` for the kernel data segment, but the live table reads back `0x93`. Bit 0 is *Accessed*, and the processor sets it the first time the descriptor is loaded into a segment register. The TSS likewise reads `0x8B` rather than `0x89`, because `ltr` sets bit 1, *Busy*. The hardware writes

into your GDT.

CHAPTER 15

The jump to 32 bits

By the end of this chapter you will know

- The exact sequence for entering protected mode
- Why a far jump is needed immediately afterwards
- What the kernel stack is and where it lives

15.1 The sequence

lytlnybl/kernel/kernel_intro.asm

```
1  enter_protected:
2      cli                ; 1. disable interrupts
3      lgdt [gdttr - start] ; 2. load the GDT
4      mov eax, cr0
5      or eax, 1          ; 3. set the PE bit in CRO
6      mov cr0, eax
7
8      CODE_SEG equ gdt_code - gdt_start
9      jmp CODE_SEG:p_mode_main ; 4. far jump -> reload CS
10 [bits 32]
11 p_mode_main:
12     mov ax, 10h          ; 5. reload the data segments
13     mov ds, ax
14     mov es, ax
15     mov fs, ax
16     mov gs, ax
17     mov ss, ax
18     mov esp, kernel_stack_top ; 6. set up the stack
19
20     mov byte [0xB8000], 'P' ; 7. sign of life
21     mov byte [0xB8001], 0x02
22
23     extern kernel_main
24     call kernel_main      ; 8. enter C
```

15.1.1 Step 1: cli

Disables interrupts. It is essential: for the next few microseconds the processor is in an inconsistent state, with a new GDT but old segments. If an interrupt arrived, it would jump into the *real mode* vector table, which no longer makes sense, and the machine would hang.

Interrupts are not re-enabled until the end of `idt_init()`, already in C, once a valid 32-bit interrupt table exists.

15.1.2 Step 2: lgdt

Loads the **GDTR** register with the table's address and size. The operand format is peculiar:

```
1  gdttr:
2      dw gdt_end - gdt_start - 1    ; limit: size MINUS ONE
3      dd gdt_start                  ; base address, 32 bits
```

The minus one

The limit is “the last valid byte”, not “the size”. With 6 descriptors of 8 bytes that is 48 bytes, so the limit is 47. This is a classic bug: put 48 and the last descriptor falls partly out of range, and using it raises a general protection fault. The same convention applies to the IDT.

15.1.3 Step 3: the PE bit

or `eax, 1` sets bit 0 of **CR0**. At that instant the processor is *already* in protected mode. But there is a subtlety.

15.1.4 Step 4: why the far jump is required

The processor internally caches (in a “descriptor cache”) the information for the current segment, and that cache still holds the real-mode interpretation of **CS**. Until **CS** is reloaded, the processor keeps executing code as if it were 16-bit.

And **CS** *cannot be modified with a mov*. The only ways to change it are a far jump, a far call, or an `iret`. Hence:

```
1  jmp CODE_SEG:p_mode_main
```

That jump is the exact moment the processor begins executing 32-bit instructions. The `[bits 32]` directive right after it generates no code: it tells NASM to assemble for 32 bits from there.

The instant of the switch

Three instructions mark the transition: `mov cr0, eax` enables the mode, the far `jmp` reloads **CS**, and the `movs` reload the data segments. Until all three are done, the processor sits in an intermediate state where almost nothing behaves as you expect. That is why the whole block runs with interrupts off and calls nothing.

15.1.5 Step 6: the stack

```
1 section .bss
2 align 16
3 kernel_stack_bottom:
4     resb 0x4000          ; 16 KB of stack
5 kernel_stack_top:
```

`resb` reserves space without initialising it, in the `.bss` section. Notice that `kernel_stack_top` sits *after* the 16 KB: since the stack grows downwards, `ESP` must point at the end of the block.

16 KB is not much

That is the entire kernel stack. Every function call consumes tens or hundreds of bytes; every local array, whatever it occupies. If the kernel overruns it, `ESP` drops below `kernel_stack_bottom` and starts clobbering whatever is there — usually the kernel's own global variables. There is no check whatsoever: nobody warns you.

Part XIII shows a real case where the system rebooted in a loop for exactly this reason, and how it was diagnosed.

CHAPTER 16

Entering C

By the end of this chapter you will know

- What compiling “freestanding” means and what disappears
- What a linker script does and why it is mandatory here
- How the `.text`, `.data` and `.bss` sections relate

16.1 C without a standard library

Writing the kernel in C means giving up nearly everything you normally take for granted. The compiler’s `-ffreestanding` option says so:

lytlnybl/Makefile

```
CC = i686-elf-gcc
CFLAGS = -g -m32 -Iinclude -ffreestanding
```

Flag	What it does
<code>-m32</code>	Generate 32-bit code
<code>-ffreestanding</code>	Do not assume a standard library or a <code>main()</code> exists; do not emit implicit calls to library functions
<code>-Iinclude</code>	Look for headers in <code>include/</code>
<code>-g</code>	Include debug information for GDB

What *disappears*: `printf`, `malloc`, `strlen`, `FILE`, all of it. If the kernel wants to copy memory, somebody has to write `memcpy`. And somebody does:

lytlnybl/kernel/kernel_utils.c

```
1 void* memcpy(void* dest, const void* src, size_t len) {
2     uint8_t* d = (uint8_t*)dest;
3     const uint8_t* s = (const uint8_t*)src;
4     for (size_t i = 0; i < len; i++)
5         d[i] = s[i];
6     return dest;
7 }
```

What you can still use

The headers `<stdint.h>`, `<stddef.h>` and `<stdbool.h>` are available even in freestanding mode, because they define no functions: only types (`uint32_t`, `size_t`, `bool`). You will see them included throughout the project.

16.2 The linker script

The compiler produces chunks of code. The linker joins them and decides what address each one goes to. Normally that decision is made by default, but here **the address matters**: the bootloader jumps to `0x9000` and the start of the kernel must be there.

```
lytlnybl/kernel/linker.ld
```

```
ENTRY(start)

SECTIONS
{
    . = 0x9000;           /* the location counter starts here */

    _kernel_start = .;   /* remember where we start */

    .text : { *(.text) } /* executable code */
    .data : { *(.data) } /* initialised globals */
    .bss  : { *(.bss) }  /* zeroed globals */

    _kernel_end = .;     /* remember where we end */
}
```

The dot `.` is the **location counter**: the address where the next thing will be placed. Assigning `0x9000` to it and then listing sections is, literally, placing pieces in memory.

16.2.1 The three sections

Section	In the file?	Contains
<code>.text</code>	yes	Machine code. Read-only in serious systems
<code>.data</code>	yes	Globals with an initial value: <code>int x = 42;</code>
<code>.bss</code>	no	Globals with no value or zero: <code>int y;</code> . Only space is reserved

Why `.bss` takes no file space

If you have a global 16 KB array of zeros, it would be absurd to store 16 KB of zeros on disk. The linker notes “16 KB of blank goes here” and whoever loads the program is responsible for zeroing it.

And here is a real trap in this project: the thing that loads the kernel is the bootloader, which zeroes nothing; it simply copies sectors. It works because RAM happens to be zero at

boot. It is a detail a serious system would have to handle explicitly.

16.2.2 The `_kernel_start` and `_kernel_end` symbols

These two labels are not variables: they are *addresses* computed by the linker. From C they are declared like this:

lytlnybl/memory/pmm.c

```
1 extern char _kernel_start;
2 extern char _kernel_end;
3
4 /* and used by taking their ADDRESS, not their value */
5 uint32_t kernel_frame_index = (uint32_t)&_kernel_start / FRAME_SIZE;
6 uint32_t kernel_frame_end   = (uint32_t)&_kernel_end   / FRAME_SIZE;
```

They are declared as `char` and used with `&` because what matters is *where they are*, not what they contain. Thanks to them the memory manager can mark exactly the pages the kernel occupies as used, with no magic numbers.

A latent bug hiding in this division

Look at those two lines again. `_kernel_end` almost never lands exactly on a page boundary: if it is 0x13468, integer division gives 19, and the loop protects pages 9 through **18**. **Page 19 — holding the last 1,128 bytes of the kernel — is left unprotected**, and the memory manager may hand it to anyone.

If important global variables live in that page, the system overwrites itself. Today the project is lucky: the critical variables fall below the boundary. A few extra lines of code and that luck runs out. The fix is one line — round up — and it is applied in the current source. Part XIII gives the complete diagnosis, with the processor trace.

16.3 And finally, C

```
1 extern kernel_main
2 call kernel_main
```

`extern` tells NASM that the linker will resolve this symbol. `call` pushes the return address and jumps. From that instruction on, we are executing compiled C, with a valid stack, 32 bits and protected mode.

lytlnybl/kernel/kernel_main.c

```
1 void kernel_main(void)
2 {
3     volatile char* vga = (volatile char*)0xB8000;
4     vga[0] = 'C';        /* sign of life: we reached C */
5     vga[1] = 0x02;       /* attribute: green on black */
6 }
```

```

7   vga_text_init(&terminal);
8   vga_text_writeline(&terminal, "Welcome to the lytlnyb1OS!");
9
10  idt_init();
11  init_pmm();
12  init_vmm();
13  init_heap();
14  init_procmem();
15  timer_init(100);
16  keyboard_init();
17  init_tss();
18  fs_manager_init();
19  /* ... */
20 }

```

That initialisation order is not arbitrary: it is exactly the dependency chain from Chapter 6. Interrupts before anything else, because if something fails we want to see the message. Physical memory before virtual, because virtual needs to request pages from it. The heap after virtual, because it needs to map. Processes after the heap, because they allocate structures with `kmalloc`.

Why `volatile`

`volatile` tells the compiler: “this pointer refers to something that can change for reasons you cannot see; do not optimise these accesses away”. Without it, an optimising compiler could delete the write to `0xB8000` entirely, reasoning that nobody reads that variable afterwards. In code that talks to hardware, `volatile` is not optional.

Try it yourself

Boot with `make run`. If a green **P** appears in the top-left corner and then a green **C**, you have passed both milestones: protected mode and entry into C. They are the two signs of life the code prints by hand, before any video driver exists.

In short

The processor is in protected mode, 32-bit, with a GDT holding four flat segments plus the TSS, its own stack, and executing C. We now have a decent programming environment. Next: being able to see what is happening.

PART V

Seeing what happens: VGA text mode

CHAPTER 17

Video memory

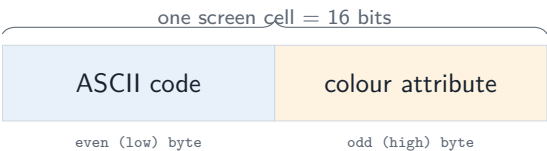
By the end of this chapter you will know

- How the VGA card’s text mode works
- What each byte at 0xB8000 means
- How to compute the address of any position on screen

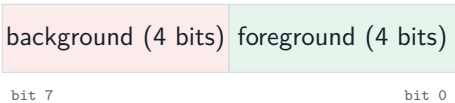
17.1 An 80 by 25 grid

When the PC boots, the graphics card is in **text mode**: it does not display pixels, it displays characters. The screen is a grid of 80 columns by 25 rows, that is 2,000 cells.

Each cell is described by **two bytes** in memory:



And the attribute byte splits again into two 4-bit halves:



Four bits give 16 possible colours, the same ones PCs had in the 1980s:

Value	Colour	Value	Colour
0	Black	8	Dark grey
1	Blue	9	Light blue
2	Green	10	Light green
3	Cyan	11	Light cyan
4	Red	12	Light red
5	Magenta	13	Light magenta
6	Brown	14	Yellow
7	Light grey	15	White

```
lytlnybl/kernel/drivers/vga\_text.c
```

```
1 void vga_text_set_color(vga_text* terminal, vga_color f, vga_color b) {
2     terminal->color = ((uint8_t)b << 4) | (uint8_t)f;
3 }
```

One line that sums up all of Chapter 2: shift the background four bits left to place it in the high half, then combine with OR.

17.2 Computing a cell's address

The 2,000 cells sit in memory one after another, row by row. The cell at row r and column c is at index:

$$\text{index} = \text{row} \times 80 + \text{column}$$

And since each cell occupies 2 bytes, its memory address is $0xB8000 + \text{index} \times 2$. In C this simplifies a lot by declaring the buffer as a pointer to `uint16_t`, because pointer arithmetic then multiplies by 2 for you:

```
lytlnybl/kernel/drivers/vga\_text.c
```

```
1 void vga_text_putchar(vga_text* terminal, char c) {
2     uint8_t color = terminal->color;
3     size_t index = terminal->row * terminal->width + terminal->column;
4     uint16_t entry = ((uint16_t)color << 8) | (uint16_t)c;
5     terminal->buffer[index] = entry;
6 }
```

Four lines and you have screen output. Compare that with what it would cost on a modern graphics system: no driver, no compositor, no fonts. You write two bytes and a letter appears.

Write to the screen by hand

This is exactly what the kernel does the moment it enters C, before any driver exists:

```
1 volatile char* vga = (volatile char*)0xB8000;
2 vga[0] = 'C';    /* character at row 0, column 0 */
3 vga[1] = 0x02;  /* attribute: background 0 (black), foreground 2 (green) */
```

Why this address works directly

`0xB8000` lies below 4 MB, and the system maps the first 4 MB with **identity**: virtual address `0xB8000` corresponds to physical `0xB8000`. That is why the driver can write there without ceremony, even with paging enabled. If the mapping were different, translation would be needed. You will see this in Part VIII.

CHAPTER 18

The text driver

By the end of this chapter you will know

- See how a minimal driver is organised around a state structure
- Understand cursor handling and line wrapping
- Spot a latent divide-by-zero and understand why it is so dangerous

18.1 Terminal state

A driver needs to remember things between calls: where the cursor is and what colour to write in. All of that lives in a structure:

```
lytlnybl/include/kernel/drivers/vga\_text.h
```

```
1 typedef struct {
2     size_t row;          /* current cursor row */
3     size_t column;       /* current column */
4     size_t width;        /* 80 */
5     size_t height;       /* 25 */
6     uint8_t color;       /* current attribute */
7     uint16_t* buffer;    /* 0xB8000 */
8 } vga_text;
```

There is a single global instance, declared in `kernel_main.c` and used across the kernel with `extern`:

```
1 vga_text terminal;      /* in kernel_main.c */
2 extern vga_text terminal; /* in every other file */
```

Passing state explicitly

Notice that every function takes `vga_text* terminal` as its first argument, even though there is only one instance. It is a deliberate design decision: it keeps the driver free of hidden state and would allow several virtual terminals in future without touching a line of the logic. It is a habit worth copying.

18.2 Initialising and clearing

lytlnybl/kernel/drivers/vga_text.c

```

1 void vga_text_init(vga_text* terminal) {
2     terminal->row = 0;
3     terminal->column = 0;
4     vga_text_set_color(terminal, VGA_COLOR_WHITE, VGA_COLOR_RED);
5     terminal->width = 80;
6     terminal->height = 25;
7     terminal->buffer = (uint16_t*)0xB8000;
8     vga_text_clear(terminal);
9 }
10
11 void vga_text_clear(vga_text* terminal) {
12     uint8_t color = terminal->color;
13     uint16_t blank = ((uint16_t)color << 8) | ' ';
14     for (size_t row = 0; row < terminal->height; row++) {
15         for (size_t col = 0; col < terminal->width; col++) {
16             size_t index = row * terminal->width + col;
17             terminal->buffer[index] = blank;
18         }
19     }
20     terminal->row = 0;
21     terminal->column = 0;
22 }

```

“Clearing” the screen means writing 2,000 spaces. There is no “erase” command: the hardware always shows whatever is in those 4,000 bytes.

18.3 Writing a string

lytlnybl/kernel/drivers/vga_text.c

```

1 void vga_text_write(vga_text* terminal, const char* string) {
2     size_t pos = terminal->row * terminal->width + terminal->column;
3
4     for (size_t i = 0; string[i] != '\0'; i++) {
5         if (string[i] == '\n') {
6             vga_text_writeline(terminal, "");
7             continue;
8         }
9         vga_text_putchar(terminal, string[i]);
10        pos++;
11        terminal->row    = pos / terminal->width;
12        terminal->column = pos % terminal->width;
13        terminal->row    = terminal->row % terminal->height;
14    }
15 }

```

The last three lines do all the positioning work with pure arithmetic:

- `pos / 80` gives the row: at position 80 it becomes 1, at 160 it becomes 2...

- `pos % 80` gives the column within that row.
- `row % 25` makes row 24 wrap back to row 0.

Wrapping is not scrolling

That last modulo makes the screen *wrap*: on reaching the bottom it returns to the top and writes over what was there. That is not what a real terminal does, which *scrolls* everything up by one line. The result is that long output ends up jumbled. Adding real scrolling is an excellent exercise: you must move 24 rows with `memcpy` and clear the last one.

18.4 A divide-by-zero waiting to happen

Look at those three lines again. There are three divisions, and all three divide by a field of the structure: `terminal->width` and `terminal->height`.

What happens if width or height is 0

On x86, dividing by zero does not give “infinity” or a strange value: it **raises exception 0**, *Divide By Zero*. The kernel handles it, and its handler does the most natural thing in the world: print an error message.

But printing a message calls `vga_text_writeline`, which calls `vga_text_write`, which divides by zero again. The exception fires again. And again. Each repetition consumes about 150 bytes of kernel stack, and **never returns**.

After 107 rounds — $107 \times 156 \approx 16$ KB, exactly the size of the kernel stack — the stack runs off the bottom of mapped memory, a page fault occurs, the fault handler tries to use the same broken stack, a double fault occurs, and on the third failure the processor gives up and resets the machine. What the user sees is a system that boots, gets to a certain point, and reboots in a loop, with no message at all.

This is not hypothetical: Part XIII contains the real processor trace showing those 107 rounds, and the step-by-step diagnosis down to the root cause. The cause was not in the video driver: it was three subsystems away.

The underlying lesson

A kernel fault rarely shows up where its cause lives. Here, a rounding error in the physical memory manager ended up causing a reboot loop in the video driver. Debugging an operating system means thinking in *chains* of causes, not about the point where it explodes. Part XIII teaches the method.

18.5 Printing numbers

There is no `printf` here, so number-to-text conversion is done by hand:

lytlnybl/kernel/drivers/vga_text.c

```

1 void vga_text_write_dec(vga_text* terminal, uint32_t value) {
2     char buffer[10];
3     size_t digits = 0;
4
5     if (value == 0) {
6         vga_text_putchar(terminal, '0');
7         /* ... advance cursor ... */
8         return;
9     }
10
11     while (value > 0) {
12         buffer[digits++] = '0' + (value % 10); /* least significant digit */
13         value /= 10;
14     }
15     while (digits > 0) { /* print in reverse */
16         vga_text_write_char(terminal, buffer[--digits]);
17     }
18 }

```

The `'0' + digit` trick works because ASCII digits are consecutive: `'0'` is 48, `'1'` is 49, and so on. Adding the numeric value to the code of `'0'` gives the right character.

And it must print in reverse because the algorithm extracts units first, then tens, and so on. Hence storing them in an array and walking it backwards.

In practice: everything the text driver can do

examples/ch18-vga-output/ -- make run

A replacement `kernel_main.c` that exercises every function in the driver: `write` versus `writeline`, decimal and hexadecimal, all sixteen colours, placing a character at an exact cell with `vga_text_put_entry_at`, and reading it back off the screen with `vga_text_getchar`.

Watch out for one real bug it exposes: `vga_text_write_dec(t, 0)` writes the `'0'` with `putchar` but never advances the cursor, so the next thing printed covers it and the zero vanishes. Every other value goes through a loop that does advance it.

Exercise 18.1. Write `vga_text_write_hex`, printing a `uint32_t` in hexadecimal with 8 digits and a `0x` prefix. Hint: instead of `% 10` use `& 0xF` and shift 4 bits; to turn a 0–15 value into a character you will need to distinguish whether it is below 10.

In short

With the video driver working, we can see what the system is doing — which is the single most important debugging tool in this environment. Next comes the most important topic in the course: interrupts.

PART VI

Interrupts: the axis of everything

CHAPTER 19

What an interrupt is

By the end of this chapter you will know

- Why a system without interrupts cannot be an operating system
- The three kinds of interrupt and how they differ
- What the processor does automatically when one fires

19.1 The problem

Suppose the kernel wants to know when a key is pressed. Without interrupts there is only one option: ask, over and over, forever.

```
1 for (;;) {  
2     if (key_available()) handle_key();  
3     if (timer_expired()) switch_process();  
4     if (disk_ready())    finish_read();  
5     /* ... and the real work? there is no room for it ... */  
6 }
```

That loop has two fatal defects. It burns 100 % of the processor asking questions whose answer is almost always “no”. And it only works if the running code cooperates: if a user program enters `while(1);`, the loop never runs again and the machine is lost.

What an interrupt gives you

An interrupt lets the hardware **take the processor away** from whatever is running, without its permission and without its cooperation, and hand it to a kernel routine. That single capability is what makes preemptive multitasking, device handling, error recovery and system calls possible. Everything.

19.2 Three kinds

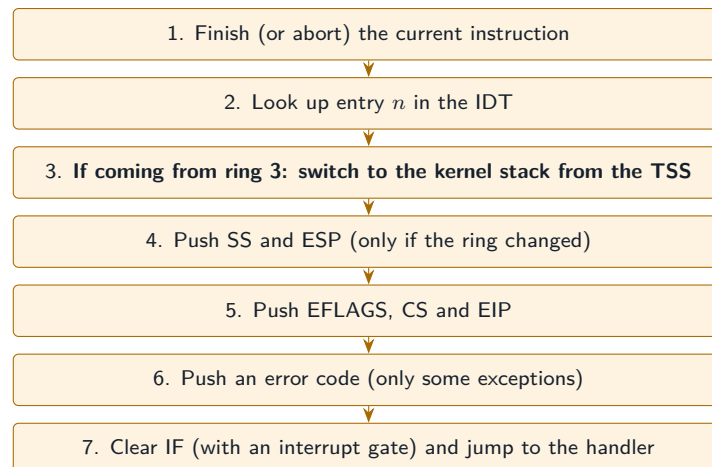
The processor treats all three the same way, but they originate differently:

Kind	Numbers	Origin
Exceptions	0–31	The processor itself, when an instruction cannot complete: divide by zero, invalid opcode, page fault
Hardware interrupts (IRQ)	32–47	An external device raising a line: timer, keyboard, disk
Software interrupts	any, we use 0x80	The program itself, executing <code>int n</code> on purpose

The difference matters for how you respond. An exception means something went wrong *now*: either you fix it (a page fault whose page can be brought in) or you kill the culprit. A hardware interrupt means a device has news. A software interrupt means a program is asking for a service.

19.3 What the processor does by itself

When an interrupt fires, the processor performs a fixed sequence *in hardware*, before a single instruction of yours runs:



Step 3 is the one that makes user space possible, and it is worth dwelling on. If the interrupted code was running in ring 3, its stack is a user stack — untrusted, possibly corrupt, possibly unmapped. The kernel cannot run on it. So the processor automatically switches to a kernel stack whose address it reads from a structure called the TSS. Part X covers this in detail.

Not every exception pushes an error code

Exceptions 8, 10, 11, 12, 13, 14 and 17 push an extra 32-bit value describing what went wrong. The others do not. That inconsistency forces the assembly stubs to push a fake zero for the ones that do not, so that the stack layout is identical in all cases. You will see exactly that trick in Chapter 21.

The IDT: the table of handlers

By the end of this chapter you will know

- What the Interrupt Descriptor Table is and how it is laid out
- How to build a gate descriptor
- What the difference between 0x8E and 0xEE is, and why it matters enormously

20.1 256 entries

The **IDT** is an array of 256 descriptors, one per interrupt number. Each entry says which address to jump to and under what conditions.

lytlnybl/include/kernel/interrupts.h

```
1 typedef struct {
2     uint16_t offset_low;    /* bits 15..0 of the handler address */
3     uint16_t selector;      /* code segment selector: 0x08 */
4     uint8_t reserved;       /* always 0 */
5     uint8_t flags;          /* type and privileges */
6     uint16_t offset_high;   /* bits 31..16 of the handler address */
7 } __attribute__((packed)) idt_entry_t;
```

Note the handler address is split in two halves with other fields in between — the same historical mess as the GDT. Filling it in means splitting the address by hand:

lytlnybl/kernel/interrupts.c

```

1 void idt_set_gate(uint8_t interrupt, uint32_t handler_address,
2                   uint16_t selector, uint8_t flags) {
3     idt[interrupt].offset_low = handler_address & 0xFFFF;
4     idt[interrupt].selector = selector;
5     idt[interrupt].reserved = 0;
6     idt[interrupt].flags = flags;
7     idt[interrupt].offset_high = (handler_address >> 16) & 0xFFFF;
8 }

```

Chapter 2's idioms again: `& 0xFFFF` to keep the low half, `>> 16` to extract the high half.

20.2 The flags byte

Bit	Name	Meaning
7	P	Present
6–5	DPL	Minimum ring required to invoke this interrupt with <code>int</code>
4	S	0 for gates
3–0	Type	0xE = 32-bit interrupt gate, 0xF = trap gate

Which produces the two values used in the project:

Value	Binary	Meaning
0x8E	10001110	Present, DPL 0 , interrupt gate
0xEE	11101110	Present, DPL 3 , interrupt gate

Why the system call gate is 0xEE and everything else 0x8E

The DPL in an IDT entry answers one question: *may a ring 3 program trigger this interrupt deliberately with `int n`?*

For the CPU exceptions and the hardware IRQs the answer must be no. If a user program could execute `int 14` it would fake a page fault and confuse the kernel into examining a **CR2** that has nothing to do with reality. Hence DPL 0, hence 0x8E.

For `int 0x80` the answer must be yes: it is precisely how a program requests a service. Hence DPL 3, hence 0xEE.

That single hex digit is the difference between having system calls and having a completely inaccessible kernel. Getting it wrong in either direction is a classic bug: set them all to 0x8E and every system call becomes a protection fault; set them all to 0xEE and any program can forge exceptions.

lytlnybl/kernel/interrupts.c

```

1 idt_set_gate(0, (uint32_t)isr0, 0x08, 0x8E); /* divide by zero */
2 idt_set_gate(14, (uint32_t)isr14, 0x08, 0x8E); /* page fault */
3 idt_set_gate(32, (uint32_t)irq0, 0x08, 0x8E); /* timer */
4 idt_set_gate(33, (uint32_t)irq1, 0x08, 0x8E); /* keyboard */
5 /* ... */
6 idt_set_gate(0x80, (uint32_t)syscall_entry, 0x08, 0xEE); /* system calls */

```

20.3 Interrupt gate versus trap gate

The only difference: an **interrupt gate** clears **IF** on entry (interrupts disabled inside the handler), a **trap gate** does not.

lytlnyblOS uses interrupt gates for everything, including system calls. That means **the kernel is not reentrant**: while a system call is running, no timer interrupt can fire, so no process switch can preempt it. That simplifies the code enormously — no locks, no races — at the price of latency: a slow system call blocks the whole system.

A consequence you will meet later

Because system calls run with interrupts disabled, the kernel can manipulate process lists, page directories and the TSS without worrying about being interrupted halfway. Several places in this codebase rely on that implicitly. Anyone thinking of switching to trap gates would have to audit all of them first.

20.4 Loading the table

Like the GDT, the IDT is registered through a small descriptor holding base and limit:

```

1 idtr.limit = sizeof(idt) - 1; /* 256 * 8 - 1 = 2047 */
2 idtr.base = (uint32_t)idt;
3 idt_load(&idtr);

```

```

1 idt_load:
2     mov eax, [esp + 4]
3     lidt [eax]
4     ret

```

The same “minus one” as the GDT. And notice `memset(idt, 0, sizeof(idt))` at the start of `idt_init`: the 200-odd unused entries are left at zero, which means “not present”. If any of them ever fires, the processor raises a general protection fault rather than jumping into garbage. Zeroing is what makes that safe.

CHAPTER 21

Interrupt service routines

By the end of this chapter you will know

- Why handlers cannot be written directly in C
- How the assembly stubs build a uniform stack frame
- How `registers_t` maps onto that frame, and why it is the key to everything

21.1 Why C is not enough

An interrupt handler is not a normal function. It must:

- preserve *every* register, because it interrupted arbitrary code at an arbitrary point;
- end with `iret` instead of `ret`, because the stack holds `EIP`, `CS` and `EFLAGS`, not just a return address;
- cope with the fact that some exceptions pushed an error code and others did not.

C can express none of that. So each of the 48 entry points is a small assembly stub that normalises the situation and then calls a C function.

21.2 The two macros

lytlnybl/kernel/interrupts.asm

```
1 %macro ISR_NOERRCODE 1
2 isr%1:
3     push dword 0          ; FAKE error code, so the layout always matches
4     push dword %1         ; the interrupt number
5     jmp isr_common_stub
6 %endmacro
7
8 %macro ISR_ERRCODE 1
9 isr%1:
10    ; the processor already pushed the error code
11    push dword %1         ; just the interrupt number
```

```

12     jmp isr_common_stub
13 %endmacro

```

That fake zero is the whole trick. After either macro runs, the stack looks identical regardless of whether the exception provided an error code — so a single C function can handle all 32.

```

1  ISR_NOERRCODE 0      ; divide by zero: no error code
2  ISR_NOERRCODE 1
3  ...
4  ISR_ERRCODE   8      ; double fault: has an error code
5  ISR_NOERRCODE 9
6  ISR_ERRCODE   10     ; invalid TSS
7  ISR_ERRCODE   11
8  ISR_ERRCODE   12
9  ISR_ERRCODE   13     ; general protection fault
10 ISR_ERRCODE   14     ; page fault

```

21.3 The common stub

lytlnybl/kernel/interrupts.asm

```

1  isr_common_stub:
2      pusha                ; push EAX,ECX,EDX,EBX,ESP,EBP,ESI,EDI in one go
3
4      mov ax, ds
5      push eax             ; save the data segment
6
7      push esp             ; the argument: a pointer to everything we just built
8      call isr_handler
9      add esp, 4           ; discard the argument
10
11     pop eax               ; restore DS (as part of EAX)
12
13     popa                  ; restore the eight registers
14
15     add esp, 8            ; discard interrupt number and error code
16     iret                 ; return: pops EIP, CS, EFLAGS (and SS, ESP if the ring changed)

```

push esp is the most important line in the file

Right before the call, **ESP** points at the lowest byte of everything the stub and the processor have pushed. Pushing **ESP** therefore passes to C **a pointer to the complete saved state of the interrupted code**.

From C, that pointer is a `registers_t*`. And because it points at the real stack, *it is not a copy*: if C modifies a field of that structure, it modifies what `popa` and `iret` will restore. Changing `regs->eip` changes where execution resumes. Changing `regs->eax` changes the value the interrupted code sees in **EAX**.

This is how system calls return a value. And it is how the context switch works: to switch process, the kernel simply overwrites every field of that structure with another process's

saved values, and lets `iret` do the rest.

21.4 The `registers_t` structure

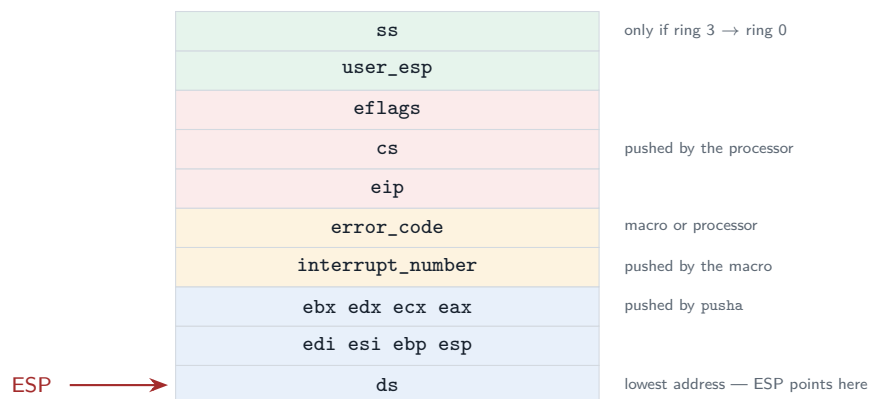
The structure must match the stack layout exactly, and in reverse order — the last thing pushed sits at the lowest address, so it comes first in the struct:

`lytlnybl/include/kernel/interrupts.h`

```

1  typedef struct {
2      uint32_t ds;                /* pushed by the stub */
3
4      uint32_t edi, esi, ebp, esp; /* pushed by pusha */
5      uint32_t ebx, edx, ecx, eax;
6
7      uint32_t interrupt_number; /* pushed by the macro */
8      uint32_t error_code;      /* pushed by the macro or by the CPU */
9
10     uint32_t eip;              /* pushed by the processor */
11     uint32_t cs;
12     uint32_t eflags;
13
14     uint32_t user_esp;          /* only if the ring changed */
15     uint32_t ss;
16 } registers_t;

```



The two last fields are conditional

`user_esp` and `ss` only exist on the stack if the interrupt came from ring 3. If the kernel interrupts itself, the processor does not switch stacks and pushes nothing extra — so those fields of the struct point at whatever happened to be above. That is why the context-saving code checks the process type before touching them:

```

1  if (process->type == PROCESS_KERNEL) {
2      process->regs.esp = regs->esp;    /* from pusha */
3  } else {
4      process->regs.esp = regs->user_esp; /* pushed by the processor */
5      process->regs.ss = regs->ss;

```

```
6 }
```

Getting this wrong produces a bug that only shows up with user processes, and looks like random corruption.

21.5 The C handler

lytlnybl/kernel/interrupts.c

```
1 void isr_handler(registers_t* regs) {
2     switch (regs->interrupt_number) {
3         case 14:
4             page_fault_handler(regs);
5             break;
6         default:
7             vga_text_writeline(&terminal,
8                 exception_messages[regs->interrupt_number]);
9             break;
10    }
11
12    if ((regs->cs & 3) == 3) { /* did it come from user space? */
13        current_process->state = PROCESS_TERMINATED;
14        context_switch(current_process, get_next_process(), regs);
15        return;
16    }
17
18    for (;;) /* kernel fault: stop dead */
19 }
```

The test `(regs->cs & 3) == 3` reads the RPL of the saved code segment: if it is 3, the fault came from a user program. The policy is then clear: kill the program and switch to another. The system survives.

If the fault came from the kernel itself, there is no safe recovery, so it halts in an infinite loop. That is deliberate: a frozen system with a message on screen is far easier to debug than one that keeps running in a corrupt state.

The reboot loop this cannot catch

Notice that the `for(;;)` is only reached *after* printing. If printing itself faults — as in the divide-by-zero scenario from Chapter 20 — the handler never gets there, and the exception recurses until the stack runs out. A production kernel would use a separate, pre-allocated fault stack and a panic routine that writes to screen without calling anything. Part XIII revisits this.

In practice: adding an interrupt vector of your own

```
examples/ch21-custom-interrupt/ -- make run
```

Adds vector `0x30` to the IDT in the three places it has to go: an assembly stub (C cannot end a function with `iret`), a declaration so C can take its address, and the `idt_set_gate` call plus a handler.

Then it fires `int $0x30` with a known value in `EBX`, and the handler overwrites `regs->ebx`. The value that comes back is the handler's, not the caller's — demonstrating on a vector you built yourself the exact mechanism a system call uses to return a result.

Note the handler ends with `return`, not `break`: the code after the switch halts the kernel forever, which is right for a fault and wrong for a deliberate interrupt.

CHAPTER 22

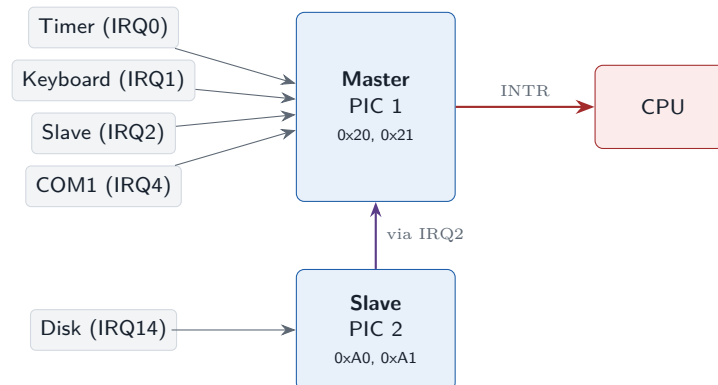
The PIC: routing hardware interrupts

By the end of this chapter you will know

- What the 8259 controller does and why there are two of them
- Why they must be remapped before anything else
- What the End Of Interrupt is and what happens if you forget it

22.1 The chip

Devices do not signal the processor directly: they raise a line on a **Programmable Interrupt Controller** (PIC), which arbitrates between them and tells the CPU which one won. The original PC had one, with 8 lines. The PC/AT added a second chained to line 2 of the first, giving 15 usable lines.



22.2 Why remapping is mandatory

By default, the master PIC sends IRQ0–7 as interrupt numbers 8–15. That was fine in 1981. It is a disaster in protected mode, because **numbers 8–15 are already CPU exceptions**: 8 is double fault, 13 is general protection fault, 14 is page fault.

Without remapping, a simple timer tick would arrive as “double fault”. The kernel would print `Double Fault` a hundred times a second and halt. It is one of the most confusing symptoms a beginner can hit, because the message is completely unrelated to the cause.

The fix is to reprogram the PICs so they use free numbers: 32–39 for the master, 40–47 for the slave.

lytlnybl/kernel/interrupts.c

```
1 pic_remap(0x20, 0x28); /* 32 and 40 */
```

22.3 The remapping sequence

The 8259 is configured by sending four **initialisation command words** in a fixed order:

lytlnybl/kernel/interrupts.c

```
1 void pic_remap(int offset1, int offset2) {
2     uint8_t a1 = inb(PIC1_DATA); /* save the current masks */
3     uint8_t a2 = inb(PIC2_DATA);
4
5     outb(PIC1_COMMAND, ICW1_INIT | ICW1_ICW4); /* ICW1: start init */
6     io_wait();
7     outb(PIC2_COMMAND, ICW1_INIT | ICW1_ICW4);
8     io_wait();
9
10    outb(PIC1_DATA, offset1); /* ICW2: vector offsets */
11    io_wait();
12    outb(PIC2_DATA, offset2);
13    io_wait();
14
15    outb(PIC1_DATA, 4); /* ICW3: slave is on line 2 (bit 2 = 4) */
16    io_wait();
17    outb(PIC2_DATA, 2); /* ICW3: slave identity = 2 */
18    io_wait();
19
20    outb(PIC1_DATA, ICW4_8086); /* ICW4: 8086 mode */
21    io_wait();
22    outb(PIC2_DATA, ICW4_8086);
23    io_wait();
24
25    outb(PIC1_DATA, a1); /* restore the masks */
26    outb(PIC2_DATA, a2);
27 }
```

Three details worth noting:

The `io_wait()` between every write

The 8259 is a chip from 1976. Modern processors issue writes faster than it can process them. That delay is the trick from Chapter 5.

ICW3 is asymmetric

The master is told *which line* the slave is on as a bitmask ($1 \ll 2 = 4$); the slave is told *its own number* as a plain value (2). Same information, two encodings. Pure 1970s design.

Masks are saved and restored

Each PIC has a register saying which lines are enabled. The initialisation sequence would clear it, so the code reads it first and puts it back afterwards.

22.4 EOI: the acknowledgement you must not forget

When a device raises an interrupt, the PIC records that it is *in service* and refuses to deliver any further interrupt of the same or lower priority until it is told the handler is done. That acknowledgement is the **End Of Interrupt**.

lytlnybl/kernel/interrupts.c

```

1 void pic_send_eoi(uint8_t irq) {
2     if (irq >= 8)
3         outb(PIC2_COMMAND, PIC_EOI); /* the slave too, if it came from there */
4         outb(PIC1_COMMAND, PIC_EOI); /* always the master */
5 }
6
7 void irq_handler(registers_t* regs) {
8     switch (regs->interrupt_number - 32) {
9         case 0: timer_handler(regs); break;
10        case 1: keyboard_handler(); break;
11        case 2: break;
12    }
13    pic_send_eoi(regs->interrupt_number - 32);
14 }

```

Forgetting the EOI

It is the classic beginner bug, and its symptom is deeply confusing: **the interrupt fires exactly once and never again**. The keyboard registers one key and goes dead. The timer ticks once and multitasking never starts. Nothing crashes, no error appears — the system simply stops reacting.

Note also that the EOI must always go to the master, and *additionally* to the slave if the IRQ came from there. Sending it only to the slave leaves the master permanently blocked.

Try it yourself

With the timer at 100 Hz, adding a counter that prints something every 100 ticks should print once per second. If it prints once and stops, the EOI is missing or misplaced. If it never prints, the PIC masks are blocking IRQ0 or `sti` was never executed.

22.5 Turning interrupts on

The last line of `idt_init` is:

```
1  asm volatile("sti");
```

`sti` sets `IF`. From this instant the system is interruptible: the timer will start firing, the keyboard will respond, and — crucially — the code that follows can be preempted at any moment.

Everything before this point ran with interrupts off, in a single-threaded world where nothing could go wrong concurrently. Everything after it lives in a world where it can.

In short

The kernel can now react to the outside world. The next part puts that to use with the two drivers that make an interactive system possible: the timer, which gives it a heartbeat, and the keyboard, which gives it ears.

PART VII

Two drivers: heartbeat and ears

CHAPTER 23

The timer

By the end of this chapter you will know

- How the PIT works and how to program a frequency
- Why a system needs a heartbeat
- What a “tick” is and what is measured with it

23.1 Why a timer matters so much

Without a periodic interrupt, an operating system cannot:

- take the processor away from a program that will not give it up,
- measure time at all,
- wake a sleeping process,
- time out a device that stopped responding.

The timer is what turns a passive kernel into an active one. It is the only thing in the system that happens *without anyone asking*.

23.2 The PIT 8253/8254

The chip has an input clock of 1,193,182 Hz — an absurd number that comes from dividing the original PC’s colour television crystal. You cannot choose the frequency directly; you choose a **divisor**, and the chip fires at 1,193,182/divisor Hz.

lytlnybl/kernel/drivers/timer.c

```
1 void timer_init(uint32_t frequency) {
2     freq = frequency;
3     uint16_t divisor = 1193182 / frequency;
4
5     /* tell the PIT how we will send the divisor, and which mode */
6     outb(PIT_COMMAND, PIT_ACCESS_LOHIBYTE | PIT_MODE3 | PIT_CHANNEL0 | PIT_BINARY);
7     io_wait();
```

```

8
9      /* the divisor is 16 bits but the port is 8: send it in two halves */
10     outb(PIT_CHANNEL0_DATA, divisor & 0xFF);      /* low byte first */
11     io_wait();
12     outb(PIT_CHANNEL0_DATA, divisor >> 8);      /* then the high byte */
13 }

```

The project calls `timer_init(100)`, so the divisor is $1,193,182/100 = 11,931$ and the interrupt fires 100 times per second. **One tick equals 10 milliseconds.**

The frequency is not exact

$1,193,182/11,931 = 100.0069$ Hz. The error is tiny but it accumulates: after an hour the clock has drifted by about a quarter of a second. Systems that need accurate time use a different source and periodically correct. For a teaching system, and for scheduling, it is irrelevant.

Choosing the frequency is a trade-off

Higher frequency means finer time resolution and snappier process switching, but more interrupts per second and therefore more overhead: every tick costs a full save and restore of state. Lower frequency is cheaper but makes the system feel sluggish and coarse. 100 Hz is a classic compromise — it was Linux's default for years.

23.3 What happens on every tick

lytlnybl/kernel/drivers/timer.c

```

1  volatile uint32_t ticks = 0;
2
3  void timer_handler(registers_t* regs) {
4      ticks++;
5
6      /* wake any process whose sleep has expired */
7      process_t* traversal_process = process_head;
8      while (traversal_process) {
9          if (traversal_process->state == PROCESS_SLEEPING
10             && ticks >= traversal_process->wake_tick) {
11              traversal_process->state = PROCESS_READY;
12          }
13          traversal_process = traversal_process->next;
14      }
15
16      schedule(regs);      /* maybe switch process */
17 }

```

Three jobs, in order: advance the clock, wake sleepers, and offer the scheduler a chance to switch. That last call is where preemptive multitasking actually happens, and Part IX takes it apart.

Why volatile on ticks

`ticks` is modified inside an interrupt handler and read from ordinary code. Without `volatile`, a loop like `while (ticks < target);` could be optimised into “read once, loop forever” — the compiler has no way to know an interrupt changes it. This is the same reason as the VGA buffer in Chapter 19, from the opposite direction.

23.4 Waiting without a scheduler

Very early in boot, before processes exist, the kernel sometimes needs to wait. It does so crudely:

lytlnybl/kernel/drivers/timer.c

```
1 void timer_wait_ms(uint32_t ms) {
2     uint32_t start = ticks;
3     while ((ticks - start) < ms) {
4         asm volatile ("hlt");
5     }
6 }
```

`hlt` halts the processor until the next interrupt. It is far better than an empty spin loop: it consumes no power and, in a virtual machine, does not peg a host CPU core at 100%. The timer interrupt wakes it 100 times a second, the condition is rechecked, and off it goes again.

The name lies

The parameter is called `ms` but is really counted in ticks. With a 100 Hz timer, one tick is 10 ms, so `timer_wait_ms(10)` waits 100 ms, not 10. It is a real naming bug in the project. Harmless today because it is used once during boot, but exactly the kind of thing that wastes an afternoon later.

In practice: measuring time with the PIT

examples/ch23-timer/ -- make run

Times a piece of work by subtracting two tick readings, then reprograms the PIT to 1000 Hz and times the same work again. The number of ticks changes by a factor of ten while the work does not: the ruler changed, not the thing being measured.

It also demonstrates the naming bug above — `timer_wait_ms(100)` is measured taking 100 ticks, that is a full second, not 100 ms.

The keyboard

By the end of this chapter you will know

- What a scancode is and why it is not ASCII
- How press and release are distinguished
- How modifier keys are tracked
- How a keystroke reaches a blocked user process

24.1 Scancodes are not characters

When you press a key, the keyboard does not send a letter. It sends a **scancode**: a number identifying *a physical key in a physical position*. The key that says A on a UK layout sends the same scancode as the one that says Q on a French layout, because it is the same physical switch. Translating scancode to character is entirely the operating system's job, and it is what makes keyboard layouts possible.



24.2 Press and release

Every key generates *two* events: one when pressed (**make**) and one when released (**break**). The distinction is encoded in bit 7:

Scancode	Bit 7	Meaning
0x1E	0	The A key was pressed
0x9E	1	The A key was released (0x1E + 0x80)

lytlnybl/kernel/drivers/keyboard.c

```

1 if (scancode & KEY_RELEASED) { /* KEY_RELEASED is 0x80 */
2     return; /* we ignore releases, except for modifiers */
3 }

```

24.3 Modifier keys

Shift and Control are different: what matters is not the event but the *state*. The driver keeps a boolean for each and updates it on both press and release:

lytlnybl/kernel/drivers/keyboard.c

```

1 bool keyboard_modifier_keys(uint8_t scancode) {
2     if (scancode == SHIFT_PRESS) {
3         shift_pressed = true;
4         return true;
5     } else if (scancode == (SHIFT_PRESS | KEY_RELEASED)) {
6         shift_pressed = false;
7         return true;
8     }
9
10    if (scancode == L_CTRL_PRESS) {
11        ctrl_pressed = true;
12        return true;
13    } else if (scancode == (L_CTRL_PRESS | KEY_RELEASED)) {
14        ctrl_pressed = false;
15        return true;
16    }
17
18    return false;
19 }

```

Returning true means “I handled this, produce no character”. Two keymap tables then decide the result:

```

1 char c[2];
2 c[0] = keymap[scancode];
3 c[1] = '\0';
4 /* ... */
5 if (shift_pressed) {
6     c[0] = shift_keymap[scancode];
7 }

```

Extended scancodes

Arrow keys, the right-hand Control, Home, End and others send a 0xE0 prefix followed by the real code. The driver detects the prefix and reads a second byte, but currently does nothing with it:

```

1 void keyboard_extended_scancodes() {
2     uint8_t scancode = inb(PS2_DATA);

```

```

3     if (scancode == 0x5B) { }    /* left GUI key: recognised, ignored */
4 }

```

Implementing arrow keys — and with them command history and line editing in the shell — starts right here. It is one of the most rewarding extensions to attempt.

24.4 From keystroke to blocked process

This is the most interesting part of the driver, because it is where two subsystems meet. When a user program calls `read(0, buffer, n)`, the kernel cannot answer immediately — nobody has typed anything yet. So it *blocks* the process:

lytlnybl/kernel/interrupts.c

```

1 case SYSCALL_READ:
2     /* ... */
3     current_process->state = PROCESS_BLOCKED;
4     current_process->reading_state.buffer = (void*)regs->ecx;
5     current_process->reading_state.size  = regs->edx;
6     current_process->reading_state.count = 0;
7     context_switch(current_process, get_next_process(), regs);
8     break;

```

The process is parked with a note attached: *where* its buffer is, *how big* it is, and *how much* has been filled so far. Then the kernel switches to another process. The blocked one is simply never scheduled.

Later, a key arrives and the keyboard handler walks the process list looking for anyone waiting:

lytlnybl/kernel/drivers/keyboard.c

```

1 process_t* traversal_process = process_head;
2 while (traversal_process) {
3     if (traversal_process->state == PROCESS_BLOCKED
4         && traversal_process->reading_state.size > 0) {
5
6         void* buffer = traversal_process->reading_state.buffer;
7         uint32_t size = traversal_process->reading_state.size;
8
9         /* a rather crude way of doing this (really only the context switcher
10          * should be changing cr3), but it works */
11         set_cr3((uintptr_t)traversal_process->page_directory);
12         ((char*)(buffer))[traversal_process->reading_state.count++] = c[0];
13         set_cr3((uintptr_t)kernel_directory);
14
15         if (traversal_process->reading_state.count >= size || c[0] == '\n') {
16             traversal_process->regs.eax = traversal_process->reading_state.count;
17             traversal_process->reading_state.count = 0;
18             traversal_process->reading_state.size = 0;
19             traversal_process->state = PROCESS_READY;    /* wake it up */
20         }
21     }
22     traversal_process = traversal_process->next;
23 }

```

Why the CR3 dance

The buffer address (say `0xBFFFEF00`) is a *virtual* address, and it only means anything inside that process's own address space. The keyboard interrupt, however, can arrive while a completely different process is running — so `CR3` points somewhere else and that address means something else entirely, or nothing at all.

So the handler temporarily switches `CR3` to the sleeping process's page directory, writes the byte, and switches back. It works because the kernel is identity-mapped into every process's directory, so the code keeps executing correctly throughout.

The comment in the source is honest about this being crude, and it is right. A cleaner design keeps a kernel-side ring buffer, has the interrupt write only there, and copies into user memory later, when that process is the one actually running. That removes the `CR3` manipulation from interrupt context entirely.

Where the return value comes from

Look at `traversal_process->regs.eax = count;`. The count is written into the *saved* register state of the sleeping process. Later, when the scheduler picks it, `load_context` copies that into the `registers_t` on the stack, and `iret` restores it into the real `EAX`.

The user program called `read()` and sees a number come back in `EAX` — with no idea that between the call and the return, seconds passed, other programs ran, and an interrupt filled its buffer. That illusion is the whole point of an operating system.

The buffer is never NUL-terminated

The kernel fills `count` bytes and returns `count`. It never writes a `'\0'`. That is correct behaviour — it matches Unix `read()` — but the shell then does:

```
1 char input[50];
2 read(0, input, 50);
3 input[strlen(input) - 1] = '\0';
```

`strlen` on a buffer that was never terminated, and whose stack memory was never initialised, walks until it happens to find a zero. It works in practice because a freshly allocated stack page is zeroed, but it is reading uninitialised memory by luck. The correct form uses the return value:

```
1 int n = read(0, input, sizeof(input) - 1);
2 if (n > 0 && input[n-1] == '\n') n--;
3 input[n] = '\0';
```

Part XIII lists this among the fixes worth making.

24.5 Special keys

lytlnybl/kernel/drivers/keyboard.c

```

1  switch (c[0]) {
2      case 27:                /* Escape */
3          vga_text_clear(&terminal);
4          break;
5      case '\t':
6          vga_text_write(&terminal, "    ");
7          break;
8      case '\b':
9          vga_text_backspace(&terminal);
10         /* deliberate fall-through */
11     case '\n':
12     default:
13         /* ... echo and deliver to the blocked process ... */
14         break;
15 }

```

The missing `break` after backspace is intentional: backspace must erase on screen *and* still travel down to the process so it can shorten its buffer. The code below guards with `if (c[0] != '\b')` to avoid echoing it twice.

It is correct, but it is exactly the kind of fall-through that a compiler warns about (`-Wimplicit-fallthrough`) and that a reader assumes is a bug. A one-line comment saying “intentional” would save the next person several minutes.

Try it yourself

Boot the system and press Escape at the shell prompt. The screen clears. That path goes: key → PIC → IRQ1 → `irq_handler` → `keyboard_handler` → `vga_text_clear`. Five subsystems for one keystroke, and all of them are yours.

In practice: reading a line properly

examples/ch24-keyboard-input/ -- make run

A user program that reads a line, trims the newline, terminates it, and prints it back. Four lines of code, and they are the four the shell gets wrong.

It reads `sizeof(buf) - 1` to leave room for the terminator, uses the *return value* rather than `strlen` on an unterminated buffer, and only then writes the `'\0'`. Compare it with the shell’s version and the difference is the whole bug.

In short

The system has a heartbeat and it has ears. It can react to time and to the user. What it still cannot do is manage memory properly — and without that, there are no real processes and no isolation. That is the next part, and the most important one in the course.

PART VIII

Memory management

The physical memory manager

By the end of this chapter you will know

- Why memory is handed out in fixed-size chunks
- What a bitmap allocator is and how it works
- How the kernel protects itself from its own allocator
- The rounding bug that makes it fail, and why it is so hard to spot

25.1 Pages and frames

The first decision any memory manager makes is the size of the unit it deals in. lytlnybIOS, like every x86 system, uses **4096 bytes**. The hardware forces this: the paging unit we will meet in the next chapter works in 4 KB granules and nothing else.

The vocabulary matters and people mix it up constantly:

Term	Meaning
Frame	A 4 KB chunk of <i>physical</i> RAM. A real, existing piece of hardware
Page	A 4 KB chunk of <i>virtual</i> address space. A name that may or may not refer to anything

The physical manager in this chapter deals only with *frames*. It answers one question, and nothing else: *which 4 KB pieces of real RAM are in use?*

An analogy

Think of a hotel. The frames are the rooms: they physically exist, there is a fixed number of them, and each is either occupied or free. The physical manager is the receptionist with the occupancy board. It does not know who is in which room, what they are doing, or why — only occupied or free.

The next chapter introduces room numbers that guests see, which need not match the real ones. That is virtual memory.

25.2 A bitmap

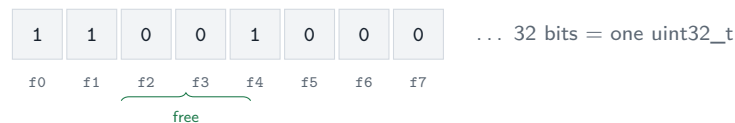
With 128 MB of RAM there are $128 \times 1024 / 4 = 32,768$ frames. How do you track 32,768 yes/no facts? With 32,768 bits, that is 4 KB total. That is a **bitmap**: one bit per frame, 1 = used, 0 = free.

lytlnybl/memory/pmm.c

```
1 uint32_t* bitmap = (uint32_t*)BITMAP_BASE;  /* BITMAP_BASE is 0x100000 */
2
3 void bitmap_set_frame(uint32_t frame_index) {
4     bitmap[frame_index / 32] |= (1 << (frame_index % 32));
5 }
6
7 void bitmap_clear_frame(uint32_t frame_index) {
8     bitmap[frame_index / 32] &= ~(1 << (frame_index % 32));
9 }
```

The bitmap is an array of `uint32_t`, so each element holds 32 frames. To reach frame i :

- $i / 32$ says which array element,
- $i \% 32$ says which bit within it,
- $1 \ll (i \% 32)$ builds a mask with only that bit set.



25.3 Allocating and freeing

lytlnybl/memory/pmm.c

```
1 void* alloc_frame() {
2     uint32_t frame_i;
3     for (frame_i = 0; frame_i < total_frames; frame_i++) {
4         uint32_t is_reserved = (bitmap[frame_i / 32] & (1 << (frame_i % 32)));
5         if (!is_reserved) {
6             bitmap_set_frame(frame_i);
7             return (void *) (frame_i * FRAME_SIZE);
8         }
9     }
10    return NULL;  /* out of memory */
11 }
12
13 void free_frame(void* frame_address) {
14     uint32_t frame_i = (uint32_t) frame_address / FRAME_SIZE;
15     bitmap_clear_frame(frame_i);
16 }
```

This is **first fit**: scan from the beginning and take the first free frame. Simple, correct, and $O(n)$ — with 32,768 frames the loop can run 32,768 times for a single allocation.

Why it is fast enough here, and would not be in general

Because allocations are rare (a few at boot, a handful per process) and because free frames tend to be near the start, the loop usually exits after a few iterations. A real system keeps a free list, a pointer to the last-found position, or a multi-level bitmap. Worth knowing the limit exists, not worth fixing here.

`free_frame` validates nothing

It does not check that the address is page-aligned, that it is within range, or that the frame was actually allocated. `free_frame((void*)0x1001)` silently clears the bit for frame 1, and `free_frame` called twice on the same frame hands it out to two owners. In a kernel, that is the seed of arbitrarily strange bugs appearing arbitrarily far away.

25.4 Initialisation: deciding what is off limits

This is the delicate part. The manager starts by marking *everything* reserved and then frees only what the BIOS map says is usable:

lytlnybl/memory/pmm.c

```

1  /* mark every region reserved, for safety */
2  for (uint32_t i = 0; i < bitmap_entries; i++) {
3      bitmap[i] = 0xFFFFFFFF;
4  }
5
6  /* then free the usable regions */
7  for (uint32_t i = 0; i < map_entry_count; i++) {
8      if (memory_map[i].type == 1) {          /* usable RAM */
9          uint64_t starting_frame = memory_map[i].base / FRAME_SIZE;
10         uint64_t frame_length   = memory_map[i].length / FRAME_SIZE;
11         for (uint32_t j = starting_frame; j < starting_frame + frame_length; j++) {
12             bitmap_clear_frame(j);
13         }
14     }
15 }

```

Deny by default

Starting with everything reserved and opening up only what is explicitly known to be safe is a security principle worth internalising. The opposite — start everything free and mark the bad parts — fails catastrophically the moment you forget one region, and it fails silently.

Then come the specific protections: the BIOS area, the kernel itself, the bitmap, the boot sector

and video memory.

```

1  bitmap_set_frame(0);                                /* BIOS data */
2
3  uint32_t kernel_frame_index = (uint32_t)&_kernel_start / FRAME_SIZE;
4  uint32_t kernel_frame_end   = (uint32_t)&_kernel_end   / FRAME_SIZE;
5  for (; kernel_frame_index < kernel_frame_end; kernel_frame_index++) {
6      bitmap_set_frame(kernel_frame_index);           /* the kernel */
7  }
8
9  uint32_t bitmap_frame_start = BITMAP_BASE / FRAME_SIZE;
10 uint32_t bitmap_size_bytes  = bitmap_entries * sizeof(uint32_t);
11 uint32_t bitmap_frame_count = (bitmap_size_bytes + FRAME_SIZE - 1) / FRAME_SIZE;
12 for (uint32_t i = 0; i < bitmap_frame_count; i++) {
13     bitmap_set_frame(bitmap_frame_start + i);       /* the bitmap itself */
14 }
15
16 bitmap_set_frame(0x7C00 / FRAME_SIZE);             /* the boot sector */
17
18 uint32_t video_frame_index = 0xA0000 / FRAME_SIZE;
19 uint32_t video_frame_end   = 0xFFFFF / FRAME_SIZE;
20 for (; video_frame_index < video_frame_end; video_frame_index++) {
21     bitmap_set_frame(video_frame_index);           /* video memory */
22 }

```

25.5 The rounding bug

Look carefully at two lines above. The bitmap-size calculation uses the round-up idiom:

```

1  uint32_t bitmap_frame_count = (bitmap_size_bytes + FRAME_SIZE - 1) / FRAME_SIZE;

```

The kernel-end calculation does not:

```

1  uint32_t kernel_frame_end = (uint32_t)&_kernel_end / FRAME_SIZE;

```

The consequence, with real numbers

In one build of this project, `_kernel_end` is 0x13468.

$$0x13468 \div 0x1000 = 19.27 \rightarrow 19 \text{ (integer division truncates)}$$

The loop condition is `index < 19`, so it protects frames 9 through **18**: addresses 0x9000 to 0x12FFF. **Frame 19 — 0x13000 to 0x13FFF — is left free**, even though the last 1,128 bytes of the kernel live in it.

What lives in those 1,128 bytes? In that build: `idtr`, `terminal`, `keymap`, `kernel_directory`, `current_directory`, `heap_start_head`, `open_files`, `process_head`, `current_process` and the `tss`. Essentially every critical global in the kernel.

So `alloc_frame()` eventually returns 0x13000, the loader copies a user program's code into it, and the kernel's own state is destroyed. `terminal.width` and `terminal.height` become

whatever bytes of the shell binary landed there — possibly zero. And a zero width means the divide-by-zero cascade from Chapter 20.

The fix is one line, and it is **now applied in the source tree**:

```
1 uint32_t kernel_frame_end = ((uint32_t)&_kernel_end + FRAME_SIZE - 1) / FRAME_SIZE;
```

Why this bug is so nasty

It is **latent**. Whether it bites depends on exactly where `_kernel_end` falls and which globals end up in the last partial page — which changes every time you add or remove code. A build that works perfectly can break because you added a comment that changed a string constant’s length.

That is the worst class of bug: one where “it works on my machine” and “it works on the same machine yesterday” are both true, and both meaningless. Part XIII documents how it was actually found.

In practice: asking for and returning physical frames

```
examples/ch25-physical-memory/ -- make run
```

Allocates three frames, prints their physical addresses, fills one with `memset` to prove it is real memory, frees the middle one and allocates again — getting the same frame straight back, because the allocator is first fit.

The addresses it prints are worth reading: the third frame is not adjacent to the second, because the frame holding the boot sector at `0x7C00` is protected and gets skipped.

Exercise 25.1. The region between `0x1000` and `0x8FFF` is marked usable by the BIOS map and is never explicitly protected. The E820 memory map itself was stored at `0x5000`. Is that a bug? Justify your answer by considering *when* the map is read for the last time.

CHAPTER 26

Virtual memory, slowly

By the end of this chapter you will know

- Understand, with no prior knowledge, what virtual memory is and what problem it solves
- Follow an address translation by hand, with real numbers
- Understand why there are two levels of table and not one
- Know what every bit of a page table entry means

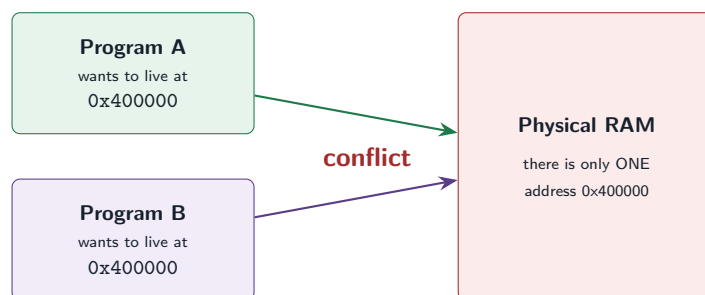
This is the hardest chapter in the course and the one everything else rests on. It is written to be read slowly. If a section does not click, do not push on — reread it. Nothing later makes sense without this.

26.1 The problem, told as a story

Suppose we have two programs and we want to run both.

Program A was compiled to live at address `0x400000`. Its code says things like “jump to `0x4001A0`” and “the variable `counter` is at `0x400C20`”. Those addresses are baked into the machine code at compile time.

Program B was compiled the same way — because the compiler always puts programs at `0x400000`. Its code also says “jump to `0x4001A0`”.



There is only one physical address `0x400000`. Both cannot have it.

26.1.1 Three bad solutions

Recompile every program for a free address

Then you cannot run the same binary twice, and you must know at compile time what else will be running. Absurd.

Relocate the code at load time

Rewrite every address in the binary as you load it. This was actually done historically. It is slow, error-prone, and requires the binary to carry a table of every address to patch.

Ask programs to be polite and avoid each other

This requires trusting every program on the machine. One buggy program takes down everything. This is real mode, and it is why real mode is useless.

26.1.2 The good solution

Give each program **its own private set of addresses**, and have the hardware translate those private addresses into real ones on every single access.

Program A says 0x400000 and the hardware quietly turns it into physical 0x800000. Program B says 0x400000 and the hardware quietly turns it into physical 0xC00000. Neither program knows. Neither program can find out. Neither program can reach the other's memory, because *there is no address it can write down* that leads there.

The central idea of this chapter

A **virtual address** is a name. A **physical address** is a place. Virtual memory is a **dictionary from names to places**, and each program gets its own dictionary.

Two programs can use the same name because their dictionaries are different. A program cannot reach another's memory because its dictionary has no entry leading there. Isolation is not enforced by checking — it is enforced by *the absence of a translation*.

Back to the hotel

Every guest is handed a map on which their room is labelled “Room 1”. Guest A's Room 1 is really room 314; guest B's Room 1 is really room 502. Both can say “I'm going to Room 1” and both arrive somewhere different, without confusion and without ever meeting.

If guest A tries to walk to “Room 7” and their map has no Room 7, they do not end up in someone else's room — they end up in front of a wall. The hotel manager (the kernel) gets a call about it. That call is a *page fault*.

26.2 Why a dictionary of every address is impossible

The obvious implementation would be a table with one entry per address: 4 billion entries. At 4 bytes each that is 16 GB of table, to manage 4 GB of memory. Obviously absurd.

The fix is to translate in **blocks**, not individual bytes. If we translate in 4 KB blocks, we need only $4\text{ GB}/4\text{ KB} = 1,048,576$ entries. At 4 bytes each: 4 MB of table.

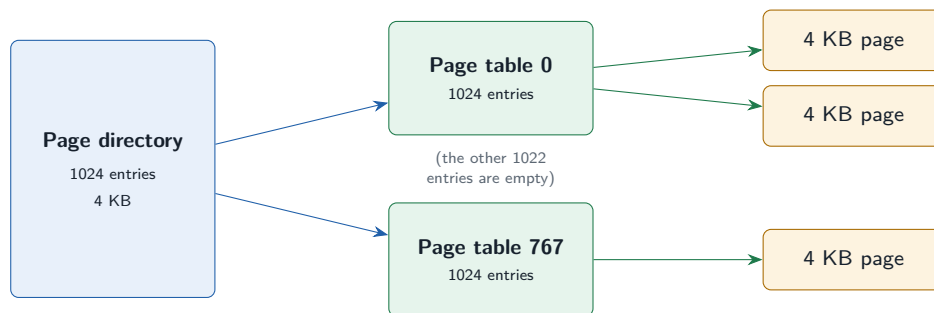
Better — but still 4 MB *per process*. Ten processes would be 40 MB of pure bookkeeping, most of it describing addresses nobody uses.

26.2.1 The trick: split the table in two

Instead of one table of 1,048,576 entries, use:

- one small table of **1024** entries, called the **page directory**;
- where each entry points to another table of **1024** entries, called a **page table**;
- where each of *those* entries describes one 4 KB page.

$1024 \times 1024 = 1,048,576$. The coverage is identical. But the saving is enormous, because **a directory entry can be empty**. A process that uses 8 KB of code and 4 KB of stack needs a directory (4 KB) and one or two page tables (4–8 KB) — about 12 KB, not 4 MB.



Where 1024 and 4096 come from

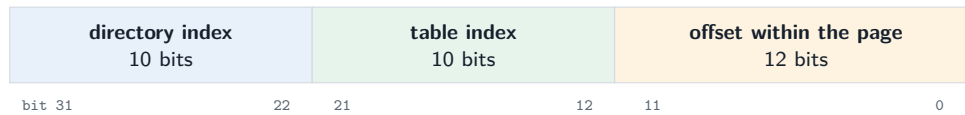
These numbers are not arbitrary; they fall out of the arithmetic:

- A page is 4096 bytes. To address a byte inside it you need $\log_2 4096 = 12$ bits.
- That leaves $32 - 12 = 20$ bits to name the page.
- Split those 20 evenly: 10 bits and 10 bits. And $2^{10} = 1024$ — the size of each table.
- Each entry is 4 bytes, so a table of 1024 entries is exactly $1024 \times 4 = 4096$ bytes: one page. The tables fit in the very thing they describe.

The whole design closes on itself. That elegance is why it has survived since 1985.

26.3 Splitting an address

Every 32-bit virtual address is read by the hardware as three fields:



And the code that does the splitting is exactly the idiom from Chapter 2:

```
lytlnybl/memory/vmm.c
1 uint16_t dir_index = (virtual_address >> 22);
2 uint16_t table_index = (virtual_address >> 12 & 0x3FF);
3 /* the offset is virtual_address & 0xFFF */
```

26.3.1 A worked example, digit by digit

Let us translate the virtual address 0x00401A2C completely by hand. This is the kind of address a user program actually uses in this system.

Step 1 — write it in binary.

0x00401A2C is:

0000 0000 0100 0000 0001 1010 0010 1100

Step 2 — cut it into 10, 10 and 12 bits.

0000000001	0000000110	100000101100
directory = 1	table = 6	offset = 0xA2C = 2604

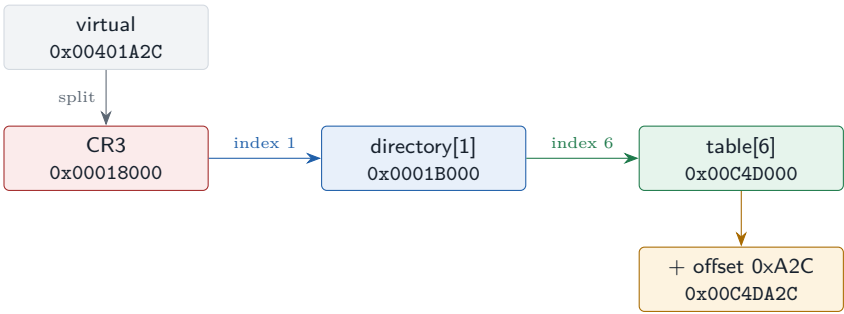
Check it with arithmetic instead of bit-counting:

$\text{directory} = 0x00401A2C \gg 22 = 4,201,004 \div 4,194,304 = 1$
 $\text{table} = (0x00401A2C \gg 12) \& 0x3FF = 1030 \& 1023 = 6$
 $\text{offset} = 0x00401A2C \& 0xFFF = 0xA2C = 2604$

Step 3 — the hardware walks the tables.

1. Read CR3. Say it holds 0x00018000. That is the physical address of this process's page directory.
2. Go to entry 1 of that directory: physical address $0x18000 + 1 \times 4 = 0x18004$. Suppose it contains 0x0001B007.
3. Mask off the flags: $0x0001B007 \& 0xFFFFF000 = 0x0001B000$. That is the physical address of the page table.
4. Go to entry 6 of that table: $0x1B000 + 6 \times 4 = 0x1B018$. Suppose it contains 0x00C4D007.
5. Mask again: 0x00C4D000. That is the physical address of the *frame*.
6. Add the offset: $0x00C4D000 + 0xA2C = 0x00C4DA2C$.

So virtual 0x00401A2C is physical 0x00C4DA2C. The program has no idea. It never will.



This happens on every single access

Not once per program. Not once per function. **Every time the processor reads or writes memory** — every instruction fetch, every variable access, every stack push — it performs this two-step lookup.

If it really did two extra memory reads per access, computers would run three times slower. They do not, because of a cache called the **TLB** (Translation Lookaside Buffer) that remembers recent translations. The TLB is why the next section matters.

26.3.2 A second example: a translation that fails

The first example worked. Now let us take an address in the *same* address space that does not resolve, because understanding failure is what makes the mechanism click.

We will translate 0x00500000 — five megabytes in — inside the shell’s address space.

Step 1 — split it.

$$\begin{aligned} \text{directory} &= 0x00500000 \gg 22 = 1 \\ \text{table} &= (0x00500000 \gg 12) \& 0x3FF = 1280 \& 1023 = 256 \\ \text{offset} &= 0x00500000 \& 0xFFF = 0 \end{aligned}$$

Step 2 — the directory lookup succeeds.

Directory entry 1 is present: it covers the whole 4 MB region 0x00400000–0x007FFFFFFF, and 0x500000 falls inside it. So far, so good. The hardware reads the entry, masks off the flags, and walks to the page table.

Step 3 — the table lookup fails.

Entry 256 of that table is **zero**. Bit 0 — the present bit — is clear. The address has a table to look in, and the table has nothing to say about it.

The hardware stops here. It does not guess, it does not return zero, it does not read whatever happens to be at physical 0x500000. It raises **exception 14**, and before jumping to the handler it leaves two pieces of evidence:

Where	Value	Meaning
CR2	0x00500000	The address that was being translated
error code	0x4 or 0x6	bit 0 clear: <i>not present</i> ; bit 2 set: from ring 3; bit 1 set if it was a write

So a user program reading that address produces error code 0x4, and writing to it produces 0x6. Run it through the handler from Chapter 28 and you get:

```
PAGE FAULT
Address: 0x00500000
Error code: 0x00000006
Reason: Page not present
Access: Write
Mode: User
```

Two ways to fail, and they mean different things

Notice there are two distinct failures hiding here, and the error code distinguishes them:

- **Bit 0 clear — not present.** There is no translation. The name means nothing in this address space. This is a wild pointer, a stack that grew too far, or memory that was never allocated.
- **Bit 0 set — protection violation.** There *is* a translation, but the access was not allowed: writing to a read-only page, or a ring 3 program touching a page whose `PAGE_USER` bit is clear. The page exists; you just may not do that to it.

The second kind is how the kernel stays protected. When a user program tries to read kernel memory, the translation succeeds — the kernel *is* mapped into its address space — and it is the permission check that stops it, producing error code 0x5 (present + user). A program cannot tell the difference between “kernel memory” and “memory that does not exist” except by the error code it never gets to see.

Why an unmapped address is not always an error

In a full operating system, most page faults are not bugs. The page might be on disk (swapped out), or it might be a stack page the kernel deliberately did not allocate until first touched, or a copy-on-write page being written for the first time. In all of those cases the handler fixes the situation, and then *re-executes the faulting instruction*, which now succeeds. The program never finds out.

lytlnyblOS does none of this: every fault is fatal to the process that caused it. But the mechanism is identical, and Part XIII lists on-demand stack growth as one of the most rewarding extensions to attempt — it is about fifteen lines, and it turns this exact failure into a feature.

26.4 Why a page is 4096 bytes

The number looks arbitrary. It is not, and seeing why fixes the whole design in your mind.

Page size is a three-way trade-off, and 4 KB is close to the sweet spot for a 32-bit machine.

26.4.1 The cost of a small page

Halve the page to 2 KB and you need twice as many entries to describe the same memory. Worse, the arithmetic stops being neat: 2048 bytes needs 11 offset bits, leaving 21 to split between two levels — which do not divide evenly, and whichever way you cut it the tables no longer fit in one page.

Take it down to 1 KB and a two-level split gives 11 and 11 bits: tables of 2048 entries, 8192 bytes each. A page table now spans *eight* pages, which the allocator has to find contiguously. The elegance is gone.

26.4.2 The cost of a large page

Go up to 16 KB and the tables shrink to 512 entries — 2 KB, wasting most of a 16 KB page each. And now **internal fragmentation** bites: every allocation wastes on average half a page.

Page	Offset bits	Split	Table size	Average waste per allocation
1 KB	10	11 + 11	8 KB (8 pages!)	512 B
4 KB	12	10 + 10	4 KB (exactly 1 page)	2 KB
16 KB	14	9 + 9	2 KB (1/8 of a page)	8 KB

4 KB is the size where the design closes on itself

Only at 4096 bytes does everything line up:

- 12 offset bits leave exactly 20, which split evenly into 10 + 10;
- $2^{10} = 1024$ entries, at 4 bytes each, is exactly 4096 bytes;
- so **a page table is exactly one page**.

That last point is what makes the implementation so short. `create_page_table` can just call `alloc_frame()` and use the result directly — no special allocator, no contiguity problem, no leftover space. Look at how little code it takes:

```

1 page_table_t* create_page_table(page_directory_t* directory,
2                                 uint32_t directory_index, uint32_t flags) {
3     page_table_t* new_table = alloc_frame();
4     memset(new_table, 0, 4096);
5     (*directory)[directory_index] = (uintptr_t)new_table | flags;
6     return new_table;
7 }
```

Four lines, because the page size was chosen so that four lines would do.

And the reason modern systems also offer huge pages

Each TLB entry covers one page. With 4 KB pages, a program touching 1 GB of memory needs 262,144 distinct translations, and the TLB holds a few hundred — so it misses constantly.

This is why x86 also supports 2 MB and 1 GB pages. One TLB entry then covers 2 MB, and the same program needs 512 translations instead of 262,144. The cost is fragmentation: allocate a 2 MB page for a 4 KB structure and you have wasted 2,093,056 bytes.

Databases and virtual machines use huge pages for exactly this reason. lytlnyblOS does not — the bit that enables them lives in the directory entry, and turning it on is a small but genuinely instructive change.

26.5 The TLB and why stale entries kill

The TLB caches virtual → physical translations. It makes paging affordable. It also means that **if you change a page table, the processor may not notice**, because it is still using a cached answer.

Two instructions fix that:

lytlnybl/memory/vmm.asm

```

1 flush_tlb:                ; throw away EVERYTHING
2     mov eax, cr3
3     mov cr3, eax          ; reloading CR3 with itself clears the whole TLB
4     ret
5
6 flush_tlb_page:           ; throw away ONE entry
7     mov eax, [esp + 4]
8     invlpg [eax]
9     ret

```

invlpg invalidates a single page's cached translation, which is far cheaper than dumping the whole TLB. The mapping code uses it:

lytlnybl/memory/vmm.c

```

1 (*selected_table)[table_index] = physical_address | flags;
2 if (directory == current_directory) {
3     flush_tlb_page(virtual_address);
4 }

```

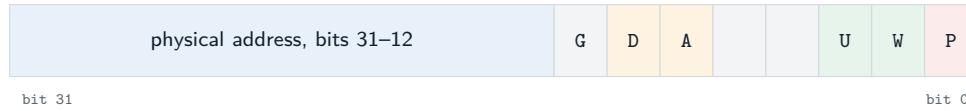
Note the condition

The flush only happens if we are modifying the *currently active* directory. Modifying another process's directory needs no flush, because the TLB holds nothing from it — and when that process is scheduled, CR3 is reloaded, which clears the TLB anyway.

Forgetting a flush produces one of the most maddening bugs in systems programming: you map a page, the table clearly contains the right entry, you can print it and see it — and the access still faults, because the processor is answering from a cache you cannot see.

26.6 Anatomy of a table entry

Each entry is 32 bits. The top 20 hold a physical address (which is enough, because everything is page-aligned so the low 12 bits are always zero). Those freed-up 12 bits carry the flags:



Bit	Name	Constant	Meaning
0	P	PAGE_PRESENT	The page exists. If 0, any access raises a page fault
1	W	PAGE_WRITABLE	Writable. If 0, the page is read-only
2	U	PAGE_USER	Ring 3 may access it. If 0, kernel only
5	A	—	Accessed; set by the processor on any access
6	D	—	Dirty; set by the processor on a write

```
lytlnybl/include/memory/vmm.h
```

```
1 #define PAGE_PRESENT 0x1
2 #define PAGE_WRITABLE 0x2
3 #define PAGE_USER 0x4
```

Bit 2 is where user isolation actually lives

Everything in Part X about user space comes down to this bit. A kernel page has `PAGE_USER` clear; a user page has it set. When a ring 3 program touches an address whose page lacks that bit, the processor raises a page fault *before* the access happens. No data leaks. No check in software. One bit, enforced in silicon, on every access.

And the corollary: if you accidentally set `PAGE_USER` on a kernel page, you have handed user programs the keys, and nothing will warn you.

Permissions are checked at both levels

The directory entry *also* has W and U bits, and the effective permission is the *stricter* of the two. A page marked writable in the table but read-only in the directory is read-only. Forgetting this leads to faults that make no sense until you remember to look one level up.

26.7 A real address space, captured live

Everything so far has been described. This section shows the actual bytes, read out of a running lytlnyblOS while the shell sat at its prompt.

How this was captured

QEMU's monitor can read guest memory. Started with `-monitor unix:/tmp/mon.sock,server,nowait`, the commands are `memsave` (virtual addresses, using the current CPU state) and `pmemsave` (physical addresses).

The kernel's `process_head` global is at `0x13400` in this build — find it with `i686-elf-nm build/kernel.elf`. Reading four bytes there gives the head of the process list; walking `next` at offset 96 and reading `page_directory` at offset 100 gives each process's directory, which `pmemsave` can then dump in full.

26.7.1 The process list

```
process_head -> 0xc000000c
pid=1  KERNEL  RUNNING  pd=0x00001000  cs=0x08  eip=0x0000b022  esp=0x00012998
pid=2  KERNEL  RUNNING  pd=0x00001000  cs=0x08  eip=0x0000ac08  esp=0x00012534
pid=3  USER    BLOCKED  pd=0x00019000  cs=0x1b  eip=0x004010f2  esp=0xbffff9c
```

Read that carefully, because several things from earlier chapters are visible at once:

- `process_head` is `0xC000000C` — in the kernel heap, twelve bytes past `0xC0000000`, which is exactly `sizeof(heap_header_t)`. It is the very first `kmalloc` the system ever performs.
- Both kernel processes share `pd=0x00001000`: the one kernel directory. The shell has its own at `0x19000`.
- The shell's `cs` is `0x1B`. That is `0x18 | 3` — the user code selector with RPL 3. **This is the number that proves it is running unprivileged.**
- Its `eip` is `0x4010F2`: inside the code mapped at `0x400000`. Its `esp` is `0xBFFFF9C`: just under `0xBFFFF000`, the stack top.
- Its state is `BLOCKED` — it is waiting in `read()` for a keystroke, exactly as Chapter 24 described.

26.7.2 The shell's page directory

Dumping all 4096 bytes at physical `0x19000` and listing only the non-zero entries gives this — **five entries out of 1024**:

```
dir[ 0] = 0x00002023  table@0x00002000  flags=0x023  covers 0x00000000-0x003fffff
dir[ 1] = 0x0001f027  table@0x0001f000  flags=0x027  covers 0x00400000-0x007fffff
dir[ 2] = 0x0001d007  table@0x0001d000  flags=0x007  covers 0x00800000-0x00bfffff
dir[767] = 0x0001b027  table@0x0001b000  flags=0x027  covers 0xbfc00000-0xbfffffff
dir[768] = 0x00004023  table@0x00004000  flags=0x023  covers 0xc0000000-0xc03fffff
```

This one listing is the entire argument for two-level tables. A flat table would need 1,048,576 entries and 4 MB. This process describes its whole world with a 4 KB directory and four 4 KB tables: **20 KB instead of 4 MB**, because 1019 of the 1024 entries are simply zero.

Decoding the flags with the table from the previous section:

Entry	Flags	Decoded	What it is
dir[0]	0x023	P, W, A	The identity-mapped kernel. No U bit
dir[1]	0x027	P, W, U, A	User code at 0x400000
dir[2]	0x007	P, W, U	User heap region. No A bit — never touched yet
dir[767]	0x027	P, W, U, A	User stack
dir[768]	0x023	P, W, A	The kernel heap. No U bit

The U bit, doing its job, visible in real data

Three of these regions are reachable from ring 3 and two are not, and the only difference is bit 2. The kernel is *present* in this address space — it has to be, or interrupts could not be handled — but dir[0] and dir[768] lack the U bit, so the shell cannot read a single byte of it. Not because anything checks: because the hardware refuses.

That is the whole of user-space isolation, and here it is as two hexadecimal digits differing by 4.

The accessed bit tells you what has actually run

dir[2] has no A bit, while every other entry does. The processor sets A on first access, so this says the shell *has never touched its heap*.

And that is exactly right: the shell blocks in read() on its very first loop iteration, before it ever reaches tokenize_line, which is where the first calloc happens. The hardware is reporting, accurately, that a line of C has not been reached yet. It is a small thing, but it is a good demonstration that these bits are real and live.

26.7.3 Inside the tables

Dumping the four tables and listing the non-zero entries:

```
table for dir[1] (0x00400000) : 2 entries
[  0] 0x0001e027 -> phys 0x0001e000 [P,W,U,A]   virt 0x00400000
[  1] 0x00020027 -> phys 0x00020000 [P,W,U,A]   virt 0x00401000

table for dir[2] (0x00800000) : 1 entry
[512] 0x0001c007 -> phys 0x0001c000 [P,W,U]     virt 0x00a00000

table for dir[767] (0xbfc00000) : 1 entry
[1022] 0x0001a067 -> phys 0x0001a000 [P,W,U,A,D] virt 0xbfffe000

table for dir[0] (0x00000000) : 1024 entries, contiguous identity map
[  0] 0x00000003 -> phys 0x00000000 [P,W]       virt 0x00000000
[  1] 0x00001063 -> phys 0x00001000 [P,W,A,D]   virt 0x00001000
...
[1023] 0x003ff003 -> phys 0x003ff000 [P,W]       virt 0x003ff000
```

Every number here can be predicted from the code, which is the best possible confirmation that you have understood it:

Two code pages, not one

`rootfs/bin/shell` is 4,392 bytes, and $\lceil 4392/4096 \rceil = 2$. That is precisely the loop bound in `load_program`. The two frames, `0x1E000` and `0x20000`, are *not adjacent* — `0x1F000` went to the page table itself, allocated in between. Physical scattering, virtual contiguity: the entire point of paging, visible in three numbers.

The heap page is at table index 512

`USER_HEAP_START` is `0xA00000`. $(0xA00000 \gg 12) \& 0x3FF = 512$. It is the only mapped page in that 4 MB region, which is why the region shows one entry out of 1024.

The stack page is at index 1022

`USER_STACK_TOP` is `0xBFFFF000` and `create_uprocess` maps `USER_STACK_TOP - 4096 = 0xBFFFE000`. $(0xBFFFE000 \gg 12) \& 0x3FF = 1022$. And the saved `esp` was `0xBFFFEF9C`, which is inside that page — 3,996 bytes into it, with 100 bytes of stack used.

The stack page is the only dirty one

Flags `0x067` include `D`. The processor sets `D` on the first *write*. Code pages are read and executed but not written; the heap has not been touched; only the stack has been written to. The hardware has independently confirmed what the program does.

And the one-page stack limit, in the same listing

That table has exactly one entry. The next page down, `0xBFFFD000` at index 1021, is zero. So the moment `ESP` drops below `0xBFFFE000` — 4,096 bytes of stack — the translation fails exactly like the `0x500000` example earlier, and the process dies. The saved `esp` shows 100 bytes in use, so there are 3,996 bytes of headroom. A local array of 4 KB, or roughly forty nested calls, and the shell is gone.

This is the limitation described in Chapter 33, and here is the measurement that proves it.

26.8 The same address in two address spaces

The promise at the start of this chapter was that two programs could both use `0x400000` without colliding. The dumps let us check the claim directly.

In the *shell's* directory, entry 1 is present and maps `0x400000` to physical `0x1E000`. In the *kernel's* directory — dumped separately from physical `0x1000` — entry 1 is:

```
dir[0] = 0x00002023    (the identity map)
dir[1] = 0x00000000    <- nothing
```

The same number, two different truths

`0x400000` is a perfectly valid address in the shell's world: it holds the first instruction the shell executes. In the kernel's world the identical number means nothing at all — touching it raises a page fault.

Nothing about the address changed. What changed is which dictionary was open, and `CR3` is what selects the dictionary. **Changing `CR3` changes what every address in the machine means**, in one instruction.

That is why `set_cr3` appears in `context_switch`, and why the keyboard handler has to change it before writing into a sleeping process's buffer. And it is why running a second copy of a program needs no relocation, no cooperation and no trust: the loader allocates fresh frames, writes them into *that* process's table at the same virtual address, and the two programs are now mutually invisible while both believing they live at `0x400000`.

With one deliberate exception

Look again at `dir[0]` in both directories: **both are `0x00002023`, pointing at the same page table at physical `0x2000`**.

That is not a coincidence and not a copy — it is literally the same table, shared. It is what `create_uprocess` does when it copies all 1024 directory entries from the kernel directory, and it is why the kernel appears at the same addresses in every process.

It is also why the bug in `destroy_process` described in Chapter 32 is serious: calling `unmap_page` on `kernel_directory` edits that shared table, so the hole it punches appears in every address space in the system at once.

In practice: mapping, aliasing, translating and unmapping

```
examples/ch27-paging/ -- make run
```

The whole of this chapter as running code. It maps a frame at `0x40000000` — chosen because it is above the identity map, below the kernel heap and outside every user region — writes through the virtual address and reads the value back through the physical one, then maps a *second* virtual address onto the same frame and shows a write through one appearing through the other.

It then splits the address by hand, compares against `get_physical_address`, unmaps, and builds a complete new address space with the kernel copied in. Along the way it demonstrates that `get_physical_address` never checks the present bit, so it keeps reporting an address for a page you have just unmapped.

Exercise 26.1. The virtual address `0xC0000000` is the start of the kernel heap. Split it into its three fields by hand and state which directory entry it uses. (Check: the answer is a well-known round number, and it explains why `0xC0000000` was chosen rather than, say, `0xC0000400`.)

Paging in practice

By the end of this chapter you will know

- Read the real mapping code line by line
- Understand identity mapping and why the kernel needs it
- See how a process gets its own address space

27.1 Turning it on

lytlnybl/memory/vmm.c

```

1 void init_vmm() {
2     kernel_directory = alloc_frame();
3     page_table_t* tbl_ptr = alloc_frame();
4
5     memset(kernel_directory, 0, 4096);
6     memset(tbl_ptr, 0, 4096);
7
8     /* map the first 4 MB with identity: virtual X -> physical X */
9     for (uint32_t i = 0; i < 1024; i++) {
10         (*tbl_ptr)[i] = (i * 0x1000) | PAGE_PRESENT | PAGE_WRITABLE;
11     }
12
13     (*kernel_directory)[0] = ((uintptr_t)tbl_ptr) | PAGE_PRESENT | PAGE_WRITABLE;
14
15     current_directory = kernel_directory;
16     set_cr3((uintptr_t)kernel_directory);
17     set_cr0(); /* enable paging: the moment of truth */
18 }

```

The loop fills entry i with physical address $i \times 4096$. So entry 0 maps virtual 0x0000–0x0FFF to physical 0x0000–0x0FFF, entry 1 maps 0x1000 to 0x1000, and so on for 1024 entries: exactly 4 MB.

Why identity mapping is necessary, not just convenient

Think about what happens the instant `set_cr0()` executes. Before it, addresses were physical. After it, they are virtual. But **EIP** does not change — the very next instruction is fetched

from the same numeric address it would have been anyway.

If that address did not map to itself, the processor would fetch the next instruction from somewhere completely different, and the system would die instantly and inexplicably.

Identity mapping the region containing the kernel makes the transition a non-event. The kernel sits at 0x9000, the stack, the bitmap at 0x100000 and the video buffer at 0xB8000 are all below 4 MB, so all of them keep working unchanged.

Only 4 MB is identity-mapped

`alloc_frame()` hands out frames from anywhere in physical RAM — but the kernel can only *reach* the first 4 MB. If an allocation ever returns a frame above 0x400000, the `memset` in `create_page_table` or the `memcpy` in the program loader will touch an unmapped address and fault.

It does not happen today because `alloc_frame` scans from frame 0 and the system uses only a few dozen frames. But it is a ceiling, not a safety property. Any change that increases memory usage — more processes, a bigger heap, larger programs — walks towards it. The proper fixes are either to identity-map more of RAM, or to give the kernel a temporary mapping window it can point at any frame.

27.2 Mapping a page

lytlnybl/memory/vmm.c

```

1 void map_page(page_directory_t* directory, uintptr_t virtual_address,
2               uintptr_t physical_address, uint32_t flags) {
3     uint16_t dir_index = (virtual_address >> 22);
4     uint16_t table_index = (virtual_address >> 12 & 0x3FF);
5
6     uint32_t dir_entry = (*directory)[dir_index];
7     page_table_t* selected_table;
8
9     if (!(dir_entry & PAGE_PRESENT)) {
10         /* no table for this 4 MB region yet: create one */
11         selected_table = create_page_table(directory, dir_index, flags);
12     } else {
13         selected_table = (page_table_t*)(dir_entry & 0xFFFFF000);
14     }
15
16     (*selected_table)[table_index] = physical_address | flags;
17
18     if (directory == current_directory) {
19         flush_tlb_page(virtual_address);
20     }
21 }

```

This is the function that does everything. Split the address, find or create the table, write the entry, flush if needed.

A subtlety in `create_page_table`

Notice that the same `flags` are used for both the page table entry and the directory entry. If you map a user page with `PAGE_USER`, the newly created directory entry also gets `PAGE_USER` — which is what you want here. But it means the *first* page mapped into a 4 MB region decides the directory-level permissions for that whole region. Map a kernel-only page first and a user page second into the same region, and the user page will be unreachable from ring 3 because the directory entry lacks the U bit. This is a real trap.

27.3 A process's own address space

```
lytlnybl/tasks/procman.c
```

```
1 void create_uprocess(process_t* new_process) {
2     new_process->page_directory = (page_directory_t*)alloc_frame();
3     memset((page_directory_t*)new_process->page_directory, 0, 4096);
4
5     /* copy every kernel mapping */
6     for (uint32_t i = 0; i < 1024; i++) {
7         (*new_process->page_directory)[i] = (*kernel_directory)[i];
8     }
9
10    /* then add what is private to this process */
11    uintptr_t ustack_frame = (uintptr_t)alloc_frame();
12    map_page(new_process->page_directory, USER_STACK_TOP - 4096, ustack_frame,
13            PAGE_PRESENT | PAGE_WRITABLE | PAGE_USER);
14
15    map_page(new_process->page_directory, USER_HEAP_START,
16            (uintptr_t)alloc_frame(),
17            PAGE_PRESENT | PAGE_WRITABLE | PAGE_USER);
18    /* ... */
19 }
```

Why the kernel is mapped into every process

It looks wasteful and even dangerous to copy the kernel's mappings into every user address space. It is neither, and it is what every real operating system does.

The reason is the interrupt. When a user process makes a system call or the timer fires, the processor switches to ring 0 — but **it does not change CR3**. The kernel therefore begins executing with the *user's* page directory still active. If the kernel's code were not mapped there, the very first instruction of the handler would fault.

Safety comes from the `PAGE_USER` bit: those kernel mappings lack it, so they are present but unreachable from ring 3. The kernel is visible to the hardware and invisible to the program.

The tables are shared, not copied

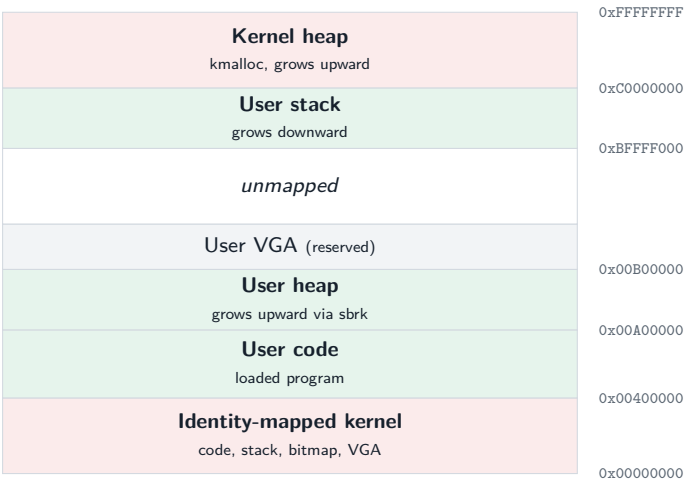
The copy loop copies *directory entries*, which are pointers. Every process therefore points at the *same* kernel page tables. That is intentional and efficient — a change to a kernel mapping is instantly visible everywhere.

But it also means that unmapping something from `kernel_directory` affects every process simultaneously. There is a place in `destroy_process` that does exactly that, and Part XIII explains why it is a bug.

The source code even carries a warning about this:

```
1  /* if you ever try to create more mappings and get a general protection fault
2   * it is likely that you have overwritten an existing kernel mapping, as we copy
3   * the kernel mappings to the user process for when we use our interrupts, just
4   * be careful of this */
```

27.4 The address space, laid out



Note the deliberate design: the user stack grows down from 0xBFFFFFF000 and the user heap grows up from 0xA00000. They approach each other with a large gap between. If they ever met, one would silently corrupt the other — and nothing in this system checks for that.

CHAPTER 28

Page faults

By the end of this chapter you will know

- What a page fault is and why it is not always an error
- How to decode the error code
- How to read a fault and find the cause

28.1 The most useful exception

A **page fault** (exception 14) fires whenever the processor cannot complete a translation. In a mature operating system that is often *normal* and even desirable: it is how demand paging, swapping, copy-on-write and lazy stack growth are implemented. The kernel fixes the situation and resumes the instruction as if nothing happened.

lytlnyblOS does none of that. Here a page fault always means a bug. But its handler is still the single most informative diagnostic in the system.

28.2 Two sources of information

CR2

The processor puts *the address that was accessed* here. This is the first thing to look at.

The error code

Pushed on the stack, it describes *what kind* of access failed.

Bit	If 0	If 1
0	The page was not present	Protection violation (page existed, access not allowed)
1	It was a read	It was a write
2	From the kernel (ring 0)	From a user program (ring 3)
3	—	A reserved bit was set in a table entry
4	—	It was an instruction fetch

lytlnybl/memory/vmm.c

```

1 void page_fault_handler(registers_t* registers) {
2     uintptr_t address = get_cr2();
3     uint32_t error_code = registers->error_code;
4
5     vga_text_writeline(&terminal, "PAGE FAULT");
6
7     vga_text_write(&terminal, "Address: ");
8     vga_text_write_hex(&terminal, address);
9     vga_text_writeline(&terminal, "");
10
11     if (error_code & PRESENT_FAULT)
12         vga_text_writeline(&terminal, "Reason: Protection violation");
13     else
14         vga_text_writeline(&terminal, "Reason: Page not present");
15
16     if (error_code & WRITE_FAULT) vga_text_writeline(&terminal, "Access: Write");
17     else                          vga_text_writeline(&terminal, "Access: Read");
18
19     if (error_code & USER_FAULT) vga_text_writeline(&terminal, "Mode: User");
20     else                          vga_text_writeline(&terminal, "Mode: Kernel");
21
22     vga_text_write(&terminal, "Instruction address: ");
23     vga_text_write_hex(&terminal, registers->eip);
24     vga_text_writeline(&terminal, "");
25 }

```

28.3 Reading a fault

The combination of address, reason and mode usually identifies the bug immediately:

Address	Reason	Almost certainly
0x00000000 or near	not present	A null pointer dereference
Just below a stack region	not present	Stack overflow
A kernel address, mode User	protection	A user program reaching where it should not
A page you just mapped	not present	A missing TLB flush
An address in a mapped page	protection	Writing to a read-only page, or missing PAGE_USER

The stack overflow signature

This one is worth memorising because it is the most common serious fault in a kernel. You see a fault at an address *just below* the bottom of a stack region, with Mode: `Kernel` and usually Access: `Write`, and the faulting instruction is a push or a function prologue.

Example from this project: a fault at 0xBFFFFFFC with ESP = 0xC0000000. Since 0xC0000000 is the start of the kernel heap, a stack had grown downwards right off the bottom of the heap's first page. That single observation identified the entire failure chain described in Part XIII.

In practice: reading a page fault report

```
examples/ch28-page-fault/ -- make run
```

Three lines that write to 0x50000000, which nothing maps. The comment above the write predicts the report — address, reason, access, mode, and an error code of 0x2 — before you run it, so you can check your understanding against the processor’s own answer.

The kernel heap

By the end of this chapter you will know

- Why pages are not enough and a byte allocator is needed
- How a linked-list heap works
- Understand splitting and merging of blocks
- Know the limitations of this implementation

29.1 Why pages are not enough

The physical manager hands out 4 KB at a time. But a `process_t` structure is about 100 bytes. Giving it a whole page wastes 97 % of it, and with a few dozen structures you have burned megabytes on nothing.

The **heap** sits on top: it asks the page manager for pages, and hands out *bytes* from within them.

29.2 The design: a linked list of blocks

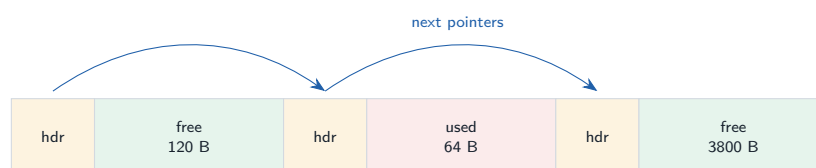
Every block of the heap is preceded by a small header:

lytlnybl/include/memory/heap.h

```

1 typedef struct heap_header {
2     size_t size;           /* usable bytes in this block */
3     bool free;             /* is it available? */
4     struct heap_header* next; /* the next block */
5 } heap_header_t;

```



The header lives *immediately before* the memory it describes. That is why `kmalloc` returns a pointer past the header, and `kfree` works backwards to find it:

lytlnybl/memory/heap.c

```
1 return (void*)((uintptr_t)block + sizeof(heap_header_t)); /* in kmalloc */
2
3 heap_header_t* get_header(void* ptr) { /* in kfree */
4     return (heap_header_t*)((uintptr_t)ptr - sizeof(heap_header_t));
5 }
```

An analogy

Every block is a parcel with a label stuck to the front. The label says how big the parcel is, whether it is empty, and where the next parcel is. When you are handed a parcel you are given the address of the *contents*; to read the label you step backwards by the label's width.

And if you scribble past the end of your parcel, you write on the next parcel's label — which is why heap overflows corrupt the allocator itself rather than just neighbouring data.

29.3 Allocating

lytlnybl/memory/heap.c

```
1 void* kmalloc(size_t requested_size) {
2     if (!requested_size) return NULL;
3
4     heap_header_t* block = find_free_block(requested_size);
5     if (block) {
6         split_block(block, requested_size);
7         block->free = false;
8         return (void*)((uintptr_t)block + sizeof(heap_header_t));
9     } else {
10        expand_heap(requested_size);
11        return kmalloc(requested_size); /* try again */
12    }
13 }
```

29.3.1 Splitting

If a 4000-byte block is used to satisfy a 100-byte request, the remaining 3900 bytes should not be lost. `split_block` cuts it in two:

```
1 void split_block(heap_header_t* block, size_t requested_size) {
2     size_t space_remaining = (block->size - requested_size);
3     if (space_remaining < sizeof(heap_header_t) + sizeof(uint8_t)) {
4         return; /* not worth splitting: the remainder could not even hold a header */
5     }
6     /* ... create a new header in the middle and link it in ... */
7 }
```

That guard matters: a remainder smaller than a header plus one byte would produce a block that can never be used, and whose header would overlap the next block.

29.3.2 Freeing and merging

```

1 void merge_blocks(heap_header_t* block) {
2     if (!block->next || !block->next->free) return;
3     heap_header_t* block_to_merge = block->next;
4
5     block->size += block_to_merge->size + sizeof(heap_header_t);
6     block->next = block_to_merge->next;
7     return merge_blocks(block);    /* keep going while the next is free */
8 }
9
10 void kfree(void* ptr) {
11     if (!ptr) return;
12     heap_header_t* block_to_free = get_header(ptr);
13     block_to_free->free = true;
14     merge_blocks(block_to_free);
15 }

```

Merging matters because without it the heap fragments: you end up with plenty of free bytes, none of them adjacent, and a 200-byte request fails surrounded by free memory.

It only merges forwards

`merge_blocks` joins a block with the ones *after* it. If you free a block whose *predecessor* is already free, they stay separate — the list is singly linked, so there is no way to look back.

The classic fix is a doubly linked list, or storing a copy of the size at the end of each block (a “footer”) so the previous block’s header can be found by arithmetic. As it stands, the heap fragments more than it needs to.

29.4 Growing

```

1 void expand_heap(size_t required_size) {
2     uint32_t required_pages = (required_size + 4096 - 1) / 4096;
3     for (size_t i = 0; i < required_pages; i++) {
4         void* new_page_physical_start = alloc_frame();
5         map_page(kernel_directory, heap_end,
6                 (uintptr_t)new_page_physical_start,
7                 PAGE_PRESENT | PAGE_WRITABLE);
8         /* ... append or extend the last block ... */
9         heap_end += 0x1000;
10    }
11 }

```

There is the round-up idiom again, this time used correctly. And there is the dependency chain from Chapter 6 made concrete: the heap calls `map_page` (virtual memory), which calls

`alloc_frame` (physical memory). Three layers, each built on the one below.

No alignment

`kmalloc(1)` followed by `kmalloc(4)` returns a 4-byte block at an odd address. On x86 unaligned access works, just slower; on many other architectures it faults outright. A production allocator rounds every request up to 8 or 16 bytes. Worth knowing if you ever port this.

In practice: `kmalloc`, `kfree`, and watching the block list

```
examples/ch29-heap/ -- make run
```

Allocates two blocks, then prints the whole block list after every operation, so you can watch splitting happen and then watch the forward-only merge leave a hole behind.

The first block it shows is not one of ours: it is the 112-byte `process_t` that `init_procman` allocated during boot — the very first `kmalloc` the system performs.

It also demonstrates something easy to trip over when writing your own examples: the file keeps the full initialisation sequence even though it only uses the heap, because `idt_init` enables interrupts, the timer then fires, and the scheduler walks a process list that `init_procman` has to have built first.

Try it yourself

The kernel exercises the heap during boot:

```
1 uint32_t* numbers = (uint32_t*)kmalloc(5 * sizeof(uint32_t));
2 for (int i = 0; i < 5; i++) numbers[i] = (i + 1) * 10;
3 /* ... print them ... */
4 kfree(numbers);
```

If you see `Values: 10 20 30 40 50` on screen, the whole three-layer stack works: physical manager, paging and heap.

In short

Memory is solved: the kernel can find free RAM, give each program its own private view of it, and hand out individual bytes. Everything needed for real processes is now in place. That is the next part.

PART IX

Processes and scheduling

What a process is

By the end of this chapter you will know

- The difference between a program and a process
- What context is and why it must be saved completely
- What information the process control block holds

30.1 Program versus process

A **program** is a passive thing: bytes on a disk. A **process** is a program *running*, plus everything that describes the state of that execution.

Run the same program twice and you get two processes. They share not a single byte of state: two different stacks, two different heaps, two different sets of register values, two different positions in the code. The program is a recipe; the process is a particular meal being cooked.

Context

The **context** of a process is everything that must be saved and restored so that it can be stopped and resumed without noticing. For a 32-bit x86 process that is:

- the eight general-purpose registers,
- **EIP**: where it was executing,
- **EFLAGS**: its flag state,
- **ESP** and **SS**: its stack,
- **CS** and **DS**: its segments, which encode its privilege,
- **CR3**: its address space.

Forget *any* of these and the process resumes subtly wrong — a comparison behaves backwards, a pointer aims at nothing, a return goes to the wrong place. Context switching bugs are notoriously hard to find precisely because the symptom appears long after the cause.

30.2 The process control block

All of that lives in one structure, conventionally called the PCB:

lytlnybl/include/tasks/procman.h

```

1  typedef struct process {
2      uint32_t pid_waiting_for;  /* pid of a child we are waiting on */
3      uint32_t parent_pid;
4
5      uintptr_t user_heap_end;   /* for sbrk */
6      uint32_t heap_pages_allocated;
7
8      uint32_t wake_tick;       /* for sleep */
9      process_reading_state_t reading_state; /* for a blocked read */
10
11     uint32_t pid;
12     process_registers_t regs;  /* THE CONTEXT */
13
14     process_states_t state;
15     process_type_t type;       /* kernel or user */
16
17     struct process* next;      /* singly linked list */
18
19     page_directory_t* page_directory;
20
21     void* kstack;              /* kernel stack */
22     void* ustack;              /* user stack */
23 } __attribute__((packed)) process_t;

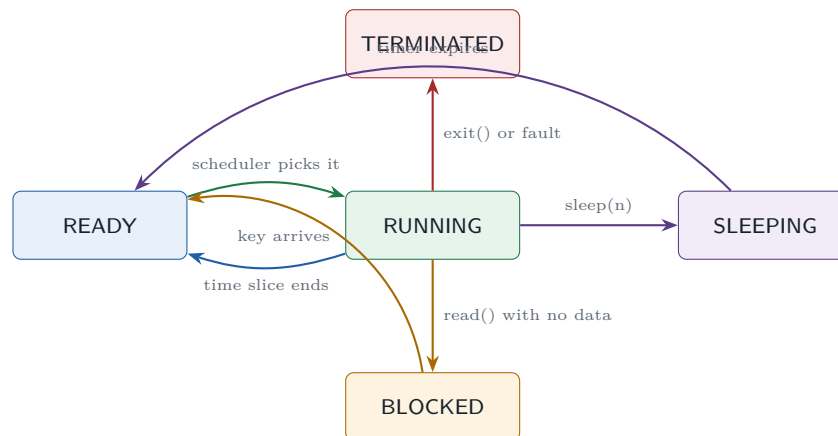
```

packed does not belong here

`__attribute__((packed))` is correct on structures that mirror a hardware layout — the TSS, GDT entries, filesystem records. `process_t` is purely internal: nothing outside the kernel ever sees its byte layout.

Packing it forces unaligned field access, which is slower on x86 and would fault outright on ARM or MIPS. It also makes taking a pointer to a field technically undefined behaviour. Removing `packed` here would be a small, safe improvement.

30.3 The five states



State	Meaning
RUNNING	Executing right now. Exactly one process is in this state
READY	Could run; waiting for its turn
BLOCKED	Waiting for an event: a keystroke, a child exiting
SLEEPING	Waiting for a specific time to arrive
TERMINATED	Finished; its resources have not been reclaimed yet

The distinction between BLOCKED and SLEEPING is worth noting: a sleeping process is woken by the timer, which knows exactly when; a blocked process is woken by whichever subsystem it was waiting on, which does not know when. They are handled in different places — sleeping in `timer_handler`, blocked in `keyboard_handler` or in the `exit` system call.

30.4 Creating a process

lytlnybl/tasks/procman.c

```

1 process_t* create_process(void* task_address, process_type_t type) {
2     process_t* new_process = kmalloc(sizeof(process_t));
3
4     new_process->wake_tick = 0;
5     new_process->type = type;
6     new_process->pid = next_pid++;
7     new_process->state = PROCESS_READY;
8     new_process->kstack = kmalloc(KERNEL_STACK_SIZE); /* 16 KB */
9     new_process->next = NULL;
10
11     new_process->regs.eax = 0; /* ... all registers zeroed ... */
12     new_process->regs.eip = (uintptr_t)task_address;
13     new_process->regs.eflags = 0x202; /* interrupts enabled! */
14
15     if (type == PROCESS_KERNEL) create_kprocess(new_process);
16     else create_uprocess(new_process);
17
18     /* append to the end of the list */

```

```

19  process_t* traversal_process = process_head;
20  while (traversal_process) {
21      if (!traversal_process->next) {
22          traversal_process->next = new_process;
23          break;
24      }
25      traversal_process = traversal_process->next;
26  }
27  return new_process;
28  }

```

The process has not run yet, but its context already exists

This is the elegant part. The new process is given a *fabricated* context: registers at zero, **EIP** at its entry point, flags with interrupts on, a fresh stack pointer.

Nothing distinguishes that fabricated state from one produced by genuinely interrupting a running process. So the scheduler needs no special “start” path: it simply restores this context like any other, **iret** runs, and the process begins. **Starting a process and resuming one are the same operation.**

30.4.1 Kernel processes versus user processes

```

1  void create_kprocess(process_t* new_process) {
2      new_process->regs.esp = (uint32_t)(new_process->kstack) + KERNEL_STACK_SIZE;
3      new_process->regs.cs = 0x08;    /* kernel code, ring 0 */
4      new_process->regs.ds = 0x10;    /* kernel data */
5      new_process->regs.ss = 0x10;
6      new_process->page_directory = kernel_directory; /* shares the kernel's space */
7  }

```

A kernel process runs in ring 0, in the kernel’s own address space, on a heap-allocated stack. It is essentially a thread. Note **esp** points at the *end* of the allocation, because the stack grows down.

A user process, by contrast, gets its own page directory, its own stack mapped at **0xBFFFFFF00**, its own heap, and ring 3 selectors. Part X covers it.

In practice: creating kernel processes

```
examples/ch30-kernel-process/ -- make run
```

Creates two kernel processes that do nothing but increment a counter, then prints both counters six times from the main kernel process. All three advance, and none of them yields: the timer takes the CPU away regardless. That is **preemption**, in about twenty lines.

The comments state the two rules for a kernel process: it must never return, because there is no return address on its stack, and it cannot call **sleep()** or **exit()**, because those are system calls and it is already inside the kernel.

Context switching

By the end of this chapter you will know

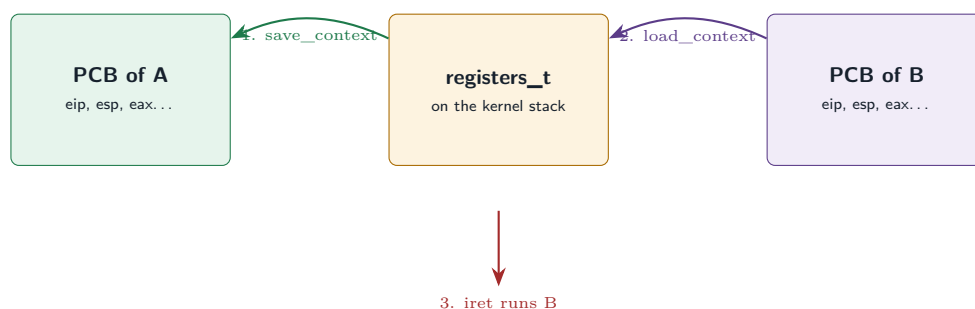
- Understand how a process is swapped for another
- See why the interrupt stack frame is the key to the whole mechanism
- Know what the TSS has to do with it

31.1 The idea

Recall from Chapter 21 that the interrupt stub passes `C` a pointer to the interrupted code's complete saved state, and that `iret` restores whatever that structure contains.

A context switch therefore needs no special machinery at all:

1. Copy the current contents of `registers_t` into the current process's PCB.
2. Copy the *new* process's PCB into that same `registers_t`.
3. Change `CR3` to the new process's page directory.
4. Return normally. `iret` resumes the new process.



31.2 The code

lytlnybl/tasks/context.c

```

1 void context_switch(process_t* old_process, process_t* new_process,
2                     registers_t* regs) {
3     current_process = new_process;
4
5     if (old_process->state == PROCESS_RUNNING) {
6         old_process->state = PROCESS_READY;
7     }
8
9     save_context(old_process, regs);
10    new_process->state = PROCESS_RUNNING;
11
12    return_to_user = (new_process->type == PROCESS_USER);
13
14    if (new_process->type == PROCESS_USER) {
15        tss.esp0 = (uintptr_t)(new_process->kstack) + KERNEL_STACK_SIZE;
16        set_cr3((uintptr_t)new_process->page_directory);
17    } else if (new_process->type == PROCESS_KERNEL) {
18        set_cr3((uintptr_t)kernel_directory);
19    }
20
21    if (old_process->state == PROCESS_TERMINATED) {
22        destroy_process(old_process);
23    }
24
25    load_context(new_process, regs);
26 }

```

31.2.1 Why the state is checked before overwriting it

The first if only demotes RUNNING to READY. That matters: if the process arrived here because it called `exit` (state `TERMINATED`), or because it blocked on a read (state `BLOCKED`), or because it slept, that state must be preserved. Unconditionally setting `READY` would make a blocked process immediately runnable again, and the read would return garbage.

31.2.2 Saving the stack correctly

lytlnybl/tasks/context.c

```

1 void save_context(process_t* process, registers_t* regs) {
2     process->regs.eip = regs->eip;
3     process->regs.cs = regs->cs;
4     process->regs.eflags = regs->eflags;
5     process->regs.ds = regs->ds;
6     /* ... the eight general-purpose registers ... */
7
8     if (process->type == PROCESS_KERNEL) {
9         process->regs.esp = regs->esp; /* from pusha */
10    } else {
11        process->regs.esp = regs->user_esp; /* pushed by the processor */
12        process->regs.ss = regs->ss;
13    }
14 }

```

This is the conditional from Chapter 21 made concrete. For a kernel process there was no ring change, so the only ESP available is the one pushed. For a user process the processor pushed the real user ESP and SS onto the kernel stack, and those are the ones that matter — the pusha value is the kernel stack pointer, which is meaningless to the user program.

Getting this backwards

Save `regs->esp` for a user process and on resumption it would run with a *kernel* stack pointer while in ring 3. The first push would fault — or worse, if that address happened to be mapped user-accessible, it would silently corrupt kernel memory. This is exactly the class of bug that appears “randomly”, hours later, in an unrelated subsystem.

31.2.3 tss.esp0: preparing for the next interrupt

```
1 tss.esp0 = (uintptr_t)(new_process->kstack) + KERNEL_STACK_SIZE;
```

When a ring 3 process is interrupted, the processor reads `esp0` from the TSS and switches to that stack automatically. So before running a user process, the kernel must point `esp0` at *that process's* kernel stack.

Forget this line and every process would share one kernel stack: a nested interrupt would clobber another process's saved state. Part X returns to the TSS in detail.

31.3 The scheduler

lytlnybl/tasks/scheduler.c

```
1 void schedule(registers_t* regs) {
2     scheduler_tick_count++;
3     if (scheduler_tick_count >= time_slice) {
4         scheduler_tick_count = 0;
5         process_t* next_process = get_next_process();
6         if (next_process != current_process) {
7             context_switch(current_process, next_process, regs);
8         }
9     }
10 }
```

`schedule` is called on every timer tick, but only switches every `time_slice` ticks. That constant is the **quantum**: how long a process gets before being preempted.

The quantum is a trade-off

Short quantum: the system feels responsive, but more time is spent switching than working — every switch costs a save, a restore, a CR3 reload and a full TLB flush.

Long quantum: less overhead, but a process hogging the CPU makes everything else feel frozen.

Real schedulers vary it dynamically and add priorities: interactive processes get short, frequent slices; compute-bound ones get long, rare ones. lytlnyblOS uses a fixed quantum, which is the right choice for something you want to be able to read.

31.3.1 Round robin

```

1 process_t* get_next_process() {
2     process_t* traversal_process = current_process;
3     do {
4         if (traversal_process->state == PROCESS_READY) {
5             return traversal_process;
6         }
7         if (!traversal_process->next) {
8             traversal_process = process_head;    /* wrap around */
9         } else {
10            traversal_process = traversal_process->next;
11        }
12    } while (!(traversal_process->state == PROCESS_RUNNING));
13
14    return current_process;
15 }

```

Start at the current process, walk forward, wrap at the end, return the first READY one. Everyone gets a turn, in order, with no favourites. It is the simplest fair policy that exists.

What happens when nobody is ready

The loop stops when it comes back round to a RUNNING process, and then returns `current_process`. But consider: the shell calls `read` and blocks. There is no other runnable process. `get_next_process` walks the whole list, finds nothing READY, comes back to the shell — which is now BLOCKED, not RUNNING — and the loop condition `while (!(state == RUNNING))` keeps it spinning.

In practice the kernel's own initial process is always RUNNING, so the loop terminates and returns it. But that is luck, not design: the exit condition depends on a process that happens to sit in an infinite loop at the end of `kernel_main`.

The standard fix is an **idle process**: a kernel process, always READY, that does nothing but `hlt` in a loop. It guarantees the scheduler always has somewhere to go, and it lets the CPU actually sleep when there is no work — which the current `for(;;)` at the end of `kernel_main` does not.

31.4 Destroying a process

lytlnybl/tasks/procman.c

```

1 void destroy_process(process_t* proc) {
2     if (!proc || proc->state == PROCESS_RUNNING) return;
3
4     /* unlink it from the list */
5     process_t* traversal_process = process_head;
6     while (traversal_process) {
7         if (traversal_process->pid == process_head->pid
8             && !traversal_process->next
9             && traversal_process->pid == proc->pid) {
10             process_head = traversal_process->next;
11             break;
12         }
13         if (!traversal_process->next) { traversal_process = NULL; break; }
14         if (traversal_process->next->pid == proc->pid) {
15             traversal_process->next = traversal_process->next->next;
16             break;
17         }
18         traversal_process = traversal_process->next;
19     }
20     if (!traversal_process) return;
21
22     kfree(proc->kstack);
23     if (proc->type == PROCESS_USER) {
24         unmap_page(proc->page_directory, USER_STACK_TOP - 4096);
25         unmap_page(proc->page_directory, USER_CODE_BASE);
26         unmap_page(proc->page_directory, USER_VGA);
27         unmap_page(kernel_directory, (uintptr_t)(proc->page_directory));
28     }
29     kfree(proc);
30 }

```

This function has several real problems, and it is worth being explicit about them because they are representative of how cleanup code goes wrong.

Three bugs in one function

- 1. The head is never removed if it has a successor.** The first condition requires `!traversal_process->next` — that the head is also the *last* node. If the head has a successor, that branch fails, the second returns early, and the third only ever inspects `next`. A terminated head process stays in the list forever.
- 2. `unmap_page(kernel_directory, ...)` damages every process.** Every process copies the kernel's directory entries, which means they all point at the *same* kernel page tables. Removing a mapping from `kernel_directory` removes it from the shared table — so it disappears from every address space at once, permanently punching a hole in the identity map.
- 3. No physical memory is ever reclaimed.** `unmap_page` clears the present bit but never calls `free_frame`. The page directory, the code frames, the stack frame and the heap frames all stay marked used forever. Run enough programs and the machine runs out of RAM with nothing running.

And a fourth, subtler one

`destroy_process` is called from inside `context_switch`, which is running *on the kernel stack of the process being destroyed*. `kfree(proc->kstack)` marks that stack free while the code is still using it, and it continues using it until the `iret`.

Nothing breaks today because nothing allocates in between. But it is a textbook use-after-free, and the correct pattern is deferred cleanup: mark the process as reappable and free its resources later, from a context that is not standing on them.

Watch the teardown problem

Boot the system and run `run /bin/user_test` five or six times in a row from the shell. The program works each time, but the display progressively degrades — output from different runs interleaves and overwrites earlier lines. That is the cleanup path leaving the system in a progressively worse state. A correct teardown would leave the system indistinguishable from before.

In short

The kernel can now create, schedule, switch and (imperfectly) destroy processes. Everything so far runs in ring 0, though. The next part does the thing this whole course has been building towards: dropping a program into ring 3 and giving it a controlled way back in.

PART X

User space

The TSS and the way back in

By the end of this chapter you will know

- Why a ring 3 program cannot run on its own stack inside the kernel
- What the TSS is and why only one field of it matters
- How the GDT descriptor for the TSS is filled in from C

32.1 The problem

A user process runs in ring 3 with its own stack, somewhere around `0xBFFFF000`. Now the timer fires. The processor must run kernel code — but on which stack?

Not the user's. It might be nearly full, it might be corrupt, it might be deliberately pointed at kernel memory by a hostile program. If the kernel pushed its data there, a user program could read the kernel's saved state, or make the kernel fault by unmapping its own stack.

So the processor **switches stacks automatically** on a privilege change. And it needs to be told, in advance, which stack to switch to. That is what the **TSS** (Task State Segment) is for.

An analogy

The user stack is the visitor's own bag. The kernel will not put its confidential paperwork in a visitor's bag — the visitor could read it, or walk off with it. So at the door there is a dedicated locker, and its number is written on a card the receptionist keeps. The TSS is that card; `esp0` is the locker number.

32.2 One useful field

The TSS is a large structure — 104 bytes — designed in the 80286 era for hardware task switching, a feature nobody uses any more because it is slower than doing it in software.

Modern systems, including this one, keep a TSS only because the processor insists on reading two fields from it:

Field	Purpose
ss0	The stack <i>segment</i> to use in ring 0. Always 0x10 here
esp0	The stack pointer to use in ring 0. Updated on every switch to a user process

Everything else in the structure is zeroed and ignored.

lytlnybl/tasks/procman.c

```

1 void init_tss(void) {
2     uintptr_t base = (uintptr_t)&tss;
3     uint32_t limit = sizeof(tss_t) - 1;
4
5     /* fill in the GDT descriptor that was left blank in assembly */
6     gdt_tss[0] = limit & 0xFF;
7     gdt_tss[1] = (limit >> 8) & 0xFF;
8     gdt_tss[2] = base & 0xFF;
9     gdt_tss[3] = (base >> 8) & 0xFF;
10    gdt_tss[4] = (base >> 16) & 0xFF;
11    gdt_tss[5] = 0x89; /* present, ring 0, 32-bit available TSS */
12    gdt_tss[6] = (limit >> 16) & 0xFF;
13    gdt_tss[7] = (limit >> 24) & 0xFF;
14
15    memset(&tss, 0, sizeof(tss_t));
16    tss.ss0 = 0x10; /* kernel data segment */
17    tss.esp0 = 0; /* set properly on each context switch */
18    tss.iomap_base = sizeof(tss_t); /* no I/O permission bitmap */
19
20    load_tss();
21 }

```

This is the descriptor from Chapter 16 being filled in by hand, byte by byte, at runtime — because the address of a C variable is not known at assembly time. Access byte 0x89 means present, DPL 0, system segment, type 9 = “available 32-bit TSS”.

iomap_base

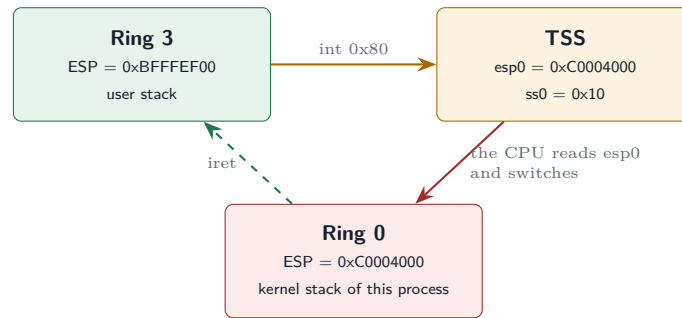
Setting it to `sizeof(tss_t)` means “the I/O permission bitmap starts past the end of this structure”, which the processor reads as “there is no bitmap, deny all port access from ring 3”. Leave it at zero and the processor would interpret the start of the TSS itself as a permission bitmap, granting user programs direct access to arbitrary hardware ports. A subtle and genuinely dangerous mistake.

```

1 load_tss:
2     mov ax, 0x28 ; the TSS selector
3     ltr ax ; load task register
4     ret

```

32.3 What actually happens on entry



Once on the kernel stack, the processor pushes the user's **SS** and **ESP** there, then **EFLAGS**, **CS** and **EIP**. That is why `registers_t` has those two extra fields at the end, and why they only exist when the ring changed.

Entering ring 3

By the end of this chapter you will know

- How a process is placed in ring 3 without a special instruction
- How a program's address space is built
- How a binary is loaded from disk into memory

33.1 There is no “enter user mode” instruction

x86 has no instruction that says “drop to ring 3”. Privilege is lowered only as a side effect of returning from an interrupt: `iret` pops `CS`, and if the popped `CS` has RPL 3, the processor switches to ring 3, pops `SS` and `ESP` too, and resumes there.

So the trick is to fabricate a context that *looks like* it came from ring 3, and then let the normal return path do the work:

lytlnybl/tasks/procman.c

```
1 void create_uprocess(process_t* new_process) {
2     new_process->regs.esp = USER_STACK_TOP;      /* 0xBFFFFFF00 */
3     new_process->regs.cs = 0x18 | 3;               /* user code, RPL 3 */
4     new_process->regs.ds = 0x20 | 3;               /* user data, RPL 3 */
5     new_process->regs.ss = 0x20 | 3;
6     new_process->regs.eflags = 0x202;             /* interrupts enabled */
7     /* ... address space set-up ... */
8 }
```

When the scheduler picks this process, `load_context` copies those values into the `registers_t` on the kernel stack, and `iret` does the rest. **Ring 3 is entered by returning to it, from an interrupt that never happened.**

Why EFLAGS must have IF set

The user process must run with interrupts enabled, otherwise the timer can never preempt it and a single `while(1);` would freeze the machine permanently — with no way to recover, because ring 3 cannot execute `sti` either.

Note this is not a favour the kernel does the program: it is the kernel protecting *itself*.

33.2 Building the address space

```

1 new_process->page_directory = (page_directory_t*)alloc_frame();
2 memset((page_directory_t*)new_process->page_directory, 0, 4096);
3
4 for (uint32_t i = 0; i < 1024; i++) {
5     (*new_process->page_directory)[i] = (*kernel_directory)[i];
6 }
7
8 uintptr_t ustack_frame = (uintptr_t)alloc_frame();
9 map_page(new_process->page_directory, USER_STACK_TOP - 4096, ustack_frame,
10         PAGE_PRESENT | PAGE_WRITABLE | PAGE_USER);
11
12 map_page(new_process->page_directory, USER_HEAP_START,
13         (uintptr_t)alloc_frame(),
14         PAGE_PRESENT | PAGE_WRITABLE | PAGE_USER);
15
16 new_process->user_heap_end = USER_HEAP_START + 4096;
17 new_process->heap_pages_allocated = 1;
18 new_process->ustack = (void*)(USER_STACK_TOP - 4096);

```

One page of stack (4 KB) and one page of heap. Note the stack page is mapped at `USER_STACK_TOP - 4096`, because `USER_STACK_TOP` is the address *above* the last usable byte — the stack grows down from there into the page below.

The stack does not grow

Exactly one page. If a user program recurses deeply or declares a large local array, `ESP` drops below `0xBFFFE000` and hits an unmapped page. The page fault handler prints a message and the process is killed.

Real systems handle this: a fault just below the stack is recognised as legitimate growth and a new page is mapped automatically. Adding that to `page_fault_handler` is one of the most instructive extensions you can make to this project — it is perhaps fifteen lines, and it turns a crash into a feature.

33.3 Loading a program from disk

lytlnybl/tasks/loader.c

```

1 process_t* load_program(const char* path) {
2     int32_t fd = fs_open(path);
3     uint32_t inode_num = fs_fd_to_inode(fd);
4
5     fs_inode_t file_inode;
6     if (!fs_read_inode(inode_num, &file_inode))
7         return NULL;

```

```

8
9     size_t file_size = file_inode.size;
10
11     uint8_t buffer[file_size];           /* variable length array */
12     if (fs_read(fd, buffer, file_size) == -1)
13         return NULL;
14
15     process_t* user_proc = create_process((void*)USER_CODE_BASE, PROCESS_USER);
16
17     for (size_t i = 0; i < ((file_size + 4095) / 4096); ++i) {
18         void* code_frame = alloc_frame();
19
20         size_t offset = i * 4096;
21         size_t bytes = file_size - offset;
22         if (bytes > 4096) bytes = 4096;
23         memcpy(code_frame, buffer + offset, bytes);
24
25         map_page(user_proc->page_directory, USER_CODE_BASE + (4096 * i),
26                 (uintptr_t)code_frame,
27                 PAGE_PRESENT | PAGE_WRITABLE | PAGE_USER);
28     }
29
30     fs_close(fd);
31     return user_proc;
32 }

```

The logic is: read the whole file into a buffer, then copy it page by page into freshly allocated frames, mapping each one at consecutive virtual addresses starting at 0x400000.

Why the copy is needed at all

`memcpy(code_frame, ...)` writes to a *physical* address, which works because the kernel identity-maps the low 4 MB. The mapping into the process's address space happens afterwards.

The kernel cannot write directly to 0x400000 in the process's space, because that address means something else entirely in the kernel's own address space. This two-step dance — write to the frame physically, then map it virtually — is a pattern you will meet in every kernel.

Three real problems in this function

1. `uint8_t buffer[file_size]` is a **variable-length array on a 16 KB kernel stack**. A program larger than about 14 KB overflows it, and Chapter 4 explained what that means. The fix is to copy page by page directly from the file, never materialising the whole thing.
2. `fs_open` is **not checked**. If the path does not exist it returns `-1`, and the code proceeds to call `fs_fd_to_inode(-1)`.
3. **Code pages are mapped writable**. `PAGE_WRITABLE` on a code page means a program can modify its own instructions. Dropping that flag would be a one-word improvement in safety.

And the consequence of problem 2, in the shell

When the shell runs `run /nonexistent`, `load_program` returns `NULL`, and the system call handler does:

```
1 process_t* child = load_program(path);
2 child->parent_pid = current_process->pid;  /* NULL dereference */
```

`parent_pid` is at offset 4 in the structure, so this writes to address `0x4` — which *is* mapped, because the low 4 MB is identity-mapped. No fault occurs. Instead it silently corrupts the interrupt vector table area, then reads a garbage pid, sets the shell to `BLOCKED` waiting for a child that will never exist, and switches away.

The observable symptom is that the shell hangs forever with no error message.

Verified on a running system: after `run /bin/usertest` (a typo), the prompt never returns and further typing echoes but does nothing.

The fix is three lines:

```
1 process_t* child = load_program(path);
2 if (!child) { regs->eax = -1; break; }
```

In practice: loading a program from the kernel

```
examples/ch33-loader/ -- make run
```

Calls `load_program` twice: once with a path that does not exist, to show it returns `NULL` as it should, and once with a real one. It then prints the new process's page directory, entry point, stack pointer and code segment before the scheduler has run it even once.

CS comes back as `0x1B`, which is `0x18|3`. Nothing else has to happen: the timer interrupt reaches the scheduler, the scheduler picks the process, and `iret` drops the CPU into ring 3. You can watch the program's output appear a moment later.

CHAPTER 34

System calls

By the end of this chapter you will know

- Follow a system call from the C library to the kernel and back
- Understand how arguments and return values travel
- Understand why validating user pointers is the most important missing piece

34.1 The user side

lytlnybl/user/libc/syscalls.asm

```
1 syscall:
2     mov eax, [esp + 4]    ; service number
3     mov ebx, [esp + 8]    ; first argument
4     mov ecx, [esp + 12]   ; second argument
5     mov edx, [esp + 16]   ; third argument
6
7     int 0x80
8
9     ret                  ; EAX already holds the return value
```

Ten lines that bridge two worlds. C pushes arguments on the stack; this routine moves them into the registers the kernel expects, triggers the interrupt, and returns — with `EAX` untouched since the kernel put the result there, which is exactly where C expects a return value.

lytlnybl/user/libc/syscalls.c

```
1 int write(int fd, void *buff, size_t count) {
2     return syscall(SYSCALL_WRITE, fd, (uint32_t)buff, count);
3 }
4
5 int read(int fd, void* buff, size_t count) {
6     return syscall(SYSCALL_READ, fd, (uint32_t)buff, count);
7 }
8
9 void* sbrk(intptr_t increment) {
10    return (void*)syscall(SYSCALL_SBRK, increment, 0, 0);
11 }
```

34.2 The kernel side

lytlnybl/kernel/interrupts.asm

```

1  syscall_entry:
2      push dword 0x80
3      push dword 0
4
5      pusha
6      mov ax, ds
7      push eax
8
9      push esp
10     call syscall_handler
11     add esp, 4
12
13     pop eax
14     popa
15     add esp, 8
16     iret

```

Structurally identical to the ISR stub. The two initial pushes fill the `interrupt_number` and `error_code` slots so the layout matches `registers_t`.

A small inconsistency

The ISR macros push the error code first and the interrupt number second, so the number lands at the lower address. Here the order is reversed: 0x80 ends up in `error_code` and 0 in `interrupt_number`.

It is harmless, because `syscall_handler` reads neither field. But it will confuse anyone debugging with a generic dump routine, who will see interrupt number 0 — “divide by zero” — on every system call. Swapping the two pushes would cost nothing and remove a trap.

34.3 The dispatcher

lytlnybl/kernel/interrupts.c

```

1  void syscall_handler(registers_t* regs) {
2      int fd;
3      uint32_t buffer;
4      size_t count;
5
6      switch (regs->eax) {
7          case SYSCALL_GETPID:
8              regs->eax = current_process->pid;
9              break;
10

```

```

11     case SYSCALL_YIELD:
12         context_switch(current_process, get_next_process(), regs);
13         break;
14
15     case SYSCALL_SLEEP:
16         current_process->wake_tick = timer_get_ticks() + regs->ebx;
17         current_process->state = PROCESS_SLEEPING;
18         context_switch(current_process, get_next_process(), regs);
19         break;
20     /* ... */
21 }
22 }

```

`regs->eax = current_process->pid`; deserves a second look. It writes into the saved state on the kernel stack; `popa` restores it into the real `EAX`; `iret` returns to ring 3; the user's `syscall` routine executes `ret` with the value already in place. The return value travels four layers without anyone explicitly returning anything.

34.4 sbrk: growing the user heap

```

1 case SYSCALL_SBRK:
2     current_process->user_heap_end += regs->ebx;
3
4     while (((current_process->user_heap_end + 4096) - USER_HEAP_START) / 4096
5           > current_process->heap_pages_allocated) {
6         map_page(current_process->page_directory,
7                 USER_HEAP_START + (current_process->heap_pages_allocated++ * 4096),
8                 (uintptr_t)alloc_frame(),
9                 PAGE_PRESENT | PAGE_USER | PAGE_WRITABLE);
10    }
11
12    regs->eax = current_process->user_heap_end;
13    break;

```

This is the interface Unix has used since the 1970s: “move the top of my heap by this much and tell me where it now is”. The kernel deals only in pages; the user-space `malloc` in `user/libc/memory.c` turns those pages into byte-sized allocations, using the same linked-list design as the kernel heap in Chapter 29.

alloc_frame can return NULL

If physical memory is exhausted, `map_page` maps address 0 as a user page. The program then gets a heap that silently aliases physical frame 0 — the interrupt vector table. Checking the return value and failing the call cleanly is a small fix with a large payoff.

34.5 The validation that is missing

This is the most important section in the chapter.

```

1 case SYSCALL_WRITE:
2     fd = regs->ebx;
3     buffer = regs->ecx;
4     count = regs->edx;
5
6     if (fd < 1) { regs->eax = -1; break; }
7     if (fd > 2) { regs->eax = fs_write(fd, (void*)buffer, count); break; }
8
9     ((char*) buffer)[count] = '\0';          /* <-- here */
10    vga_text_write(&terminal, (char*)buffer);
11
12    regs->eax = (int)count;
13    break;

```

Two distinct problems in one line

1. **It writes one byte past what the caller asked for.** If a program calls `write(1, buf, sizeof(buf))` — which the shell’s `cat` command does — the kernel writes a zero one byte beyond the end of `buf`, corrupting whatever follows on the user’s stack.
2. **The pointer is never validated.** `buffer` is a number chosen entirely by the user program. Nothing checks that it points into that process’s own memory. A program can pass a kernel address and the kernel — running in ring 0, where the `PAGE_USER` bit does not restrict it — will happily write a zero byte there.

That is a privilege escalation primitive. Not a theoretical one: an arbitrary-address single-byte write into kernel memory is enough to corrupt a function pointer or a page table entry.

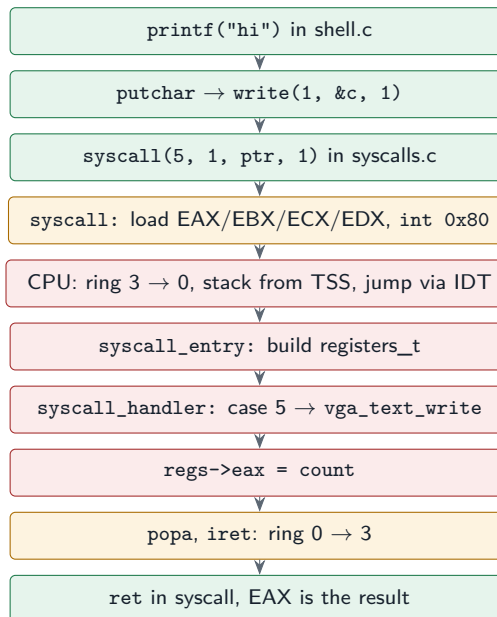
What a real kernel does instead

Every operating system has a function like `copy_from_user` / `copy_to_user`. It:

1. checks that the whole range $[p, p + n)$ lies inside user address space,
2. checks the pages are present and have the right permissions,
3. copies into a kernel buffer, so the user cannot change the data after validation,
4. returns an error rather than faulting if anything is wrong.

Adding even a minimal version of this to `lytlnyblOS` — “reject any pointer at or above `0xC0000000`, and any count that would cross out of a mapped page” — would close the entire class of bug. It is the single highest-value change one could make to this codebase.

34.6 The full round trip



In practice: adding a system call, end to end

```
examples/ch34-new-syscall/ -- make run
```

The single most useful recipe in the book, because it is the one you will want first. It adds `SYSCALL_UPTIME`, which returns the tick count, and the patch script makes exactly four edits — the four you always have to make:

1. a number in `include/kernel/interrupts.h`,
2. a case in `syscall_handler` that writes the result into `regs->eax`,
3. the same number plus a prototype in `include/user/libc/syscalls.h`,
4. the wrapper in `user/libc/syscalls.c`.

It ships a user program that calls it, sleeps for 100 ticks and calls it again. Run `run /bin/user_test` at the shell and the difference printed is exactly 100.

A minimal C library

By the end of this chapter you will know

- What a user program needs before `main` can run
- How `printf` is implemented with variadic arguments
- How user-space `malloc` sits on top of `sbrk`

35.1 What the library provides

File	Contents
<code>user/libc/syscalls.asm</code>	The <code>syscall</code> bridge
<code>user/libc/syscalls.c</code>	One wrapper per system call
<code>user/libc/output.c</code>	<code>putchar</code> , <code>puts</code> , <code>print_num</code> , <code>printf</code>
<code>user/libc/memory.c</code>	<code>malloc</code> , <code>free</code> , <code>calloc</code> , <code>realloc</code> , <code>memcpy</code> , <code>memmove</code> , <code>memset</code> , <code>memcmp</code>
<code>user/libc/string.c</code>	<code>strlen</code> , <code>strcmp</code> , <code>strcpy</code> ...
<code>user/libc/chars.c</code>	<code>isalpha</code> , <code>isdigit</code> , <code>isspace</code> , <code>tolower</code> ...

They are compiled into an archive, `libc.a`, and linked into each program:

lytlnybl/Makefile

```
$(USER_LIB): $(USER_LIB_C_OBJECTS) $(USER_LIB_ASM_OBJECTS)
    $(AR) rcs $@ $^

$(SHELL_ELF): $(BUILD_DIR)/user/programs/shell.o $(USER_LIB)
    $(LD) -m elf_i386 -T $(USER_LINKER) $^ -o $@
```

An archive is just a bundle of object files; the linker pulls out only the ones actually referenced, which keeps the binaries small.

35.2 `printf` with variadic arguments

lytlnybl/user/libc/output.c

```

1 void printf(char* format, ...) {
2     va_list args;
3     va_start(args, format);
4
5     while (*format) {
6         if (*format == '%') {
7             format++;
8             if (*format == 'c') {
9                 char c = va_arg(args, int);
10                putchar(c);
11            } else if (*format == 's') {
12                char *str = va_arg(args, char*);
13                puts(str);
14            } else if (*format == 'd') {
15                int num = va_arg(args, int);
16                print_num(num);
17            } else if (*format == '%') {
18                putchar('%');
19            } else {
20                putchar('%');
21                putchar(*format);
22            }
23        } else {
24            putchar(*format);
25        }
26        format++;
27    }
28    va_end(args);
29 }

```

How variadic arguments work

Because 32-bit x86 pushes arguments on the stack right to left, the arguments after `format` sit immediately above it in memory, contiguously. `va_start` points at that spot and `va_arg` reads one and advances.

This also explains why `printf` is fundamentally unsafe: the function has no way to know how many arguments were actually passed. `printf("%d %d", 1)` reads a second integer from whatever happens to be on the stack. Nothing detects it.

putchar writes one byte at a time

`putchar` calls `write(1, &c, 1)`, which is a full system call — an interrupt, a privilege switch, a dispatch — *per character*. Printing a 40-character line means 40 system calls.

Real libraries buffer: accumulate into an array and flush on newline or when full. Adding buffering here would be a large speed-up and a small change. It would also interact with the write-past-the-end bug from the previous chapter, so fix that first.

35.3 malloc on top of sbrk

lytlnybl/user/libc/memory.c

```

1 void *malloc(size_t bytes) {
2     if (bytes == 0) return NULL;
3     if (!heap_start_head) init_heap();
4
5     heap_header_t* block = find_free_block(bytes);
6     if (block) {
7         if (block->size != bytes) split_block(block, bytes);
8         block->free = false;
9         return (void*)((uintptr_t)block + sizeof(heap_header_t));
10    } else {
11        expand_heap(bytes);
12        return malloc(bytes);
13    }
14 }
15
16 void expand_heap(size_t requested_size) {
17     uintptr_t original_end = heap_end;
18     heap_end = (uintptr_t)sbrk(requested_size);
19     /* ... append a new free block ... */
20 }

```

This is the same design as the kernel heap — headers, a singly linked list, split and merge — but one layer up: where the kernel heap called `map_page`, this calls `sbrk`.

The same pattern at every level

Notice the recurring shape. The physical manager hands out 4 KB frames. The virtual manager turns frames into mapped pages. The kernel heap turns pages into bytes. `sbrk` turns kernel pages into user pages. User `malloc` turns those into bytes.

Five layers, each doing one job and asking the layer below for coarser units. That is what “building an operating system” looks like in practice: not one clever mechanism, but a stack of simple ones that each hide the mess of the one beneath.

In practice: a user program template

```
examples/ch35-libc/ -- make run
```

Everything a program in this system can do, in one commented file: `printf` with all its conversions, `malloc` / `calloc` / `realloc` / `free`, the string functions, and `get_pid`, `sleep`, `yield` and `exit`.

Start from it when writing your own. Note it calls its entry point `_start`, which is what `user.ld` actually names — unlike the shell, for the reason the next section explains.

35.4 The entry point problem

```
lytlnybl/user/user.ld
```

```
ENTRY(_start)

SECTIONS
{
    . = 0x00400000;
    .text : { *(.text) }
    .rodata : { *(.rodata) }
    .data : { *(.data) }
    .bss : { *(.bss) }
}
```

The kernel always jumps to 0x400000 when starting a user process, so *whatever code sits at that address* is the entry point.

The entry point works by accident

The linker script declares `ENTRY(_start)`, and `user/programs/test.c` does define `_start`. But `user/programs/shell.c` defines `start` — without the underscore.

Link the shell and the linker warns:

```
ld: warning: cannot find entry symbol _start; defaulting to 00400000
```

It then works anyway, for one reason only: the Makefile lists `shell.o` first, so its `.text` lands first, and `start()` happens to be the first function in the file.

Add a function above `start()`, or reorder the link line, and the kernel will jump into the wrong function with no diagnostic at all. The fix is trivial — rename it to `_start`, or better, write a tiny `crt0.o` that is always linked first and calls `main` — but until then this is a loaded gun.

Try it yourself

Check it for yourself:

```
i686-elf-nm -n build/shell.elf | head -3
```

You should see `U _start (undefined)` and `00400000 T start`. The address is right; the symbol name is not.

In short

User space works: programs run unprivileged, in their own address space, and can ask the kernel for services through a single controlled door. What is left is giving them something to work with — persistent storage — and a program to drive it all.

PART XI

Storage and the filesystem

Talking to the disk

By the end of this chapter you will know

- What LBA addressing is and why it replaced CHS
- How ATA PIO transfers a sector
- Why the status register must be polled, and in what order

36.1 LBA: one number instead of three

Chapter 13 used CHS, because that is what the BIOS offers in real mode. Once in protected mode we talk to the disk controller directly, and we can use **LBA** (Logical Block Addressing): sectors are numbered 0, 1, 2, ... and that is the whole model. No geometry, no off-by-one on sector numbers, no track boundaries.

28-bit LBA addresses up to 2^{28} sectors, which is 128 GB. More than enough.

36.2 The registers

The ATA controller is a set of I/O ports:

Port	Name	Purpose
0x1F0	DATA	16-bit data transfer window
0x1F2	SECTOR_COUNT	How many sectors to transfer
0x1F3	LBA_LOW	LBA bits 7–0
0x1F4	LBA_MID	LBA bits 15–8
0x1F5	LBA_HIGH	LBA bits 23–16
0x1F6	DRIVE	Drive select + LBA bits 27–24
0x1F7	STATUS / COMMAND	Read: status. Write: command
0x3F6	ALT_STATUS	Status without side effects

Note that 0x1F7 is two different registers depending on whether you read or write it. That kind of thing is normal in hardware and catches people out constantly.

36.3 Reading a sector

lytlnybl/kernel/drivers/ata.c

```

1 bool ata_read_sector(uint32_t lba, void* buffer) {
2     uint16_t *data = (uint16_t*)buffer;
3
4     if (lba > 0xFFFFFFFF) return false;    /* beyond 28-bit LBA */
5
6     ata_select_drive(lba);                /* drive + top 4 LBA bits */
7
8     outb(ATA_SECTOR_COUNT, 1);            /* one sector */
9
10    outb(ATA_LBA_LOW,  lba & 0xFF);
11    outb(ATA_LBA_MID,  (lba >> 8) & 0xFF);
12    outb(ATA_LBA_HIGH, (lba >> 16) & 0xFF);
13
14    outb(ATA_COMMAND, ATA_CMD_READ_PIO);  /* 0x20 */
15
16    if (!ata_wait_drq()) return false;    /* wait for data to be ready */
17
18    for (int i = 0; i < 256; i++) {
19        data[i] = inw(ATA_DATA);          /* 256 words = 512 bytes */
20    }
21
22    ata_400ns_delay();
23    return true;
24 }
```

The 28-bit LBA is split across four registers because each port is 8 bits wide: three full bytes plus the low nibble of the drive register.

Why 256 iterations and not 512

The data port is 16 bits wide, so each `inw` brings in two bytes. $256 \times 2 = 512$: one sector. This is why the buffer is cast to `uint16_t*` — and also why passing an odd-aligned buffer would be a bad idea.

36.3.1 Waiting correctly

```

1 static bool ata_wait_drq() {
2     uint8_t status;
3     for (;;) {
4         status = inb(ATA_STATUS);
5         if (status & ATA_STATUS_BSY) continue; /* still busy: keep waiting */
6         if (status & ATA_STATUS_ERR) return false;
7         if (status & ATA_STATUS_DF) return false; /* device fault */
8         if (status & ATA_STATUS_DRQ) return true; /* data ready */
9     }
10 }
```

The order of the checks is not optional

BSY must be tested *first*. While BSY is set, every other bit in the status register is meaningless — the controller is mid-update. Checking ERR before BSY can read a stale error bit from a previous command and abort a perfectly good read.

Note also that this loop has no timeout. A disk that never clears BSY hangs the kernel forever. Real drivers count iterations and give up.

```
1 static void ata_400ns_delay() {
2     inb(ATA_ALT_STATUS);
3     inb(ATA_ALT_STATUS);
4     inb(ATA_ALT_STATUS);
5     inb(ATA_ALT_STATUS);
6 }
```

The specification requires 400 nanoseconds between certain operations. Each read of the alternate status port takes about 100 ns on the bus, so four reads is the canonical way to wait. Reading the *alternate* status port matters: reading the normal one at 0x1F7 clears the pending interrupt, which you may not want.

36.4 Writing

Writing mirrors reading with three differences: command 0x30 instead of 0x20, outw instead of inw, and a cache flush at the end:

```
1 outb(ATA_COMMAND, ATA_CMD_WRITE_PIO);
2 if (!ata_wait_drq()) return false;
3 for (int i = 0; i < 256; i++) outw(ATA_DATA, data[i]);
4 ata_400ns_delay();
5 outb(ATA_COMMAND, ATA_CMD_FLUSH_CACHE); /* 0xE7 */
6 ata_wait_bsy();
```

Why the flush

Disks buffer writes internally and report success immediately. If power is lost before the buffer reaches the platter, the data is gone. FLUSH_CACHE forces it out and does not return until it is physically committed.

This is exactly what `fsync()` does on a real system, and why databases care so much about it.

In practice: reading and writing raw sectors

```
examples/ch37-ata-sectors/ -- make run
```

Writes 512 bytes to LBA 2000 — chosen to be far past both the kernel and the filesystem — reads them back into a different buffer and compares. Then it reads disk sector 70 and

finds 0xDEADBABE in the first four bytes: the filesystem's superblock, seen as raw bytes by a driver that knows nothing about filesystems.

Everything in the next three chapters is built on these two functions.

CHAPTER 37

Designing a filesystem

By the end of this chapter you will know

- What structures are needed to turn a disk into files
- What an inode is and why the name is not in it
- How the on-disk layout is organised

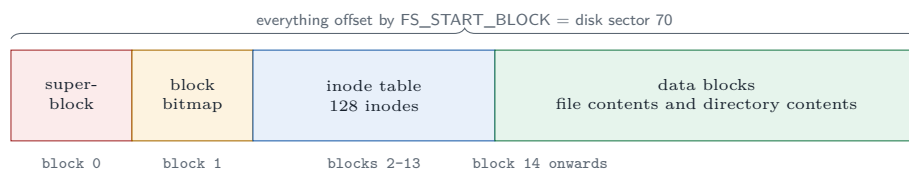
37.1 The problem

A disk is an array of numbered 512-byte sectors. To turn that into “files with names, in directories, of arbitrary size”, you need to answer three questions and store the answers *on the disk itself*:

1. Which blocks are free?
2. For a given file, which blocks hold its contents, and how big is it?
3. Given a name, which file is it?

Each question gets its own structure: a **bitmap**, an **inode**, and a **directory**.

37.2 The layout



lytlnybl/include/filesystem/fs_layout.h

```
1 #define FS_BLOCK_SIZE 512
2 #define FS_START_BLOCK 70      /* the filesystem starts at disk sector 70 */
3 #define FS_SUPERBLOCK 0
4 #define FS_BITMAP_BLOCK 1
5 #define FS_INODE_START 2
```

```

6  #define FS_TOTAL_BLOCKS 1000
7  #define FS_TOTAL_INODES 128
8  #define FS_INODE_MAX_BLOCKS 10
9  #define FS_FILENAME_LENGTH 32
10 #define FS_MAGIC 0xDEADBABE

```

The whole filesystem is shifted 70 sectors into the disk, leaving room for the boot sector and the kernel. The translation is one line:

```

1  bool fs_read_block(uint32_t block_num, void* buffer) {
2      uint32_t sector = FS_START_BLOCK + block_num;
3      return ata_read_sector(sector, buffer);
4  }

```

37.3 The superblock

Block 0 describes the filesystem itself:

```

1  typedef struct {
2      uint32_t magic;           /* 0xDEADBABE */
3      uint32_t block_size;
4      uint32_t total_blocks;
5      uint32_t bitmap_start;
6      uint32_t inode_start;
7      uint32_t inode_count;
8      uint32_t inode_blocks;
9      uint32_t root_inode;
10     uint32_t data_start;
11     uint32_t free_blocks;
12     uint32_t free_inodes;
13 } fs_superblock_t;

```

The magic number

`fs_read_superblock` refuses to proceed if the magic does not match:

```

1  if (superblock->magic != FS_MAGIC) return false;

```

Without it, pointing the driver at an unformatted disk would make it read random bytes as `inode_count` and `data_start`, and then happily write structures all over it. The magic number costs four bytes and turns silent destruction into a clean failure.

37.4 Inodes

```

1  typedef struct {
2      uint32_t size;           /* size in bytes */

```

```

3  uint16_t type;                                /* free, file or directory */
4  uint32_t blocks[FS_INODE_MAX_BLOCKS]; /* 10 direct block pointers */
5  } fs_inode_t;

```

Why the name is not in the inode

This is the design decision that defines Unix-style filesystems, and it surprises everyone the first time.

An inode describes a *file*: its size, its type, where its data lives. The *name* lives in the directory that contains it. So a file can have several names (hard links), all pointing at one inode and one copy of the data. Renaming a file touches only the directory, not the file.

lytlnyblOS does not implement hard links, but it inherits the structure — which means adding them later would be almost free.

With 10 direct pointers and 512-byte blocks, the maximum file size is $10 \times 512 = 5120$ bytes.

5 KB is small, and the code knows it

That is enough for the shell binary (4,392 bytes) but not much more — and `user_test` plus the shell together already use most of the inode table's useful capacity.

The classic solution is **indirect blocks**: the eleventh pointer points not at data but at a block full of *more* pointers. With 512-byte blocks that is 128 extra pointers, taking the maximum to about 69 KB. A double-indirect pointer gets you to 8 MB. This is exactly how ext2 works, and adding single indirection here is a very instructive exercise.

37.5 Directories are files

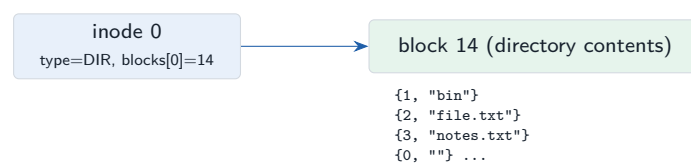
A directory is an inode of type `FS_TYPE_DIRECTORY` whose data blocks contain an array of fixed-size records:

```

1  typedef struct {
2      uint32_t inode;                /* 0 means the slot is empty */
3      char name[FS_FILENAME_LENGTH]; /* 32 bytes */
4  } fs_directory_entry_t;

```

Each entry is 36 bytes, so a 512-byte block holds 14 of them.



Recursion bottoms out at the root

To find `/bin/shell` you read the root directory to find `bin`, then read `bin` to find `shell`. But how do you find the root, when there is no directory containing it?

By convention: the root is always inode 0. `FS_ROOT_INODE` is the fixed point that makes the whole recursive structure resolvable.

37.6 Path resolution

lytlnybl/filesystem/fs_manager.c

```

1 int32_t fs_resolve_path(const char* path) {
2     if (!path || path[0] != '/') return -1;    /* absolute paths only */
3
4     uint32_t current_inode = FS_ROOT_INODE;
5     uint32_t i = 1;
6
7     while (path[i] != '\0') {
8         char name[FS_FILENAME_LENGTH];
9         uint32_t name_length = 0;
10
11         while (path[i] == '/') i++;           /* skip repeated slashes */
12         if (path[i] == '\0') break;
13
14         while (path[i] != '/' && path[i] != '\0') {
15             if (name_length >= FS_FILENAME_LENGTH - 1) return -1;
16             name[name_length++] = path[i++];
17         }
18         name[name_length] = '\0';
19
20         int32_t inode = fs_find_directory_entry(current_inode, name);
21         if (inode < 0) return -1;
22
23         current_inode = inode;
24     }
25     return current_inode;
26 }
```

Split on slashes, look each component up in the current directory, move down. The bounds check inside the inner loop is what stops a 200-character path component from overflowing the 32-byte local array — exactly the kind of check that is missing elsewhere in this codebase.

No . or ..

There is no current directory and no parent directory. Every path must be absolute, and `cd` does not exist. Adding them means creating two entries in every directory at `mkdir` time and handling them in resolution — a contained and worthwhile exercise.

CHAPTER 38

File descriptors and the manager

By the end of this chapter you will know

- What a file descriptor is and why it is just an index
- Why descriptors 0, 1 and 2 are special
- How reading and writing cross block boundaries

38.1 The open file table

lytlnybl/include/filesystem/fs_manager.h

```
1 #define FS_MAX_OPEN_FILES 16
2 #define FS_FD_OFFSET 3
3
4 typedef struct {
5     uint32_t inode;
6     uint32_t offset;    /* current read/write position */
7     bool used;
8 } fs_file_t;
```

```
1 static fs_file_t open_files[FS_MAX_OPEN_FILES];
2
3 int32_t fs_open(const char* path) {
4     int32_t inode = fs_resolve_path(path);
5     if (inode < 0) return -1;
6
7     for (int i = 0; i < FS_MAX_OPEN_FILES; i++) {
8         if (!open_files[i].used) {
9             open_files[i].used = true;
10            open_files[i].inode = inode;
11            open_files[i].offset = 0;
12            return i + FS_FD_OFFSET;
13        }
14    }
15    return -1;
16 }
```

A descriptor is an index, nothing more

A file descriptor is not a pointer and not a handle. It is the array index of a slot in the kernel's open-file table, plus 3.

That indirection is deliberate and important: the user program holds a small integer that means nothing outside the kernel. It cannot forge a pointer to a file structure, cannot corrupt the table, cannot reach another process's files. The kernel validates the index on every use:

```
1 fd -= FS_FD_OFFSET;
2 if (fd < 0 || fd >= FS_MAX_OPEN_FILES) return -1;
3 if (!open_files[fd].used) return -1;
```

This is the same design idea as virtual addresses: hand out names, not places.

38.1.1 Why the offset of 3

Unix reserves three descriptors by convention:

fd	Name	In lytlnyblOS
0	stdin	The keyboard
1	stdout	The screen
2	stderr	The screen

So real files start at 3. That is what `FS_FD_OFFSET` encodes, and it is why the system call handler can dispatch on the number alone:

lytlnybl/kernel/interrupts.c

```
1 if (fd > 2) {
2     regs->eax = fs_write(fd, (void*)buffer, count);  /* a real file */
3     break;
4 }
5 /* otherwise: the screen */
6 vga_text_write(&terminal, (char*)buffer);
```

The table is global, not per process

`open_files` is a single static array shared by every process. Two processes opening files get different slots, which works — but a process can pass any descriptor number and reach another process's open file. And there is no cleanup: if a process exits without calling `close`, the slot leaks forever. Sixteen leaked descriptors and no file can be opened again.

A real system keeps a per-process descriptor table pointing into a global open-file table, and closes everything on exit. That is two structures instead of one, and it is what makes descriptors actually safe.

38.2 Reading across blocks

A file's data lives in up to 10 scattered blocks, but the caller wants a flat byte range. The translation is the interesting part:

lytlnybl/filesystem/fs.c

```

1 int32_t fs_read_file(uint32_t file_inode_num, void* read_buffer,
2                     uint32_t size, uint32_t offset) {
3     fs_inode_t file_inode;
4     uint8_t buffer[FS_BLOCK_SIZE];
5
6     if (!fs_read_inode(file_inode_num, &file_inode)) return -1;
7
8     if (offset >= file_inode.size) return 0;      /* EOF is 0 bytes, not an error */
9
10    if (offset + size > file_inode.size)
11        size = file_inode.size - offset;          /* clamp to the real size */
12
13    uint32_t bytes_read = 0;
14    uint8_t* destination = (uint8_t*)read_buffer;
15
16    while (bytes_read < size) {
17        uint32_t position = offset + bytes_read;
18
19        uint32_t block_index = position / FS_BLOCK_SIZE; /* which block */
20        uint32_t block_offset = position % FS_BLOCK_SIZE; /* where inside it */
21
22        if (block_index >= FS_INODE_MAX_BLOCKS) break;
23
24        if (!fs_read_block(file_inode.blocks[block_index], buffer)) return -1;
25
26        uint32_t bytes = FS_BLOCK_SIZE - block_offset; /* rest of this block */
27        if (bytes > size - bytes_read)
28            bytes = size - bytes_read;                  /* or less, if that is all we need */
29
30        memcpy(destination + bytes_read, buffer + block_offset, bytes);
31        bytes_read += bytes;
32    }
33
34    return bytes_read;
35 }

```

The same division as paging

`position / BLOCK_SIZE` and `position % BLOCK_SIZE`. It is exactly the arithmetic from Chapter 26, where a virtual address split into a page number and an offset within the page.

The idea is identical: a flat linear space is mapped onto fixed-size chunks that live somewhere scattered, and a division plus a remainder does the translation. Once you see this pattern, you see it everywhere in systems — pages, disk blocks, network packets, cache lines.

EOF is 0, not an error

The comment in the source says it: `//EOF should return 0 bytes instead of an error.` This matters enormously for the shell's `cat`:

```
1 while ((n = read(fd, buffer, sizeof(buffer))) > 0) {
2     write(1, buffer, n);
3 }
```

The loop ends on 0. If EOF returned `-1`, the condition `> 0` would also end the loop — but the caller could not distinguish “finished” from “disk error”. Getting these conventions right is what makes a filesystem usable.

In practice: creating, writing and reading files

```
examples/ch39-filesystem/ -- make run
```

Makes a directory, creates a file in it, writes to it, closes and reopens it to read the contents back, shows that a second read returns 0 for end of file rather than an error, lists both directories and finally deletes the file.

The close-and-reopen is the point. A descriptor carries an offset that only moves forwards and there is no `lseek`, so after writing eight bytes the offset is at eight and reading returns nothing. The kernel's own boot-time test does the same dance for the same reason.

CHAPTER 39

mkfs: formatting from outside

By the end of this chapter you will know

- Why a tool that runs on the host is needed
- How files get into the image before the system ever boots
- How the same header serves two completely different programs

39.1 The chicken and egg

The system needs `/bin/shell` to exist before it can run anything. But to create a file you need a running system. Something has to break the cycle from outside.

`mkfs` is that something: a program compiled for *your* Linux machine, not for `lytlnyblOS`, which opens the disk image as an ordinary file and writes the filesystem structures into it directly.

lytlnybl/Makefile

```
HOST_CC = gcc

$(MKFS): tools/mkfs.c
    mkdir -p $(dir $@)
    $(HOST_CC) -g -Iinclude $< -o $@
```

Note `HOST_CC = gcc`, not `i686-elf-gcc`. This one program is deliberately built for the host.

One header, two worlds

`mkfs.c` includes `include/filesystem/fs/_layout.h` — the *same* header the kernel uses. So the definitions of the superblock, inodes and directory entries are shared between a 64-bit Linux program and a 32-bit freestanding kernel.

This is why the structures are so carefully defined with fixed-width types (`uint32_t`, not `int`). A plain `int` might be a different size on each side, and the two programs would disagree about the layout of the disk — a bug that would look like random filesystem corruption.

39.2 What it does

```

1 int main(int argc, char **argv) {
2     FILE *image = fopen(argv[1], "r+b");    /* open the image read/write */
3
4     printf("formatting %s.....\n", argv[1]);
5
6     if (!format_filesystem(image)) { /* ... */ }
7
8     fs_superblock_t sb_copy;
9     uint8_t buffer[FS_BLOCK_SIZE];
10    read_block(image, FS_SUPERBLOCK, buffer);
11    memcpy(&sb_copy, buffer, sizeof(fs_superblock_t));
12
13    if (!install_rootfs(image, &sb_copy)) { /* ... */ }
14
15    fclose(image);
16    printf("mkfs success\n");
17 }

```

Two phases: write empty structures, then copy in the contents of `rootfs/`. The host-side block functions are the exact analogue of the kernel's:

```

1 static bool write_block(FILE *image, uint32_t block_num, const void *buffer) {
2     uint32_t sector = FS_START_BLOCK + block_num;
3     if (fseek(image, sector * FS_BLOCK_SIZE, SEEK_SET) != 0) return false;
4     return fwrite(buffer, 1, FS_BLOCK_SIZE, image) == FS_BLOCK_SIZE;
5 }

```

`fseek` plus `fwrite` here; `outb` plus `inw` in the kernel. Same logical operation, two completely different mechanisms, one shared layout.

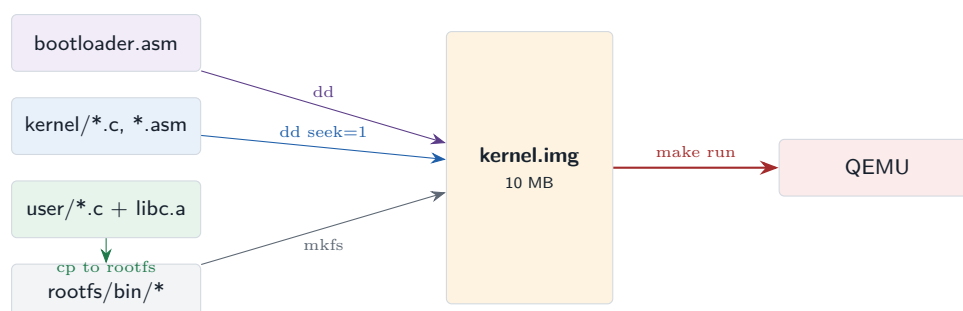
39.3 The build pipeline

lytlnybl/Makefile

```

$(KERNEL_IMG): $(BOOTLOADER) $(KERNEL_BIN) $(MKFS) $(USER_TEST_ROOTFS) $(SHELL_ROOTFS)
dd if=/dev/zero of=$@ bs=512 count=20480 # a blank 10 MB image
dd if=$(BOOTLOADER) of=$@ conv=notrunc # boot sector at sector 0
dd if=$(KERNEL_BIN) of=$@ seek=1 conv=notrunc # kernel from sector 1
$(MKFS) $@ # filesystem from sector 70

```



Try it yourself

Run `make` and watch the last lines:

```
formatting build/kernel.img.....  
mkfs success
```

Then boot. The kernel prints the contents of `/` and `/bin` during its self-test, and you should see `shell` and `user_test` listed — files that were placed there by a program running on your Linux machine, read back by a filesystem driver running on a 32-bit kernel you wrote.

In short

Storage works end to end: a disk driver, a filesystem, descriptors, and a way to get files in from outside. Everything the shell needs now exists. The next part assembles it.

PART XII

The shell

CHAPTER 40

The read-eval-print loop

By the end of this chapter you will know

- How a command interpreter is structured
- Follow a command from keystroke to effect
- See how every subsystem in the course is used at once

40.1 The whole program

lytlnybl/user/programs/shell.c

```
1 void start(void) {
2     for (;;) {
3         printf("> ");                                /* Print the prompt */
4
5         char input[50];
6         read(0, input, 50);                          /* Read */
7         input[strlen(input) - 1] = '\0';
8
9         size_t word_count;
10        char** words = tokenize_line(input, &word_count); /* Eval */
11
12        if (!execute_command(words, word_count)) {
13            printf("command failed\n");
14        }
15
16        memset(input, 0, strlen(input));               /* Loop */
17        word_count = 0;
18        free(words);
19    }
20 }
```

That is a **REPL**: Read, Eval, Print, Loop. Every interactive interpreter ever written — bash, Python, a database console — has this shape.

Everything you built, in eleven lines

Look at what those lines depend on:

- `printf` → `write` → `int 0x80` → the IDT → the VGA driver.
- `read` → the process blocks → the scheduler switches away → a key arrives → `IRQ1` → the keyboard driver writes into another address space → the process is woken.
- `malloc` inside `tokenize_line` → `sbrk` → the kernel maps a page → `alloc_frame`.
- `execute_command` → filesystem calls → path resolution → inodes → the ATA driver → I/O ports.

Every part of this course is exercised by one iteration of this loop.

40.2 Tokenising

lytlnybl/user/programs/shell.c

```

1 char** tokenize_line(const char* line, size_t *count_out) {
2     const char* p = line;
3     char **words;
4     size_t words_i = 0;
5     size_t capacity = 8;
6
7     words = calloc(capacity + 1, sizeof(char *));
8
9     while (*p != '\0') {
10         (*count_out)++;
11         const char* start;
12
13         while (isspace((unsigned char) *p)) p++;
14         if (*p == '\0') break;
15
16         start = p;
17         while (*p != '\0' && !isspace((unsigned char) *p)) p++;
18
19         size_t len = p - start;
20         char *word = malloc(len + 1);
21         if (word == NULL) return NULL;
22
23         memcpy(word, start, len);
24         word[len] = '\0';
25         words[words_i++] = word;
26     }
27     return words;
28 }
```

The algorithm is right: skip whitespace, mark the start, advance to the next whitespace, copy the slice. But the bookkeeping around it has three genuine defects.

Three bugs in the word counter

1. **word_count is never initialised.** In `start()` it is declared as `size_t word_count`; and passed by address. `tokenize_line` then does `(*count_out)++` on whatever garbage was on

the stack. Since `execute_command` tests `word_count != 2`, the shell's behaviour depends on an indeterminate value.

It works in practice only because the stack slot happens to be zero on the first iteration and the variable is reset to 0 at the end of each loop — which is itself an accident, because the reset happens *after* the value has already been used.

2. The increment is in the wrong place. `(*count_out)++` runs at the top of the loop, before checking whether a token actually exists. A trailing space makes the loop run one extra time and count a word that was never produced.

3. capacity is never enforced. The array holds 9 pointers. A line with 12 words writes past the end of a heap allocation, corrupting the next block's header — which, per Chapter 29, corrupts the allocator itself.

And a leak

`start()` calls `free(words)`, which releases the array of pointers — but every string inside it was separately `malloced` and is never freed. Each command leaks a few dozen bytes. With a 4 KB heap that grows by `sbrk`, the shell slowly consumes memory for as long as it runs.

The correct cleanup is a loop:

```
1 for (size_t i = 0; i < word_count; i++) free(words[i]);
2 free(words);
```

Why none of this is surprising

These are not sloppy mistakes; they are the normal state of code that has not yet been stress tested. The shell is the last thing built in the project, it works for the inputs it was tried with, and the bugs only appear at the edges: an empty line, a trailing space, twelve words, a long session.

The lesson worth taking is the one about *compiler help*. Note that `-Wall -Wextra` does **not** catch the uninitialised `word_count`, because its address is taken — GCC gives up on tracking it. Some bugs need tools beyond warnings: a sanitiser, a unit test, or simply trying the edge cases deliberately.

40.3 Executing

lytlnybl/user/programs/shell.c

```

1 bool execute_command(char** words, size_t word_count) {
2     if (word_count != 2) return false;      /* every command takes exactly one argument */
3
4     int fd;
5
6     if (strcmp(words[0], "ls") == 0) fs_ops(FS_LS, words[1]);
7     else if (strcmp(words[0], "mkdir") == 0) fs_ops(FS_MKDIR, words[1]);
8     else if (strcmp(words[0], "touch") == 0) fs_ops(FS_TOUCH, words[1]);
9     else if (strcmp(words[0], "rm") == 0) fs_ops(FS_RM, words[1]);
10    else if (strcmp(words[0], "run") == 0) fs_ops(FS_RUN, words[1]);
11    else if (strcmp(words[0], "cat") == 0) { /* ... */ }
12    else if (strcmp(words[0], "write") == 0) { /* ... */ }
13    else return false;
14
15    return true;
16 }

```

Exactly two words, always

if (word_count != 2) return false; means every command must have exactly one argument. So `ls` alone fails; you must type `ls /`. There is no way to implement a zero-argument command such as `help` or `exit` without changing this line.

Combined with the uninitialised counter above, this is why the shell can behave differently between runs.

In practice: adding commands to the shell

```
examples/ch40-shell-command/ -- make run
```

Adds `echo` and `rev` as two more branches in `execute_command` — which is all a new command is. Boot it and type `echo hello` or `rev lytlnybl`.

It also adds `help`, deliberately, to show what the `word_count != 2` rule costs: the command exists and works, but only if you type `help` followed by a second word, because anything that is not exactly two words is rejected before the branches are ever reached.

40.4 cat: reading a file

```

1 else if (strcmp(words[0], "cat") == 0) {
2     fd = open(words[1]);
3     if (fd < 0) return false;
4
5     char buffer[128];
6     int n;
7
8     while ((n = read(fd, buffer, sizeof(buffer))) > 0) {
9         write(1, buffer, n);
10    }

```

```

11     printf("\n");
12     close(fd);
13 }

```

This is the canonical Unix read loop, and it is correct in shape: read a chunk, write exactly what was read, stop at zero.

But it triggers the kernel bug from Chapter 34

`write(1, buffer, n)` with `n` up to 128 — and the kernel's write handler does `((char*)buffer)[count] = '\0';`. For a file whose content fills the buffer exactly, the kernel writes a zero byte to `buffer[128]`, one past the end of a 128-byte local array on the user's stack.

Two bugs meeting: a kernel that writes out of bounds and a caller that uses the full buffer. Either one alone would be harmless. This is how real security holes are made.

40.5 write: the growing buffer

```

1  size_t capacity = 128;
2  size_t length = 0;
3  char* buffer = malloc(capacity);
4
5  while (true) {
6      char input[128];
7      int n = read(0, input, sizeof(input));
8
9      if (n < 0) { free(buffer); close(fd); return false; }
10     if (n == 0) break;
11
12     if (length + n > capacity) {
13         while (length + n > capacity) capacity *= 2;
14         char *new_buffer = realloc(buffer, capacity);
15         if (new_buffer == NULL) { free(buffer); close(fd); return false; }
16         buffer = new_buffer;
17     }
18
19     memcpy(buffer + length, input, n);
20     length += n;
21
22     if (input[n-1] == '\n') break;
23 }

```

This one is genuinely well written, and it is worth saying so. It doubles capacity rather than growing by a constant (amortised $O(1)$ per byte). It checks `realloc` for failure. It keeps the old pointer until the new allocation succeeds, so a failure does not leak. It frees and closes on every error path.

That the same file contains both this and `tokenize_line` is a good illustration of how code quality varies within a project depending on how much thought a particular function received.

40.6 run: starting another program

lytlnybl/kernel/interrupts.c

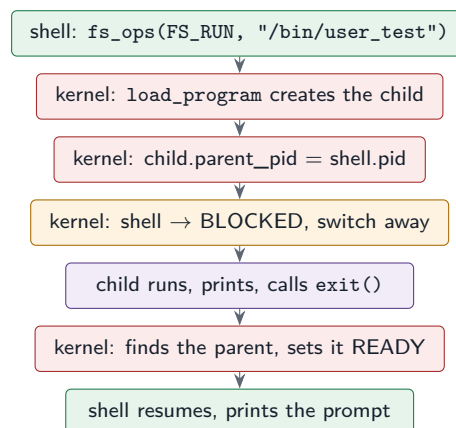
```

1 case FS_RUN: {
2     process_t* child = load_program(path);
3
4     child->parent_pid = current_process->pid;
5
6     current_process->pid_waiting_for = child->pid;
7     current_process->state = PROCESS_BLOCKED;
8     context_switch(current_process, get_next_process(), regs);
9     break;
10 }
```

And the other half, in exit:

```

1 case SYSCALL_EXIT: {
2     process_t* parent = find_process_by_pid(current_process->parent_pid);
3
4     if (parent != NULL && parent->pid_waiting_for == current_process->pid) {
5         parent->pid_waiting_for = 0;
6         parent->state = PROCESS_READY;      /* wake the shell */
7     }
8
9     current_process->state = PROCESS_TERMINATED;
10    context_switch(current_process, get_next_process(), regs);
11    break;
12 }
```



This is fork + exec + wait, collapsed into one

Unix separates these into three system calls, which is more flexible — it lets the shell set up redirection between forking and exec'ing. lytlnyblOS merges them, which is much simpler and enough for `run`.

The blocking-and-waking mechanism, though, is exactly the real thing: the parent records

which child it waits for, goes to sleep, and the child's exit path wakes it. That handshake is the heart of process management in every Unix.

The full round trip, verified

On a running system:

```
> run /bin/user_test
ab
i have a number it is: 314
malloc: hi
calloc: abc
realloc: hellooo
>
```

The child ran, exercised `printf`, `malloc`, `calloc` and `realloc` in user space, exited, and the shell got its prompt back. Every mechanism in Parts IX through XII, working together.

Unless the file does not exist

Type a path that does not exist — a typo is enough — and the missing NULL check from Chapter 33 fires:

```
> run /bin/usertest
(nothing. the prompt never returns. further typing echoes but does nothing.)
```

The shell is blocked waiting for a child that was never created, with a pid read out of memory near address zero. Nothing will ever wake it. This is reproducible on the current code and is the single easiest serious bug in the project to fix.

In short

The system is complete: it boots, protects itself, manages memory, runs isolated programs, stores files and gives you a command line. The last part steps back and looks at it as an engineer rather than a student.

PART XIII

Engineering: building, debugging, improving

CHAPTER 41

The build system

By the end of this chapter you will know

- Read the project Makefile and understand every rule
- Understand why there are two compilers
- Know the weaknesses of this build and how to fix them

41.1 Two compilers, one project

lytlnybl/Makefile

```
CC = i686-elf-gcc           # for code that runs on lytlnyblos
HOST_CC = gcc               # for mkfs, which runs on your machine
LD = ld
NASM = nasm
OBJCOPY = objcopy
AR = i686-elf-ar

CFLAGS = -g -m32 -Iinclude -ffreestanding
USER_CFLAGS = $(CFLAGS) -fno-pie -fno-pic
```

`-fno-pie -fno-pic` for user programs is worth explaining. Modern compilers default to generating *position-independent code*, which can be loaded anywhere — useful for address space randomisation on a real OS. But our loader maps every program at exactly `0x400000` and there is no dynamic relocation, so position independence is pure overhead and can produce constructs the loader would have to handle. Turning it off gives plain absolute code.

41.2 Finding sources automatically

```
KERNEL_C_FILES = $(shell find kernel memory filesystem tasks -name '*.c')
KERNEL_ASM_FILES = $(shell find kernel memory tasks -name '*.asm')

KERNEL_C_OBJECTS = $(KERNEL_C_FILES:%.c=$(BUILD_DIR)/%.o)
KERNEL_ASM_OBJECTS = $(KERNEL_ASM_FILES:%.asm=$(BUILD_DIR)/%.asm.o)
```

`$(shell find ...)` runs a command at parse time and captures its output, so adding a new `.c` file to `kernel/` requires no Makefile change. The substitution `$(FILES:%.c=$(BUILD_DIR)/%.o)` turns `kernel/interrupts.c` into `build/kernel/interrupts.o`, mirroring the source tree inside `build/`.

41.3 Pattern rules

```
$(BUILD_DIR)/%.o: %.c
    mkdir -p $(dir $@)
    $(CC) $(CFLAGS) -c $< -o $@

$(BUILD_DIR)/%.asm.o: %.asm
    mkdir -p $(dir $@)
    $(NASM) -f elf32 -g -F dwarf $< -o $@
```

Variable	Means
<code>\$@</code>	The target being built
<code>\$<</code>	The first prerequisite
<code>\$^</code>	All prerequisites
<code>\$(dir \$@)</code>	The directory part of the target

`-f elf32` tells NASM to produce a relocatable object file rather than a flat binary, so the linker can combine it with the C objects. `-g -F dwarf` adds debug information in the format GDB expects — which is what makes stepping through assembly in the debugger possible.

41.4 The critical link order

```
$(KERNEL_ELF): $(KERNEL_C_OBJECTS) $(KERNEL_ASM_OBJECTS)
    $(LD) -m elf_i386 -T $(KERNEL_LINKER) \
        $(KERNEL_ENTRY_OBJECT) \
        $(KERNEL_C_OBJECTS) \
        $(filter-out $(KERNEL_ENTRY_OBJECT),$(KERNEL_ASM_OBJECTS)) \
        -o $@
```

Why `kernel_intro.asm.o` must be first

The linker lays out `.text` in the order the object files appear. The linker script puts `.text` at `0x9000`, and the bootloader jumps to `0x9000`.

Therefore **whatever code is first in the link line ends up at the entry point**. `$(KERNEL_ENTRY_OBJECT)` is listed explicitly first, and `filter-out` removes it from the rest so it is not linked twice.

If that ordering broke, the bootloader would jump into the middle of some random function,

in 16-bit mode, with no diagnostic whatsoever. Note the linker already warns about a related issue:

```
ld: warning: cannot find entry symbol start; defaulting to 00009000
```

because `start:` in the assembly file is not declared `global`. It works only because of the link order. Adding `global start` would make the intent explicit and the build robust.

41.5 From ELF to a disk image

```
$(KERNEL_BIN): $(KERNEL_ELF)
$(OBJCOPY) -O binary $< $@
```

An ELF file has headers, a symbol table, section metadata and debug information. The bootloader understands none of that — it copies raw sectors to 0x9000 and jumps. `objcopy -O binary` strips everything and leaves just the bytes, in the order the linker placed them.

Both files are kept: `kernel.bin` goes into the image, and `kernel.elf` stays behind for GDB, which needs the symbols.

41.6 What is missing from this build

No header dependencies

This is the most practically annoying weakness. The rule says `$(BUILD_DIR)/%.o: %.c` — the object depends on the `.c` file and nothing else.

So editing `include/memory/vmm.h` rebuilds **nothing**. You change a struct, run `make`, it says “nothing to be done”, and you spend the next hour debugging a binary built from two incompatible versions of the same header.

The fix is two lines:

```
CFLAGS += -MMD -MP
-include $(shell find $(BUILD_DIR) -name '*.d' 2>/dev/null)
```

`-MMD` makes GCC emit a `.d` file listing every header the source included, and `-include` pulls those in as extra rules. After that, touching a header rebuilds exactly what depends on it.

No warnings enabled

`CFLAGS` has no `-Wall` `-Wextra`. Turning them on for this project produces only a handful of findings — a few unused variables and one intentional switch fall-through — so the cost of adopting them is almost zero and the future benefit is large.

Be aware of the limits, though: `-Wall` `-Wextra` does *not* catch the uninitialised `word_count`

in the shell, because taking its address defeats the analysis. Warnings are a floor, not a ceiling.

Host linker, and generated binaries in git

LD = ld uses the host linker with `-m elf_i386`. It works on an x86-64 Linux host but would fail on, say, an ARM machine where the host ld cannot target i386. `i686-elf-ld` would be correct.

Separately, `rootfs/bin/shell` and `rootfs/bin/user\_test` are tracked in git even though they are build outputs. They show as modified after every build, which clutters every diff. They belong in `.gitignore`.

Debugging a kernel

By the end of this chapter you will know

- Know the four techniques available and when each applies
- Be able to use GDB against a running kernel
- Be able to read a QEMU interrupt trace

42.1 Four techniques

Technique	Cost	Best for
Print to screen	trivial	“Does it reach this line?” “What is this value?”
GDB via QEMU	medium	Inspecting state at a precise point; stepping
QEMU tracing	low	Faults, resets, interrupt storms — things that happen too fast to see
Reading the code	free	Everything else, and you will do more of this than you expect

42.2 Printing

Crude and extremely effective. The kernel does it itself before any driver exists:

```
1 volatile char* vga = (volatile char*)0xB8000;
2 vga[0] = 'C'; vga[1] = 0x02;
```

Two bytes and you know execution reached that line. Using different screen positions for different checkpoints gives a crude progress display that survives even a completely broken driver.

But printing can be the thing that is broken

Chapter 20 showed the failure mode: if `terminal` is corrupt, every print attempt faults. When the system dies *during* your debug output, printing tells you nothing. That is exactly when the next two techniques earn their keep.

42.3 GDB against QEMU

QEMU can expose a GDB server and start paused:

```
lytlnybl/Makefile
run: $(KERNEL_IMG)
    qemu-system-i386 -drive format=raw,file=$(KERNEL_IMG) #-s -S
```

Uncomment `-s -S`: `-s` opens a GDB server on port 1234, `-S` freezes the CPU before the first instruction. Then, in another terminal:

```
gdb build/kernel.elf
(gdb) target remote localhost:1234
(gdb) break kernel_main
(gdb) continue
```

The project ships a `.gdbinit` to automate the connection.

42.3.1 Commands worth knowing

Command	Does
info registers	Show every register, including <code>CRO</code> , <code>CR2</code> , <code>CR3</code>
x/16xb 0xB8000	Dump 16 bytes in hex from video memory
x/10i \$eip	Disassemble the next 10 instructions
p/x terminal	Print a struct in hex
watch terminal.width	Break when a variable changes
bt	Backtrace — the chain of calls
stepi / nexti	Step one machine instruction

watch is the one that finds memory corruption

When a variable is being overwritten by something and you have no idea what, a hardware watchpoint is the tool. `watch terminal.width` tells the processor to trap on any write to that address, and GDB stops you at the exact instruction responsible — which may be in a completely unrelated file.

For the bug in the next chapter, `watch terminal.width` would have pointed straight at a `memcpy` inside the program loader.

42.4 QEMU tracing

For faults that happen faster than you can observe, QEMU can log every interrupt and every CPU reset:

```
qemu-system-i386 -drive format=raw,file=build/kernel.img \
  -display none -no-reboot \
  -d int,cpu_reset -D trace.log
```

Option	Effect
-d int	Log every interrupt and exception with full register state
-d cpu_reset	Log every processor reset
-no-reboot	Stop instead of rebooting on triple fault
-D file	Write the log to a file

-no-reboot is the important one. Without it, a triple fault silently restarts the machine and you see a boot loop with no information. With it, QEMU prints **Triple fault** and exits, leaving the log ending exactly at the failure.

A log line looks like this:

```
278: v=0e e=0002 i=0 cpl=0 IP=0008:0000ad98 pc=0000ad98 SP=0010:c0000000 CR2=bfffffff
```

Field	Meaning
278	Sequence number — how many interrupts have occurred
v=0e	Vector 0x0E = 14 = page fault
e=0002	Error code: bit 1 set = it was a write, bit 0 clear = page not present
cpl=0	Current privilege level: the kernel
IP=0008:0000ad98	Where it faulted
SP=0010:c0000000	Stack pointer at the time
CR2=bfffffff	The address that was accessed

42.5 Reading the screen without a screen

One more trick, useful for automated testing. QEMU's monitor can capture the framebuffer:

```
qemu-system-i386 -drive format=raw,file=build/kernel.img \
  -display none -monitor unix:/tmp/mon.sock,server,nowait
```

Connect to that socket and issue `screendump /tmp/shot.ppm`. You get a 720×400 image of the text screen.

pmemsave will not read video memory

It is tempting to dump 0xB8000 directly with the monitor's `pmemsave` command. It returns zeros: that address is a device region, not RAM, and the command only reads RAM.

`screendump` works because it asks the display device for its rendered output. Since the VGA font is a known 8×16 bitmap (extractable from the `vgabios.bin` that ships with QEMU), each 9×16 cell of the image can be matched back to a character — giving you the text of the screen as a string, from a script, with no display attached. That is how the boot output quoted throughout this course was captured.

CHAPTER 43

Case study: a reboot loop

By the end of this chapter you will know

- Follow a complete diagnosis from symptom to root cause
- See how to work backwards along a chain of causes
- Learn the method, not just this answer

This chapter walks through a real fault in this project, diagnosed on a real machine. The intermediate observations are exactly those produced by the tools, not reconstructed afterwards.

43.1 The symptom

A clean rebuild of the project boots, prints its self-test output, reaches the line `Running shell:` — and then the screen goes blank and the BIOS banner appears again. And again. A reboot loop, with no error message.

Curiously, the *prebuilt* image already in the repository boots fine and reaches a working shell prompt. Same source, same commands, different behaviour.

First observation: “it works for me” is data

Two builds of identical source behaving differently is not noise. It says the fault depends on something that varies between builds — almost always *layout*: addresses, sizes, alignment. That single observation already eliminates whole categories of bug (logic errors, race conditions) and points at memory.

43.2 Step 1: catch the fault instead of the reboot

```
qemu-system-i386 -drive format=raw,file=build/kernel.img \  
-display none -no-reboot -d int,cpu_reset -D trace.log
```

QEMU now exits with `Triple fault` instead of rebooting. The tail of the log:

```

278: v=0e e=0002 i=0 cpl=0 IP=0008:0000ad98 SP=0010:c0000000 CR2=bfffffff
...
279: v=08 e=0000 i=0 cpl=0 IP=0008:0000ad98 SP=0010:c0000000
...
check_exception old: 0x8 new 0xe
Triple fault

```

Reading it with the table from the previous chapter:

- `v=0e`: a page fault, `e=0002` means a write to a non-present page.
- `CR2=bfffffff`: the address accessed was `0xBFFFFFFC`.
- `SP=c0000000`: the stack pointer was `0xC0000000`.
- Then `v=08`: a double fault. Then triple fault.

Step 1 conclusion

`ESP` is `0xC0000000` and the faulting address is `0xBFFFFFFC` — exactly four bytes below. A push. **This is a stack overflow**, matching the signature from Chapter 28 precisely.

And `0xC0000000` is `HEAP_START`. So a stack allocated from the kernel heap has grown down past the very beginning of the heap.

43.3 Step 2: find out where

```
i686-elf-nm -n build/kernel.elf | grep " [tT] "
```

Matching `IP=0000ad98` against the symbol table gives `vga_text_writeline + 0x0` — the function's very first instruction, `push ebp`. Consistent: the stack was already exhausted, and the first push of the next call tipped it over.

But that only says *where it ran out*, not *why*. For that, look further back in the log.

43.4 Step 3: the pattern before the crash

```

240: v=00 e=0000 i=0 cpl=0 IP=0008:0000ad47 SP=0010:c00016ec
241: v=00 e=0000 i=0 cpl=0 IP=0008:0000ad47 SP=0010:c0001650
242: v=00 e=0000 i=0 cpl=0 IP=0008:0000ad47 SP=0010:c00015b4
...
277: v=00 e=0000 i=0 cpl=0 IP=0008:0000ad47 SP=0010:c0000060
278: v=0e ... SP=0010:c0000000

```

This is the answer, and it is unmistakable:

- `v=00` is **exception 0: divide by zero**.

- The same instruction address, 0xAD47, every time.
- ESP decreasing by exactly 0x9C (156 bytes) each round.
- Exactly 107 repetitions (log entries 171 to 277) before the stack is exhausted — and $107 \times 0x9C = 16,692$ bytes, which is the 16 KB kernel stack.

Step 3 conclusion

An infinite recursion. Something divides by zero; the handler tries to print the error; printing divides by zero again. Each round consumes 156 bytes of stack and never returns. Exactly the scenario described in Chapter 20 — now observed rather than predicted.

43.5 Step 4: what is being divided

```
i686-elf-objdump -d -S build/kernel.elf --start-address=0xad30 --stop-address=0xad80
```

```
ad39:  mov 0x8(%ebp),%eax
ad3c:  mov 0x8(%eax),%ecx      ; ECX = terminal->width
ad3f:  mov -0xc(%ebp),%eax     ; EAX = pos
ad42:  mov $0x0,%edx
ad47:  div %ecx                ; <-- the faulting instruction
```

Which is this source line:

```
1 terminal->row = pos / terminal->width;
```

So `terminal->width` is zero. But `vga_text_init` sets it to 80 during boot, and the system clearly printed many lines successfully before failing. **Therefore something overwrote it.**

43.6 Step 5: who could overwrite it

```
i686-elf-nm build/kernel.elf | grep -E " (B|b) |_kernel_end"
```

```
00013188 B terminal
00013400 B tss
00013468 B _kernel_end
```

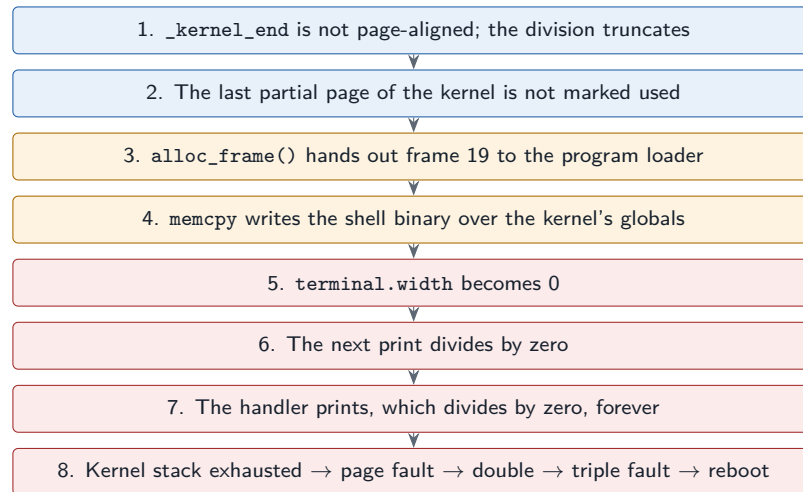
Now the arithmetic from Chapter 25:

$$0x13468 \div 0x1000 = 19.27 \rightarrow 19$$

The protection loop is for (`i = 9; i < 19; i++`), protecting frames 9–18, that is 0x9000–0x12FFF. **Frame 19 — 0x13000–0x13FFF — is left free.** And `terminal` is at 0x13188,

inside it.

The chain is now complete:



43.7 Step 6: confirm by fixing

A hypothesis is not a diagnosis until the fix proves it. One line:

```
uint32_t kernel_frame_end = ((uint32_t)&_kernel_end + FRAME_SIZE - 1) / FRAME_SIZE;
```

Rebuild, boot:

```

Welcome to the lytlnybIOS!
Kernel multiprocessing tests:
NEW KERNEL PROCESS RUNNING
BACK IN MAIN PROCESS
Memory allocation tests:
Values: 10 20 30 40 50
Filesystem tests:
Root directory contents:
bin file.txt notes.txt
Bin directory contents:
user_test shell
Running shell:
>

```

Stable, with a working prompt, and still stable ten seconds later. Confirmed.

43.8 Why the prebuilt image was fine

Because `_kernel_end` lands somewhere different in every build. In the repository's prebuilt image, `kernel.bin` is 22,920 bytes; in the clean rebuild it is 22,888. A 32-byte difference — from nothing more than toolchain version — moved which globals fall in the unprotected last page.

The general lesson

The bug was always there. It was present in every build, including the ones that worked perfectly. What varied was only whether it had visible consequences.

This is the defining property of latent memory bugs, and why “it works” is such weak evidence in systems programming. The fix is not to test more; it is to remove the class of error — here, by always rounding up when converting a byte address to a page index.

In practice: reproducing the reboot loop

```
examples/ch45-reboot-loop/ -- make run
```

Injects the fault deliberately — a kernel process sets `terminal.width = 0` and then prints — so the failure is reproducible instead of depending on where `_kernel_end` happens to land.

`make trace` is the target to use. It boots with `-no-reboot -d int,cpu_reset`, stops at the triple fault instead of restarting, and prints the last exceptions. You get the signature to recognise: a run of `v=00` at one fixed instruction address with `SP` falling by exactly `0x9C` each time.

The addresses will not match the ones quoted above — the fault is injected at a different point in boot, on a different stack — but the recursion signature is identical, and that is the part worth being able to spot.

43.9 The method, generalised

1. **Make the failure stop instead of hiding.** `-no-reboot` turns an information-free reboot loop into a log that ends at the fault.
2. **Read the last fault precisely.** Vector, error code, `CR2`, `ESP`. Each is a strong constraint.
3. **Look for a pattern before it.** A repeating vector at a fixed address with a monotonically moving `ESP` is recursion. One-off faults are different bugs entirely.
4. **Map addresses back to source.** `nm` and `objdump` turn a hex address into a line of C.
5. **Ask what could have written that value** rather than what read it. The reader is rarely the culprit.
6. **Prove it with a fix.** If the minimal fix does not eliminate the symptom, the diagnosis is wrong, however convincing it sounded.

Known issues and how to fix them

By the end of this chapter you will know

- Have a single list of every weakness identified in this course
- Know the relative priority of each
- Have a concrete starting point for contributing

Everything below was identified while writing this course, by reading the code and by running the system. They are grouped by severity. None of them makes the project less valuable as teaching material — several of them make it *more* valuable, because working through the fix teaches more than reading correct code would.

44.1 Serious: correctness and safety

Issue	Effect	Where
Unvalidated user pointers	A user program can make the kernel write a zero byte to any address, including kernel memory	<code>interrupts.c</code> , <code>SYSCALL_WRITE</code>
Write one byte past count	Corrupts the caller's buffer; triggered by the shell's <code>cat</code>	<code>interrupts.c</code>
<code>load_program</code> return not checked	run on a missing file hangs the shell forever	<code>interrupts.c</code> , <code>FS_RUN</code>
Page index rounding	Latent: the kernel's last partial page could be handed out	<code>pmm.c</code> — fixed , see Ch. 45
VLA on the kernel stack	Programs over ~14 KB overflow the 16 KB kernel stack	<code>loader.c</code>
<code>destroy_process</code> head removal	A terminated head process is never unlinked	<code>procman.c</code>
<code>unmap_page</code> on the kernel directory	Punches a permanent hole in the identity map shared by all processes	<code>procman.c</code>
No <code>free_frame</code> on teardown	Physical memory leaks on every process exit	<code>procman.c</code>
<code>kfree</code> of the running kernel stack	Use-after-free during <code>exit</code>	<code>context.c</code>

44.2 Moderate: robustness

Issue	Effect	Where
No idle process	<code>get_next_process</code> can return a non-runnable process; the kernel busy-waits instead of halting	<code>scheduler.c</code>
<code>_start</code> vs <code>start</code>	The shell's entry point works only by link order	<code>shell.c</code> , <code>user.ld</code>
Uninitialised <code>word_count</code>	Shell behaviour depends on stack garbage	<code>shell.c</code>
<code>tokenize_line</code> capacity	More than 8 words overflows a heap allocation	<code>shell.c</code>
Word strings never freed	The shell leaks memory per command	<code>shell.c</code>
<code>read</code> result ignored	<code>strlen</code> on a non-terminated buffer	<code>shell.c</code>
User stack is one page	Deep recursion kills the process instead of growing the stack	<code>procman.c</code>
Only 4 MB identity-mapped	A ceiling on total allocation; faults if crossed	<code>vmm.c</code>
No ATA timeouts	A stuck disk hangs the kernel forever	<code>ata.c</code>
Global descriptor table	Descriptors are shared between processes and leak on <code>exit</code>	<code>fs_manager.c</code>

44.3 Minor: hygiene

- No `-MMD` header dependencies in the Makefile — editing a header rebuilds nothing.
- No `-Wall -Wextra`.
- `LD = ld` rather than `i686-elf-ld`.
- `rootfs/bin/*` tracked in git although generated.
- `__attribute__((packed))` on `process_t`, where it is unnecessary and harmful.
- `syscall_entry` pushes the interrupt number and error code in the opposite order to the ISR stubs.
- `timer_wait_ms` counts ticks, not milliseconds, despite its name.
- The intentional fall-through in `keyboard_handler` is not marked as such.
- `vga_text_write_dec(t, 0)` prints the '0' but does not advance the cursor, so the next thing written covers it and the zero disappears. Every other value goes through a loop that advances correctly. Found by running the Chapter 27 example, and hit again in the Chapter 39 one.
- No scrolling: the screen wraps instead.
- VGA text wraps at 60 sectors' worth of kernel with no build-time check.

44.4 A suggested order of work

If you want to improve the project, this order gives the most benefit for the least risk:

1. **-MMD in the Makefile.** Five minutes, and it removes an entire class of confusing rebuild bugs. Do this before touching anything else.
2. **The three-line NULL check in FS_RUN.** Turns a hang into an error message.
3. **Round up in pmm.c.** Already done — see Chapter 45 — but read the diagnosis, because the same truncation pattern can appear anywhere a byte address is converted to a page or block index.
4. **A copy_from_user / copy_to_user pair.** The highest-value change in the codebase; closes the whole unvalidated-pointer category.
5. **An idle process.** Makes the scheduler correct by construction and lets the CPU actually sleep.
6. **Fix destroy_process** — unlink correctly, stop unmapping from kernel_directory, free the frames, defer the kfree of the running stack.
7. **Stream the program loader** page by page instead of the VLA.
8. **Shell hygiene:** initialise the counter, enforce capacity, free the words, use read's return value.

CHAPTER 45

Where to go from here

By the end of this chapter you will know

- Know which extensions are worth attempting and in what order
- Understand what each one teaches

45.1 Extensions, by difficulty

45.1.1 Approachable

Screen scrolling

Move 24 rows up with `memcpy` and clear the last. Teaches: the VGA buffer layout, and why terminals behave as they do.

Arrow keys

The extended-scancode path already exists and does nothing. Fill it in, then add a command history to the shell. Teaches: scancode handling and line editing.

More commands

`help`, `clear`, `echo`. Requires first relaxing the `word_count != 2` restriction. Teaches: argument parsing.

Buffered `printf`

Accumulate and flush on newline. Teaches: why real libraries buffer, and how much system calls cost.

45.1.2 Moderate

Growing the user stack on demand

Catch a page fault just below the stack region and map a new page. Fifteen lines that turn a crash into a feature. Teaches: what page faults are really for.

Indirect blocks

An eleventh inode pointer that points at a block of pointers. Raises the file size limit from 5 KB to ~69 KB. Teaches: how real filesystems scale.

`.` and `..`, and a working directory

Teaches: path resolution and per-process state.

Keyboard ring buffer

Move the interrupt handler's writes into a kernel buffer and copy to user space in process context. Teaches: why interrupt handlers should be short, and removes the **CR3** manipulation.

Priorities in the scheduler

Give interactive processes shorter, more frequent slices. Teaches: what schedulers actually do.

45.1.3 Ambitious**ELF loading**

Parse real ELF headers instead of flat binaries. Teaches: how program loading works everywhere else, and enables separate read-only text and writable data.

Separate fork and exec

Requires copying an address space. Copy-on-write makes it efficient and is a beautiful use of page faults. Teaches: the core of Unix process creation.

Disk interrupts instead of polling

Frees the CPU during I/O. Teaches: asynchronous I/O and why it complicates everything.

Higher-half kernel

Move the kernel to 0xC0000000 and give user space the whole lower range. Teaches: how real systems lay out address spaces, and a great deal about paging.

Pipes

`ls / | cat`. Requires a kernel buffer, blocking reads and writes, and shell syntax. Teaches: inter-process communication.

45.2 A closing thought

The system in this course is about 5,500 lines. Linux is around 30 million. The difference is not conceptual — it is drivers, architectures, filesystems, protocols, decades of edge cases, and performance work.

The concepts you have now are the same ones. A page table in Linux is a page table here. A context switch there saves the same registers. A system call crosses the same boundary in the same way. When you read that a CPU vulnerability breaks the isolation between user space and kernel space, you now know exactly what that sentence means and exactly which bit is involved.

That is what this was for.

In short

You started not knowing what a register was. You now understand a complete operating system, you can read a processor trace and find a memory corruption bug from it, and you have a list of real improvements you could make to a real codebase. The appendices that follow are reference material for when you do.

PART XIV

Appendices

CHAPTER A

Glossary

Address space

The complete set of addresses a process can name. Each user process has its own, defined by its page directory.

ATA / IDE

The interface used to talk to the disk. *PIO* means the processor moves the data itself, one word at a time.

BIOS

The PC firmware. Initialises hardware, loads the boot sector, and offers services that stop working once you enter protected mode.

Bitmap (memory)

One bit per 4 KB frame recording whether it is in use.

Block

The filesystem's allocation unit. 512 bytes here, the same as a disk sector.

Boot sector

The first 512 bytes of a disk, ending in 0xAA55. The BIOS loads it at 0x7C00.

Context

Everything that must be saved to suspend a process: registers, **EIP**, **EFLAGS**, stack, address space.

Context switch

Replacing the running process's context with another's.

CR0 / CR2 / CR3

Control registers: mode switches, faulting address, active page directory.

Descriptor

An 8-byte record in the GDT or IDT describing a segment or an interrupt handler.

Directory (filesystem)

A file whose contents are (inode, name) records.

DPL

Descriptor Privilege Level. The ring required to use a segment or gate.

EOI

End Of Interrupt. The acknowledgement the PIC needs before it will deliver another interrupt.

Exception

An interrupt generated by the processor itself when an instruction cannot complete.

File descriptor

A small integer indexing the kernel's open-file table. 0, 1 and 2 are reserved for keyboard and screen.

Frame

A 4 KB chunk of physical RAM.

GDT

Global Descriptor Table. Defines the memory segments and their privilege levels.

Heap

A region from which variable-sized allocations are made.

IDT

Interrupt Descriptor Table. 256 entries mapping interrupt numbers to handlers.

Identity mapping

A mapping where virtual address X corresponds to physical address X .

Inode

The structure describing a file: size, type and block pointers. Not its name.

Interrupt

A mechanism by which hardware or software diverts the processor to a kernel routine.

IRQ

A hardware interrupt line. IRQ0 is the timer, IRQ1 the keyboard.

Kernel

The privileged core of the operating system, running in ring 0.

LBA

Logical Block Addressing. Sectors numbered linearly from 0.

Little-endian

Storing the least significant byte at the lowest address. x86 does this.

Page

A 4 KB chunk of virtual address space.

Page directory

A table of 1024 entries, each pointing at a page table. CR3 points at it.

Page fault

Exception 14, raised when a translation cannot be completed.

Page table

A table of 1024 entries, each describing one 4 KB page.

Paging

Translating virtual addresses to physical ones in 4 KB units, enforced by hardware.

PCB

Process Control Block. The structure holding everything about a process.

PIC

Programmable Interrupt Controller (8259). Routes device interrupts to the CPU.

PIT

Programmable Interval Timer (8253/8254). Generates periodic interrupts.

PID

Process identifier.

Polling

Repeatedly asking a device whether it is ready, instead of waiting for an interrupt.

Preemption

Taking the processor from a process without its cooperation.

Protected mode

The 32-bit x86 mode with rings, segment permissions and paging.

Quantum

The number of timer ticks a process runs before being preempted.

Real mode

The 16-bit x86 mode the processor starts in. 1 MB, no protection.

Ring

A privilege level. 0 is the kernel, 3 is user space.

Round robin

A scheduling policy giving every ready process a turn in order.

RPL

Requested Privilege Level. The low two bits of a segment selector.

Scancode

The number a keyboard sends for a physical key. Not a character.

Scheduler

The code that decides which process runs next.

Sector

The disk's physical unit, 512 bytes.

Segment

A region of memory described by a GDT descriptor. Flat in this system.

Selector

A 16-bit value indexing the GDT and carrying an RPL.

Superblock

Block 0 of the filesystem, describing the filesystem itself.

System call

A request from a user program to the kernel, made with `int 0x80`.

TLB

Translation Lookaside Buffer. Caches recent address translations.

Triple fault

A fault while handling a double fault. The processor resets.

TSS

Task State Segment. In practice, only `esp0` and `ss0` matter.

User space

Ring 3. Where unprivileged programs run.

Virtual address

An address as a program sees it, before translation.

CHAPTER B

x86 quick reference

B.1 General-purpose registers

32-bit	16-bit	8-bit	Conventional use
EAX	AX	AH, AL	Return value; system call number
EBX	BX	BH, BL	System call argument 1
ECX	CX	CH, CL	Loop counter; argument 2
EDX	DX	DH, DL	I/O port; argument 3
ESI	SI	—	Source pointer
EDI	DI	—	Destination pointer
EBP	BP	—	Frame pointer
ESP	SP	—	Stack pointer

B.2 Instructions used in this project

Instruction	Effect
mov d, s	Copy s into d
push v / pop d	Stack: subtract 4 and store / load and add 4
pusha / popa	Push or pop all eight general-purpose registers
call a / ret	Push return address and jump / pop and jump
iret	Return from interrupt: pop EIP, CS, EFLAGS (and SS, ESP if the ring changes)
jmp s:o	Far jump: change CS and EIP together
cmp a, b	Subtract and discard, keeping only the flags
test a, b	AND and discard, keeping only the flags
in d, dx / out dx, s	Read from / write to an I/O port
cli / sti	Clear / set the interrupt flag
lgdt / lidt / ltr	Load GDT / IDT / task register
invlpg [a]	Invalidate one TLB entry
lods b	AL = [DS:SI]; SI++
hlt	Halt until the next interrupt

B.3 CPU exceptions

No.	Name	Err?	Usual cause here
0	Divide By Zero	no	Division by a zero variable
6	Invalid Opcode	no	Jumped into data
8	Double Fault	yes	A fault while handling a fault
13	General Protection	yes	Privilege violation, bad selector
14	Page Fault	yes	Unmapped address, or permission denied

B.4 Page table entry flags

Bit	Value	Meaning
0	0x1	Present
1	0x2	Writable
2	0x4	User-accessible (ring 3)
5	0x20	Accessed
6	0x40	Dirty

CHAPTER C

Memory map of lytlnyblOS

C.1 Physical memory

Address	Size	Contents
0x00000	1 KB	Interrupt vector table (real mode)
0x00400	256 B	BIOS data area
0x04FFC	2 B	E820 entry count (written by the bootloader)
0x05000	varies	E820 memory map
0x07C00	512 B	Boot sector
0x09000	~23 KB	The kernel (<code>_kernel_start</code>)
0x13468	—	<code>_kernel_end</code> (exact value varies per build)
0xA0000	128 KB	Video memory
0xB8000	4 KB	VGA text buffer
0xC0000	256 KB	BIOS and option ROMs
0x100000	4 KB	Physical memory bitmap
0x101000	rest	Free RAM for allocation

C.2 Virtual memory (per process)

Address	Constant	Contents
0x00000000	—	Identity-mapped kernel, first 4 MB
0x00400000	<code>USER_CODE_BASE</code>	User program code
0x00A00000	<code>USER_HEAP_START</code>	User heap, grows up
0x00B00000	<code>USER_VGA</code>	Reserved for direct user VGA access
0xBFFFF000	<code>USER_STACK_TOP</code>	Top of the user stack, grows down
0xC0000000	<code>HEAP_START</code>	Kernel heap, grows up

C.3 Disk image

Sector	Constant	Contents
0	—	Boot sector
1–60	—	Kernel (60 sectors are read by the BIOS)
70	FS_START_BLOCK	Filesystem superblock
71	FS_BITMAP_BLOCK	Block bitmap
72–83	FS_INODE_START	Inode table (128 inodes)
84+	—	Data blocks

CHAPTER D

System call reference

EAX	Name	EBX / ECX / EDX	Returns
0x01	exit	—	Never
0x02	getpid	—	The pid
0x03	yield	—	—
0x04	sleep	ticks	—
0x05	write	fd / buffer / count	Bytes written, or -1
0x06	read	fd / buffer / count	Bytes read, 0 at EOF, or -1
0x07	sbrk	increment	The new heap end
0x08	fsops	operation / path	—
0x09	open	path	A descriptor (≥ 3), or -1
0x0A	close	fd	—

D.1 fsops sub-operations

EBX	Name	Effect
0	FS_LS	List a directory
1	FS_MKDIR	Create a directory
2	FS_TOUCH	Create an empty file
3	FS_RM	Delete a file
4	FS_RUN	Load and run a program, and wait for it

D.2 Reserved descriptors

fd	Name	Device
0	stdin	Keyboard (blocks until Enter)
1	stdout	Screen
2	stderr	Screen
≥ 3	—	Open files

CHAPTER E

Source file index

File	Ch.	Responsibility
boot/bootloader.asm	12–15	Boot sector: print, E820, load the kernel, jump
kernel/kernel_intro.asm	16–18	GDT, protected mode, kernel stack, enter C
kernel/linker.ld	19	Places the kernel at 0x9000
kernel/kernel_main.c	19	Subsystem initialisation and self-tests
kernel/kernel_utils.c	19	memset, memcpy, strcmp, strncmp
kernel/interrupts.asm	21	ISR/IRQ stubs, port I/O, syscall entry
kernel/interrupts.c	21–22	IDT setup, PIC remap, exception and syscall dispatch
kernel/drivers/vga_text.c	19–20	Text mode output
kernel/drivers/timer.c	23	PIT, tick counter, wake sleepers, call the scheduler
kernel/drivers/keyboard.c	24	Scancodes, modifiers, delivery to blocked readers
kernel/drivers/ata.c	30	Disk read and write, PIO + LBA
memory/pmm.c	25	Frame bitmap: allocate, free, protect
memory/vmm.c	26–28	Paging: map, unmap, translate, fault handler
memory/vmm.asm	26–27	CR0, CR2, CR3, TLB
memory/heap.c	29	kmalloc / kfree
tasks/procman.c	30–32	Process creation, destruction, TSS
tasks/procman.asm	32	ltr
tasks/context.c	31	Save and restore context, switch
tasks/scheduler.c	31	Round robin, quantum
tasks/loader.c	33	Read a binary and map it into a new process
filesystem/fs.c	31–32	Blocks, inodes, directories, read and write
filesystem/fs_manager.c	32	Path resolution, descriptors, ls/mkdir/rm
tools/mkfs.c	36	Host-side formatter and rootfs installer
user/libc/syscalls.asm	34	The int 0x80 bridge
user/libc/output.c	35	printf and friends
user/libc/memory.c	35	malloc on top of sbrk
user/programs/shell.c	37	The command interpreter

CHAPTER F

Notes on the exercises

These are not full solutions — they are the shape of the answer, so you can check whether your reasoning went the right way.

1.1 Why 512 bytes cannot hold an OS

512 bytes is roughly 150 machine instructions. Enough to print a message and read from disk, and nothing else. The minimum it must do is: locate more code on the disk, load it into memory, and jump to it. Everything else is optional.

2.1 Why divide by 32 and take the remainder

The bitmap is an array of 32-bit words. $i / 32$ selects the word holding frame i ; $i \% 32$ selects the bit within that word. Together they turn a flat bit index into a (word, bit) pair.

3.1 Why CR3 holds a physical address

If it held a virtual address, translating it would require reading the page directory — which you cannot find without translating CR3 first. Infinite regress. The chain has to start somewhere untranslated, and CR3 is that starting point.

4.1 Why the boot sector loads at 0x7C00

Below 0x500 is the interrupt vector table and BIOS data. 0x7C00 is 32 KB — 1024: as high as possible in the 32 KB of the original PC while leaving the last kilobyte free. It also leaves a usable gap above it, which is where this project puts the kernel.

12.1 Why print_string uses SI

`lodsrb` implicitly reads from DS:SI and increments SI. It only works with that register. Using BX would require a separate load and an explicit increment.

16.1 cs = 0x18 without the RPL

The processor checks that the RPL is no more privileged than the descriptor's DPL, and `iret` will not lower privilege unless the target CS has RPL 3. With `cs = 0x18` (RPL 0) pointing at a DPL 3 descriptor, the `iret` raises a general protection fault instead of entering ring 3.

25.1 Is the E820 buffer at 0x5000 a bug?

No, though it looks like one. The region is marked usable and never explicitly protected, so `alloc_frame` can hand it out. But the map is read exactly once, inside `init_pmm`, which runs before any allocation happens. After that the data is dead. It is safe by ordering rather than by design — which is worth a comment in the source.

26.1 Splitting 0xC0000000

$0xC0000000 \gg 22 = 768$. Table index and offset are both 0. So the kernel heap starts exactly at the beginning of directory entry 768, meaning it occupies whole 4 MB directory slots from there up. That alignment is why the constant is 0xC0000000 and not some nearby value: it keeps the kernel heap from sharing a directory entry with anything else.

7.1 Which calls `cat file.txt` needs

`open` (0x09) to get a descriptor, then `read` (0x06) in a loop, writing each chunk with `write` (0x05) to descriptor 1, then `close` (0x0A). Four of the ten system calls, which is a good illustration of how little a useful program actually needs.

20.1 `vga_text_write_hex`

Loop from the most significant nibble down: shift right by $4i$ for $i = 7 \dots 0$, mask with 0xF, and convert with $d < 10 ? '0' + d : 'A' + d - 10$. Print "0x" first. The difference from decimal is that hexadecimal has a fixed width, so you print leading zeros rather than suppressing them — which also means no reversal buffer is needed.

CHAPTER G

The runnable examples

Every recipe box in this book points at a folder under `doc/claudedoc/examples/`. Each folder holds a small self-contained program plus a Makefile that builds a complete bootable image and starts QEMU.

Folder	Ch.	Demonstrates
ch12-bootloader	12	Editing the 512-byte boot sector and printing via the BIOS
ch16-gdt	16	Reading the live GDT with <code>SGDT</code> ; the Accessed and Busy bits
ch18-vga-output	18	Every text-driver operation, and the <code>write_dec(0)</code> bug
ch21-custom-interrupt	21	Adding your own IDT vector, in all three layers
ch23-timer	23	Timing work, and what changing the PIT frequency really changes
ch24-keyboard-input	24	Reading a line correctly, unlike the shell
ch25-physical-memory	25	<code>alloc_frame</code> / <code>free_frame</code> and first-fit reuse
ch27-paging	27	Map, alias, translate, unmap, and build an address space
ch28-page-fault	28	Touch an unmapped address and read the report
ch29-heap	29	<code>kmalloc/kfree</code> with the block list printed each step
ch30-kernel-process	30	Three kernel processes preempted by the timer
ch33-loader	33	Loading a program from the kernel, with no shell
ch34-new-syscall	34	Adding a system call: the four files you must touch
ch35-libc	35	A user-program template using the whole libc
ch37-ata-sectors	37	Raw sector read and write, and finding the superblock
ch39-filesystem	39	Create, write, reopen, read, EOF, list, delete
ch40-shell-command	40	Adding <code>echo</code> and <code>rev</code> to the shell
ch45-reboot-loop	45	Reproducing the reboot loop, and tracing it

G.1 Running one

```
cd doc/claudedoc/examples/ch27-paging
make run      # build and boot it in a QEMU window
make shell    # boot headless and print the text screen (works over ssh)
make trace    # boot with interrupt logging, stop at a triple fault
make diff     # show exactly what this example changes
make clean    # delete this example's build directory
```

`make trace` is the one to reach for when something crashes: it is the technique from Chapter 42 wired into every example.

The project is never modified

Each example copies the OS tree into its own `build/os/`, overlays its files on the copy, builds that and boots it. `lytlnybl/` is left untouched, and deleting `build/` leaves nothing behind. You can break an example as thoroughly as you like.

G.2 How an example changes the system

Two mechanisms, chosen per example.

Overlay replaces a whole file, and is what most examples use — nearly always `kernel_main.c`, since that is where the kernel's own demonstration code lives:

```
DESC      = Chapter 27 -- mapping, aliasing, translating and unmapping pages
OVERLAY   = kernel_main.c:kernel/kernel_main.c

include ../common/common.mk
```

Patches insert into existing files, for changes that touch several places at once:

```
DESC      = Chapter 34 -- adding a system call, end to end (4 files)
OVERLAY   = test.c:user/programs/test.c
PATCHES  = patch.py
SCREEN_ARGS = --wait 10 --type "run /bin/user_test\n" --settle 4

include ../common/common.mk
```

Every patch is anchored on exact existing text and *fails loudly* if that text is missing or appears more than once, so an example can never seem to apply and then silently do nothing. Read [ch34-new-syscall/patch.py](#) as the recipe itself: its four edits are the four edits adding a system call always requires.

G.3 Checking they still build

```
cd doc/claudedoc/examples && ./verify.sh
```

Builds all eighteen from scratch and reports. Worth running after changing anything in the OS an example overlays, since an overlay can go stale if the file it replaces gains something the example does not know about.

What make shell needs

It boots QEMU with no display, asks the monitor for a screenshot, and matches each 9×16 cell against the VGA font it finds inside the `vgabios` ROM that ships with QEMU — recovering the screen as text. It therefore needs Python 3 and that ROM

(`/usr/share/seabios/vgabios.bin` on Debian and Ubuntu). If it cannot find one it says so, and `make run` works regardless.